



TM1115

Basics of Machine-Centric Robotics

Requirements

Completed trainings	SEM210 Automation Studio training: Basics SEM410 Integrated motion control: mapp Axis
Software	Automation Studio V6.0 (and higher) Automation Runtime 6.0.2 (and higher) mapp Motion 6.0.0 (and higher) or mapp Motion Motion XT 6.0.0 (and higher) mapp Cockpit 6.0.0 (and higher) mapp View 6.0.0 (and higher) Scene Viewer 6.0.0 (and higher)
Licenses (for hardware only)	1TGMPROBCNC.10-01 (mapp Robotics / CNC "Premium", 4+ axes) or 1TGMPROBCNC.20-01 (mapp Robotics / CNC "Ultimate", 4+ axes) or 1TCMPROBCNC.10-01 (mapp Robotics / CNC "Premium", 4 axes) or 1TCMPROBCNC.20-01 (mapp Robotics / CNC "Ultimate", 4 axes)
Hardware	ARsim (all exercises possible) or Training hardware

1 Introduction

Robots are becoming more and more important in modern machinery and do not need to be at the center of the machine software, but more an integral part of it. Machine-Centric Robotics (MCR) offers a wide range of software functions together with a large selection of robot types to accomplish robot tasks in parallel to a standard machine control. MCR makes it possible to seamlessly integrate all the necessary components, including robotics.

The motors for the robot are directly controlled by B&R drives, which enables highly precise motion control synchronization via POWERLINK between the robot and other machine parts, e.g. individual axes for conveyors or even track systems.

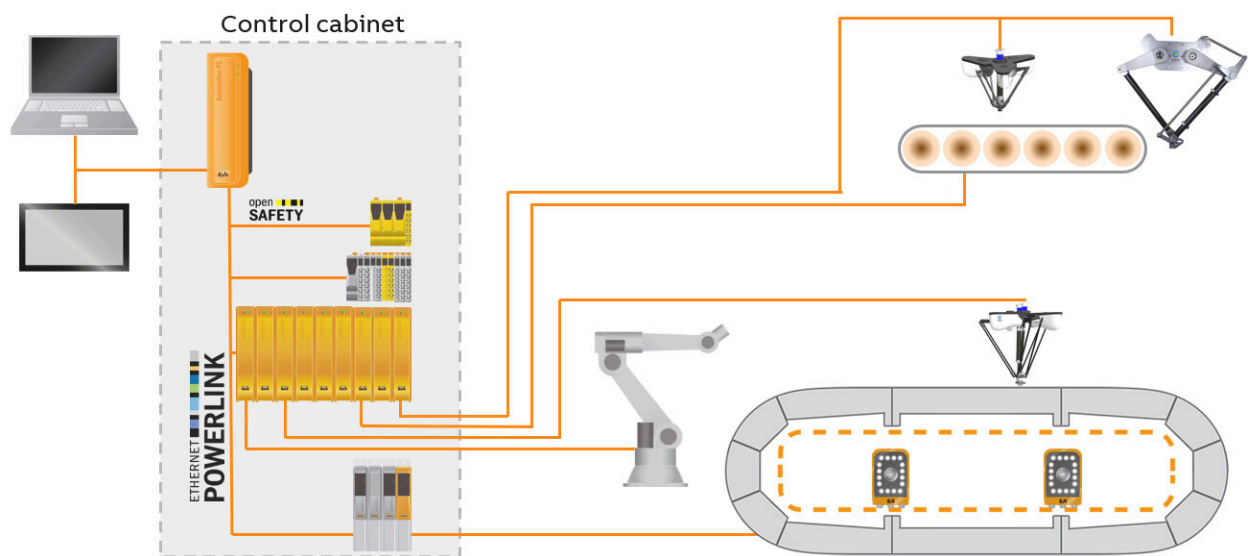


Figure 1: Seamless integration

Since the drives are directly connected to the robot, the control software for the robot, and with it the entire machine software, can be developed in Automation Studio. This simplifies development and opens up efficient diagnostics of the entire machine.

In Automation Studio, the robot is represented by a group of drive systems, one drive for each articulation of the robot. It is therefore possible to simulate the entire machine, including the robotic system. Additionally, the positions of each motor can be shown in the Scene Viewer simulation tool. Scene Viewer includes the robots mechanics, which allow simulation of robotic movements when used in connection with updated motor position values. Using simulation, machine functions and process can be validated without hardware, e.g. evaluating plausibility during conceptual planning or verifying machine code and optimizing the value chain.

Commissioning of the robot is done with mapp Cockpit. mapp Cockpit comes with a wide range of functions to control the robot during commissioning or diagnosis. Robot motion commands like movements from one point to another can be triggered and live robot values can be observed.

This training module introduces the concept and features of Machine-Centric Robotics. Participants will learn how to configure a robot, control it via mapp Cockpit and show the movement with Scene Viewer. Furthermore, they will also take a closer look at robotic commands and their parameters as well as create robotic programs in Structured Text (ST). How Automation Studio is working with frames and coordinate systems and the different functions provided is shown. How to implement geometric data from a real robotic scene, how to use tools, how to synchronize the application with the robotic program and how the system is managing everything in the background is also presented. Last but not least, the training module covers some important information about commissioning.

The training module contains exercises to help participants understand the basics of Machine-Centric Robotics. Automation Help provides support and delivers more detailed descriptions of the elements used.

1.1 Learning objectives

This training module covers all basic topics related to Machine-Centric Robotics (MCR). It introduces the concept and all the tools involved. It also includes several exercises to help participants understand the most important ideas and gain practical experience. Information about commissioning and diagnostics is presented together with links to the Automation Studio help documentation, providing further insight into each topic.

After working through this training manual and all given exercises you will be able to implement and make use of B&R's Machine-CentricRobotics solution. This means you will be able to:

- Explain the concept of MCR and identify all necessary hard- and software components
- Choose the necessary license according to your demands
- Configure a robot in Automation Studio using MCR assistant and mapp Robotics technology package
- Control a robot with mapp Robotics function blocks in Structured Text programs in simulation
- Define and use different coordinate systems using frame hierarchy
- Define and configure tools and use it in your robotics application
- Apply workspace monitoring to your robot
- Explain different types of movements and use them in an application
- Compare start and restart modes for robotics programs and test them on your simulation
- Describe the commissioning steps of a robot and know about various precautions to be taken during commissioning
- Create a digital twin in Scene Viewer and link it to your application
- Synchronize a machine application with your robotics program
- Diagnose a robotic system with different B&R tools and evaluate and handle errors

1.2 Symbols and safety notices

Safety notices in this manual are organized as follows:



Danger: Disregarding these safety guidelines and notices can result in severe injury, death or substantial damage to property.



Warning: Disregarding these safety guidelines and notices can result in severe injury or substantial damage to property.



Caution: Disregarding these safety guidelines and notices can result in injury or damage to property. These instructions are important for avoiding malfunctions.

Additional notices and information in this manual are organized as follows:



Note: Provides important tips and additional information.



Help: References additional documentation. (Automation Help, data sheets, user's manuals)

Example:

Hardware \ Motion control \ <Device>¹ \ Technical data \ (<Type>)² \ Status indicators



Example: An example illustrates the topic in greater depth.

¹ Angle brackets indicate variable placeholders "<...>"

² Parentheses indicate optional entries "(...)"



Result: The result of a completed task is summarized briefly.

Exercise: Tasks and exercises

Sections marked with an orange stripe on the left side contain information about exercises as well as the associated actions to be taken. The exercises are designed to provide a deeper understanding of the information provided.

2 Concept and features

2.1 Concept of Machine-Centric Robotics

Machine automation and robotics are still largely two isolated ecosystems. MCR unites the two into a single, uniform solution.



Figure 2: Machine with Codian manipulator

With MCR, it is possible to control robots and all the rest of the elements in the machine from one single controller. There is no need for a special robot controller anymore and, moreover, its tasks are taken over by a B&R PLC. Having the possibility to use B&R drives with robot mechanics enables many benefits as the whole control structure is handled using B&R products.

The MCR approach sees robots and machine as one. The robot becomes an integral component of the machine and thus complexity is reduced dramatically. Machine-Centric Robotics is part of the adaptive machine concept, i.e. one part of a big ecosystem that includes the control of all kind of robots, even 3rd party robots.

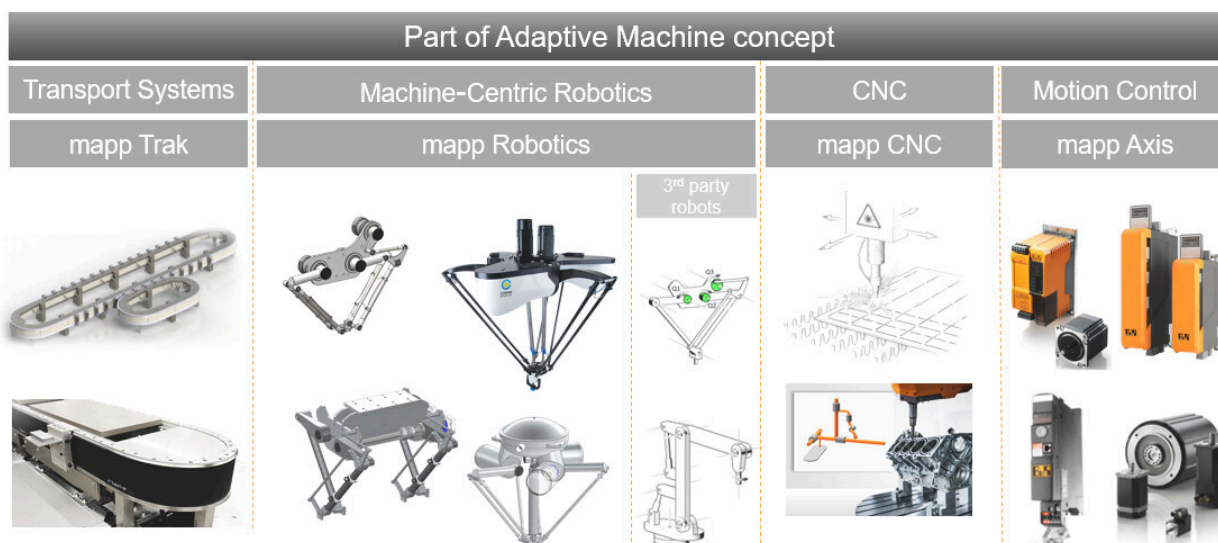


Figure 3: MCR inside mapp Motion

MCR, on the one hand side, supports all kind of manipulators. Pre-defined configuration templates such as ready-made mechatronic designs of a wide range of robots help reduce the complexity of projects as well as the development time. On the other hand, mapp Robotics software solutions include many functions to help develop robotic applications with efficient and versatile control systems, such as tools, frames, feed-forward control and so on. mapp Robotics is seamlessly integrated in the mapp software platform where solutions for other common machine functions are included as ready-to-use packages. Some of these functions, such as MpAlarmX, can work together with robotics applications and can be easily linked inside the mapp framework.

To avoid sacrificing flexibility, the functions are divided into three levels. The core level supplies all fundamental software configurations and hardware elements. Programming the core level takes place using PLCOpen function blocks, which enables modularity and flexibility. The technology level brings predefined configurations (general robot systems), ready-made mechatronic designs for Codian robots and technology function blocks to control them. The third level, the process level, reduces complexity by using ready-made solutions for application processes such as Pick-and-Place.

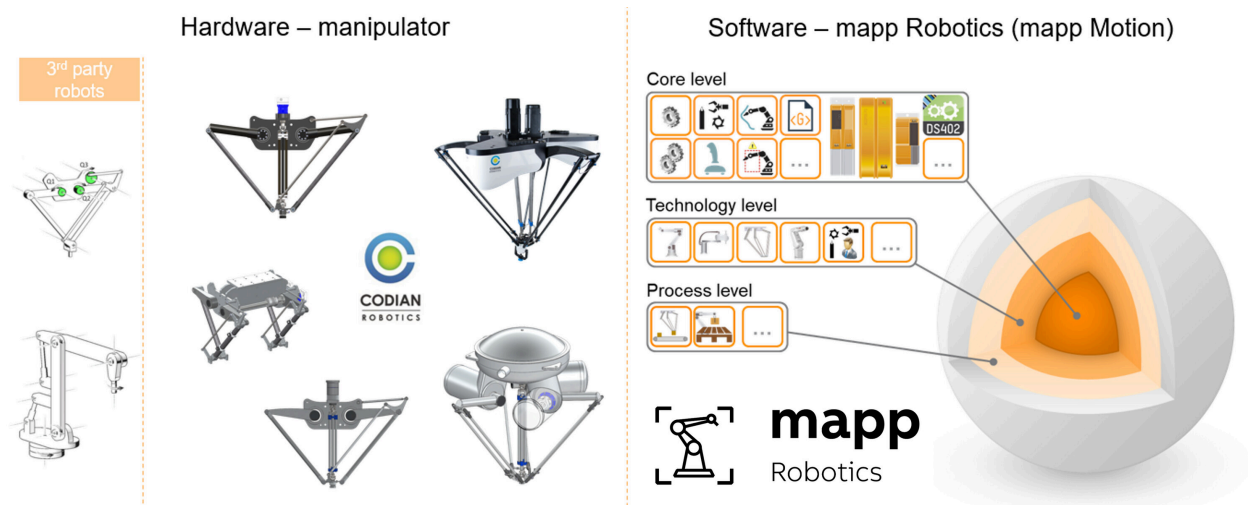


Figure 4: MCR components

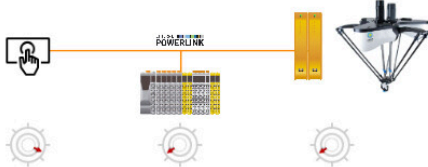
2.2 Advantages of Machine-Centric Robotics

Productivity

The robot is an integral part of the machine automation solution, communicating via the deterministic network POWERLINK. This network enables highly precise synchronization between the robot and other automation components (PLCs, motion axes, track systems or vision cameras) with a default cycle time of 400 μ s and a jitter significantly far below 1 μ s. These perfectly orchestrated components pave the way for high-speed productivity because, for example, a pick-and-place application requires deterministic communication to sensors such as a vision system. Optimal synchronization of movements and processing sequences boosts overall machine productivity.

Highly precise synchronization of all movements

Between robots and machine axes
 Between robots and track systems
 With sensors (e.g. machine vision)
 Cycle times down to 400 μ s
 $< 1 \mu$ s jitter



Increased productivity

One uniform engineering environment

The entire machine can be programmed with Automation Studio, so there is no need for any other development tool or special robotics software. In such a uniform environment, efficient diagnostic tools for the entire machine are available. The ability to implement the entire application in this way reduces complexity and the commissioning phase is streamlined. Using a single development environment for the entire machine reduces overall development effort.

A universal development environment

One development tool for the entire machine
 Machine developers work with a familiar platform
 No specialized robotics programming languages required



Integrated simulation

With Automation Studio, it's possible to simulate the entire machine, including robotics. Visualization tools like B&R Scene Viewer can easily be connected to the controller via OPC UA. Models of manipulators from the B&R robot portfolio are available in Scene Viewer. A large portion of machine development can be completed without any hardware. Simulation tools accelerate machine development and commissioning. This in turn lowers development costs and time-to-market.

Uniform user interface

With MCR, you no longer need two different user interfaces to control the robot and the machine. The entire machine is operated from one, uniform user interface. The application can run on an Automation Panel or a Mobile Panel, and you can even use both, always with the same user interface, which maximizes flexibility, minimizes complexity and reduces hardware.

Uniform user interface for entire machine



2.3 Robot portfolio

Machine-Centric Robotics offers machine builders an extensive selection of robots. B&R has access to an entire portfolio of Codian robots.

Codian is focusing on pick-and-place delta robots and offers machine builders the possibility to choose from a wide range of options like different arm lengths, hygienic designs or even two parallel axes for rotating and tilting.

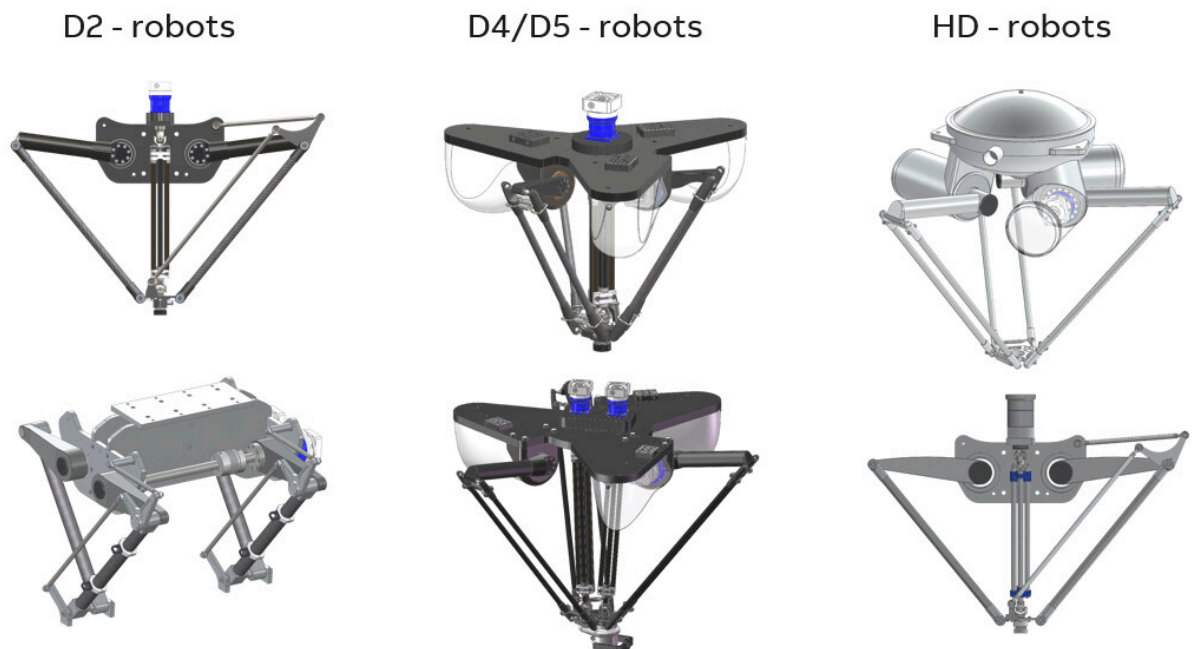


Figure 5: Robot portfolio - General

Different types of robots

- SCARA robots
- 6-axis articulated robots
- Delta robots

These types of robots are available in a broad range of payloads as well as different arm lengths or reaches. All together, they create a robot portfolio to cover the most common robotic tasks in current machines, such as pick-and-place, tending and palletizing.

open Robotics

open Robotics is part of MCR and offers mechatronic designs containing all necessary parameters for smooth operation of open Robotics systems such as Comau robots.

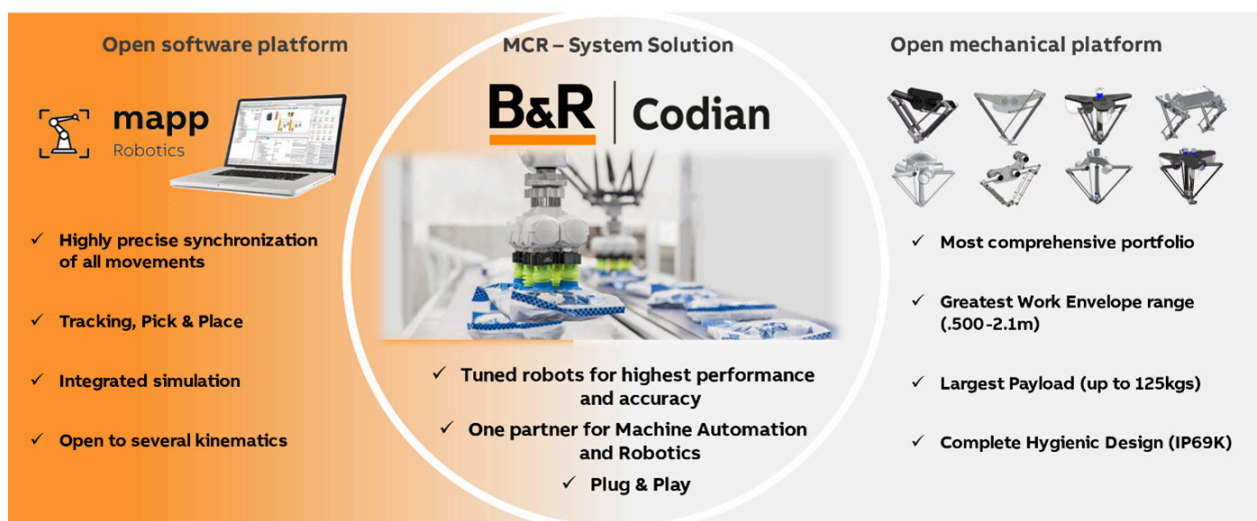
3rd party robots

The concept of Machine-Centric Robotics includes the possibility to create any kind of mechanics and, therefore, enables the control of all kinds of robots. Machine builders who created their own kinematics, for example, can easily configure them in Automation Studio and benefit from all the functionalities offered by mapp Robotics.

3 Hardware and software components

The following sections address the hardware and software components used to control the robot. These components are divided into two platforms. The Open Software Platform provides all functionalities to describe and control any robot within the software. The Open Mechanical Platform offers software-independent robotic mechanisms that can also be controlled using third-party software. The MCR-System Solution connects these two platforms and offers the possibility to use pre-configured and optimized robots.

Machine-Centric Robotics Innovative Robotics Solution



In the following chapters the components are described as individual elements to help understand their functionality. When using the MCR assistant (when inserting a specific robot) these components are created automatically. The assistant will be described in subsequent chapters.

3.1 Terminology

Global coordinate system (GCS)

The Global Coordinate System is a fixed, absolute reference frame that serves as the foundation for defining all other coordinate systems in a machine. It provides a common spatial reference, allowing multiple robots or components to be positioned and oriented relative to one another.

In practice, the Global Coordinate System may be defined relative to a fixed point in the machinery room, for example. In simple setups with only one robot system, the Global Coordinate System may coincide with the Machine Coordinate System (MCS) and the Base Coordinate System (BCS)—as it is the case in the initial state after completing the "Getting Started" tutorial.

Machine frame / Machine coordinate system (MCS)

The Machine Coordinate System defines the origin and orientation of a robot or its axis group within the physical environment. It typically represents a fixed reference point—such as the position where the robot is mounted relative to the machinery room or production cell layout.

The MCS is configured in the object hierarchy and serves as the common reference for all Cartesian coordinate systems used by the robot.

Robot base / Base coordinate system (BCS)

Based on the GCS or a MCS, the base coordinate system represents the viewpoint of the robotics system. In most cases, the zero point of this coordinate system cannot be approached because this would result in self-collision.

Joints

In the context of independent axis positioning, the term "axis" is used to refer to a motor that actuates a mechanical system. In the field of robotics, the particular focus is on joints. There is a differentiation between an axis (motor that moves a joint) and a joint axis (axis that moves a robot arm).

It is possible for one axis to simultaneously influence two or more joints. This behavior is represented as coupling in the mechanical system.

Kinematic chain

When describing a robot, the kinematic chain is the mechanical relationship between all its links, i.e. the union between joints from its base to its flange where the tools are mounted. The serial kinematic chains (e.g. a 6-axis robot) describe a structure where all links are connected one after the other. In contrast, a parallel kinematic system (a delta robot, for example) involves multiple joints working together to support a single platform.

Transformation

The tool center point position (TCP, described in Basic coordinate systems) is calculated based on the positions of the joints by performing a forward or direct transformation. An inverse transformation can be used to calculate the joint positions when the TCP position is known.

File Device

A file device is a predefined and configurable storage interface that maps to a specific path in the Automation Runtime file system. It allows access to designated directories—such as a user partition or USB stick—where files like robotic ST programs can be stored and accessed.

File devices are defined in the PLC Configuration.

Robotic ST Program

A Robotic ST Program is a source file written in Robotic Structured Text, with the file extension .st. It defines the robot's movements and actions through structured commands. These files are stored and loaded via a File Device, such as a user partition or USB stick.

One key advantage is that the program can be modified during runtime without the need to rebuild the project, allowing for flexible and dynamic adjustments.

An example command is MoveL(P1), which instructs the robot to perform a linear movement to the position P1.

3.2 Software

In this chapter, the basic software components for robotic applications are described. When using the MCR assistant, the basic components are added automatically. The linking between the components via references is also established. Thanks to a modular configuration concept, the components can be easily modified and extended.

3.2.1 Object hierarchy

The object hierarchy defines the Global Coordinate System for motion components such as robots. It is also used to define the global units of measurement and to set the order of rotation angles in the system.

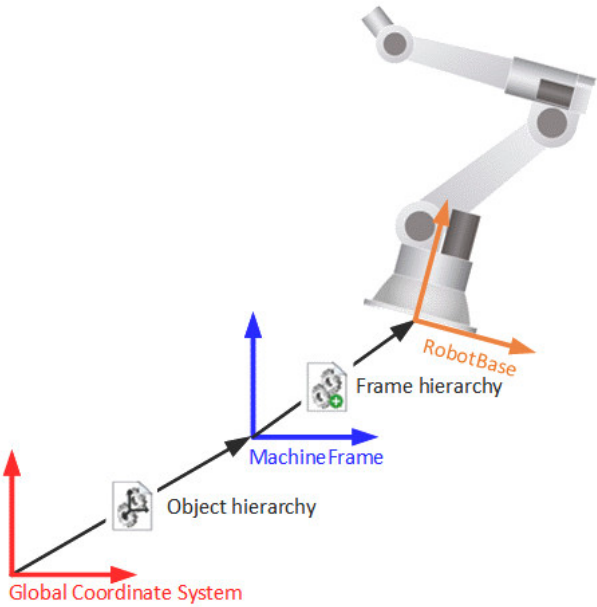


Figure 6: Object hierarchy - Global coordinate system to machine frame

The position of the machine frame is referenced to the global coordinate system using the following parameters (the order of angles 1, 2 and 3 is defined in "Rotation order"):

- Translation in X, Y and Z
- Orientation of angles 1, 2 and 3

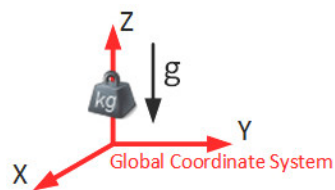
	value	unit	Description
Object-Hierarchy			
Measurement Units			Defines the measurement units
Length Measurement Unit	Millimeters		Defines the measurement unit for length
Angle Measurement Unit	Degrees		Defines the measurement unit for angles
Rotation description	Cardan		Defines the description of the rotation
Rotation order	XYZ		Order of rotations
Angle interpretation	Intrinsic		Interpretation of angles
Rotation matrix interpretation	Direct		Interpretation of angles for rotation matrix
Mathematical sense	Positive		Mathematical sense of angles
Frame Transformations			Defines frame transformations properties
Scene Viewer			Defines Scene Viewer export settings
File device	FileDeviceSceneViewer		File device where the Object Hierarchy Scene Viewer file will be created
Global Coordinate System			Systemwide reference frame
gAxesGroup_D4r1100R_1			
Type	Axesgroup component		Object type
Axesgroup reference	gAxesGroup_D4r1100R_1		
Translation			Translation parameters
X	400	measurement...	Translation in X direction
Y	0.0	measurement...	Translation in Y direction
Z	1350	measurement...	Translation in Z direction
Orientation			Orientation parameters
Angle 1	0.0	measurement...	Rotation around the first coordinate axis of the rotation order
Angle 2	0.0	measurement...	Rotation around the second coordinate axis of the rotation order
Angle 3	0.0	measurement...	Rotation around the third coordinate axis of the rotation order
Type	Axesgroup component		Object type
Type	Axesgroup component		Object type
Type	Axesgroup component		Object type

Figure 7: Object hierarchy configuration file

If there are several robots in the project, another object can be created.

Effective direction of gravitational force

Gravitational force is defined as the negative direction of the Z-axis in the global coordinate system. If a robot is rotated around a Cartesian axis by setting orientation, the gravitational force is still pointing in the negative direction of the Z-axis and taken into account for further calculations.



Further information



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Basic elements \ Object hierarchy](#)

3.2.2 Axis group

3.2.2.1 Axis group

An axis group brings together multiple axes to simplify the application program. A path generator can be used to perform a path-controlled movement with the axes in a group. Path-controlled movements are required in order to implement CNC and robotics applications.

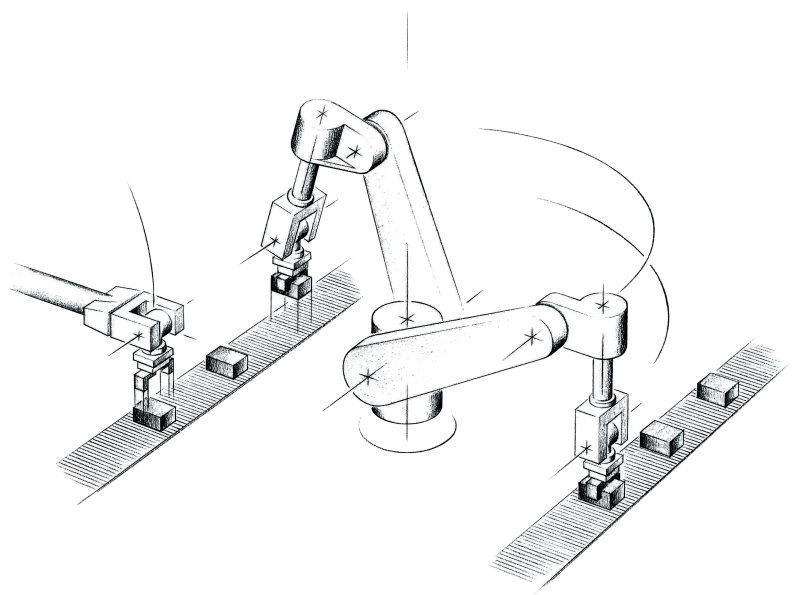


Figure 8: Robotics application



The path generator will be described later in more detail.

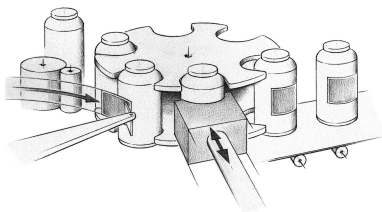


Figure 9: Labeling machines

Scalability+

For many applications it is necessary to create axis groups so that multiple axes can be switched on with just one command or to simplify error handling for all the axes in the group.

In most cases, only selected axes in the axis group need to be operated via centrally generated setpoints in order to implement path-controlled movements.

Axes within the axis group that are not included in the path planning can be moved with single-axis commands independently of path-controlled movements.

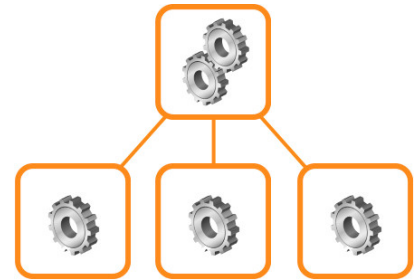
The number of axes contained in an axis group does not affect the complexity of the program. The methods used for control and operation with mapp Motion technology or core function blocks do not change.

Axis types

An axis group may include the following types of axes:

Path-controlled axes

- **Joint axes**
Joint axes are controlled by the axis group. The way in which the joint axes are combined produces the path in space.
- **Slave axes**
Like joint axes, slave axes are controlled by the axis group. However, they do not contribute to the path in space. If slave axes are programmed together with joint axes, the movements of the slave axes are planned so that they reach the target position at the same time as the joint axes.

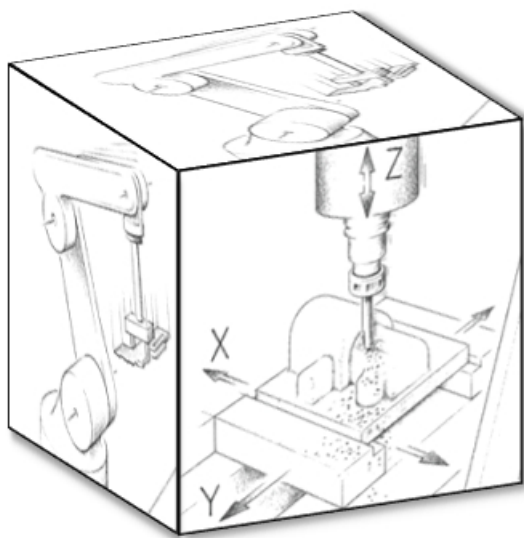


Non-path-controlled axes

- **Single axes**
Movement of these axes is triggered using single-axis commands and is not dependent on axis group movements.

A path-controlled axis group is a group of multiple axes that can be used to perform common interpolated movements. This is essential for CNC and robotics applications where path-controlled movements form the basis for precise and complex manufacturing processes.

Path control allows axes to be controlled simultaneously in space. In addition to the position in space, it is also possible to control the orientation in space.



The programmed path is generally composed of straight lines and arcs. Straight segments are defined by setting a new endpoint. Arcs also require information about the position of the center of the circle and its radius as well as the required direction of rotation.

The code for the path movement is either written in G-code or Structured Text motion commands (according to IEC 61131-3). The B&R path control solution can be adapted flexibly to accommodate applications ranging from a basic 2-axis system to a 6-axis robot.

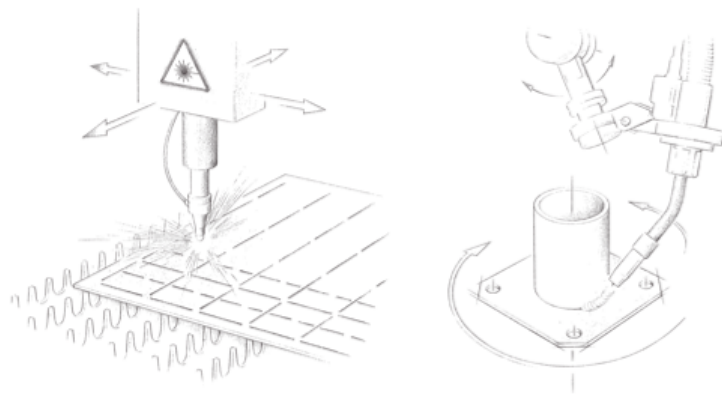


Figure 10: 2-axis CNC and robot welding application

Path-controlled movements are based on the path calculation functionality. It generates cyclic position setpoints for the axis group's path-controlled axes from the motion command.

Predefined mechatronic designs are provided for CNC machines and robotics applications to make the configuration process as easy as possible.



Figure 11: Division of mechatronic designs into mapp Axis, mapp CNC, mapp Robotics

A more detailed description of how axis groups can be used is provided in the Automation Help:



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Concept \ Components \ Axis group](#)
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Libraries \ Core libraies \ McAx-Group](#)

Robots as axis groups

Inside mapp Motion technology, robots are defined as axis groups. An axis group brings together multiple axes into a single new element to simplify programming and reduce development effort when dealing with machines that have multiple axes. This new axis group element has its own characteristics and properties that define the relationship between the axes inside it and how they must interact to complete different commands.

An axis group can also have some specific extra features that help when developing new common functionalities which several types of systems, like robots, usually need in their applications (see axis group features). Each axis group has a specific interface that allows the application to check its status and command it to perform different tasks. The system translates the commands given to the axis group to the proper instructions for each one of the single axis.

The concept of an axis group is very useful in robotics. The joints of the robot are the single axes inside of the axis group and the relationship between them is defined in the mechanical parameters and properties of the axis group. By using the axis group as the interface for the robot, it is then possible to command it to perform different tasks and all the single joints of the robot will be properly handled by the system itself.

Robots are path-controlled axis groups. Path control allows axes to be controlled simultaneously in space. A certain position and orientation of the robot in space is achieved by moving each one of its joints to a certain position. Path calculation functionality generates cyclic setpoints for all the path-controlled axes in the axis group to correctly achieve the desired robot position and follow the desired robot path.

3.2.3 mapp Motion

mapp Motion offers the axis group component for controlling CNC machines. This can vary and ranges from CNC systems in various numbers to robots with various designs.



Figure 12: mapp components for controlling axis groups

Automation Studio provides predefined function blocks for different numbers of axes. Descriptions can be found in the Automation Help. These function blocks provide a uniform operation and analysis, make the program easier to read and maintain – and also save programmers valuable time.



Figure 13: mapp RoboArm



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Libraries \ Technology libraries \ MpRobotics \ Function blocks](#)

[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Libraries \ Core libraies \ McAx-Group](#)

3.2.4 Mechanical system

The configuration of a mechanical system includes all the dimensions, rotations, angles, etc. that are needed to calculate its positioning. This includes everything from the base of the robot, along the lengths of all its joints, all the way to the flange (foremost end where the tool is mounted).

This way, the user does not need to worry about how a given movement results from the sum of the individual joints. This is yet another example of B&R's fully scalable products and the advantages of hardware independence. If a different mechanical layout is used, all that is needed is a new mechanical system configuration and you are ready to go. These values are used to calculate the TCP position from known joint angles (forward/direct transform) or the joint angles from a known TCP position (inverse transform).

Compatible mechanical systems can be found in the Automation Help.

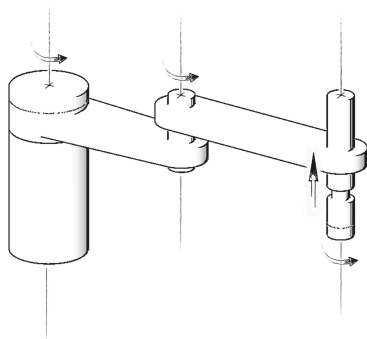


Figure 14: 4-axis SCARA

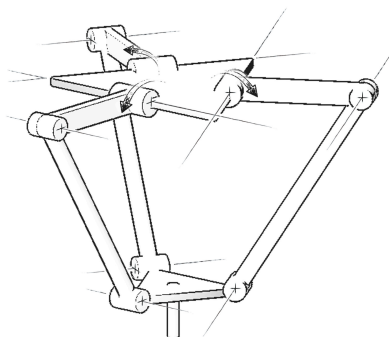


Figure 15: 3-axis delta

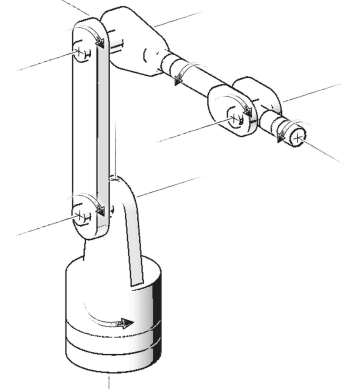


Figure 16: 6-axis RoboArm



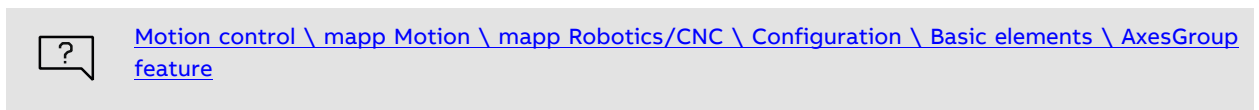
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Basic elements \ Mechanical system](#)

3.2.5 Axis group features

With axis group configuration files for features, it is possible to create different axis group features. These features can provide a solution for some common robotics functionalities such as defining the programming language for robotics, using different tools, creating a set of coordinate systems (frames), monitoring robot workspace, etc.



Figure 17: mapp Motion - Example of features



3.3 Hardware

Earlier systems were controlled by hydraulics and valves, which grew increasingly flexible over time. Later, it became possible to send commands, and the bus systems in today's solutions are able to transmit position information. Some manufacturers also offer fieldbus connections, which can be addressed via POWERLINK, for example.

When a new robotics unit is added to an existing line, it is still often connected via a defined I/O interface. In these cases, documentation is required to describe things like which program is started when a given input is enabled. These solutions tend to have numerous disadvantages.

- Inflexible to expand – require integration into design and controls
- Limited interface options – modifications are costly
- Sluggish runtime behavior due to signal processing via I/O
- Rigid programming and limited ability to intervene
- Generally not possible to synchronize with production process

B&R software solutions function independently of the hardware used. The connection between the two is established by the defined mechanical system. Scalability+ means that once a program has been written, it can be executed on all types of different hardware platforms. This ensures long-term stability and makes replacing robotic systems very easy. The advantages of integrated robotics include access to B&R's completely open product portfolio, which features HMI systems, countless X20 I/O modules, safety technology, process control technology and more.

A robot's mechanics generally include servo motors controlled via inverters. B&R offers a broad range of motion control products, and thanks to Scalability+, it makes no difference which specific hardware is used. Setpoints are calculated on the controller. Differences with regard to additional functionality can vary from device to device. (See figure with regard to compact design, power regeneration, supply voltage, etc.)

- ACOPOSmicro servo
- ACOPOS
- ACOPOS P3
- ACOPOSmulti
- etc.

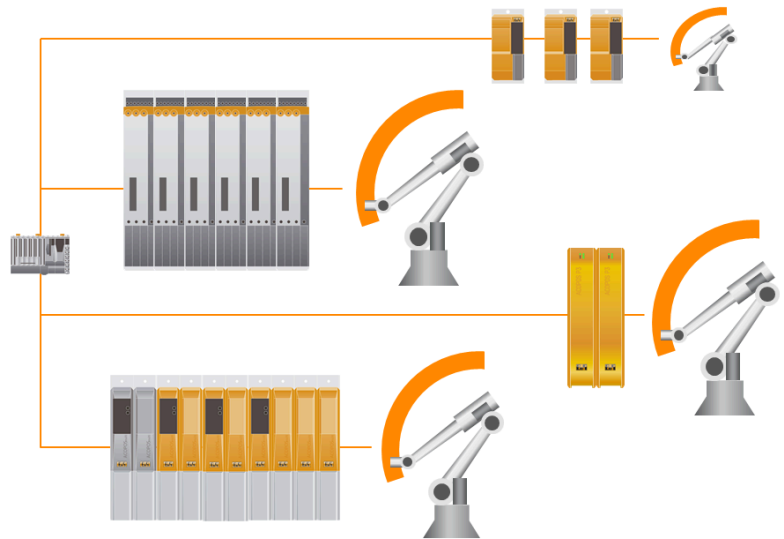


Figure 18: Diagram of a B&R ACOPOS servo family (combinations also possible)

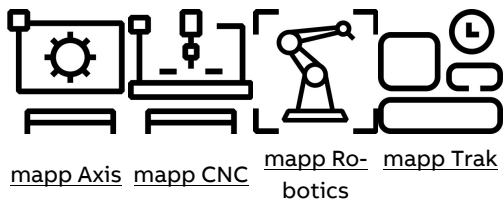
3.4 Mechatronic designs

Mechatronic design refers to a configuration that is composed of several individual elements and has certain physical properties. It is used to easily add complex machine or machine components such as CNC or robotics systems in Automation Studio.

For example, a mechatronic design for a 6-axis robot is added in Automation Studio, the associated axes are added automatically and all the necessary settings are made.



Classification of mechatronic designs



Further information



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Mechatronic designs](#)

3.4.1 Generic robots

This group describes mechatronic designs that can be used to configure any robot. The drive hardware is not added and parameterized automatically. When using generic robots, the following configuration elements are added:

- Axis group
- Axes (amount depending on robot type)
- Mechanical system (open)
- Axis group features

The axis group links to the individual axes are already configured and the mechanical system contains default values for the dimensions which can be changed as needed.

Available configuration packages

- SCARA
- Delta
- 3-axis robotic arm
- 4-axis robotic arm
- 5-axis robotic arm
- 6-axis robotic arm

Example - 6-axis robotic arm (A)

Mechatronic design - 6-axis robotic arm (A) is a predefined configuration package for a 6-axis robotic arm (A) mechanical system. It contains an axis group configuration with six physical axes, axis group features for starting CNC programs and jogging as well as the configuration of the mechanical system. Mechanical system "6-axis robotic arm (A)" is a serial kinematic chain consisting of a sequence of arm elements connected via 6 revolute joints. Its mechanical structure permits translational and rotational movement. As a result, there are 6 degrees of freedom for positioning a tool.

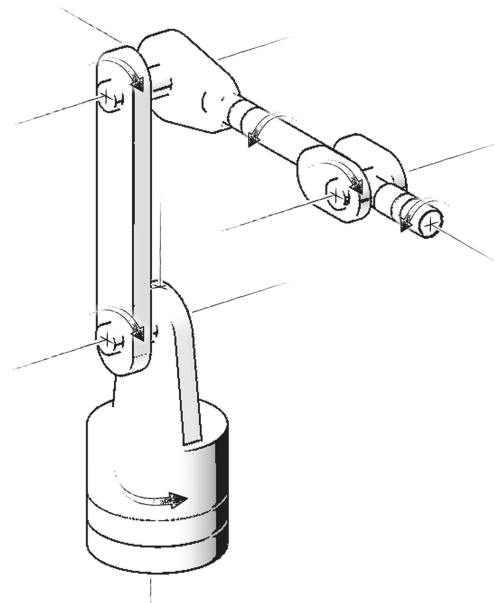
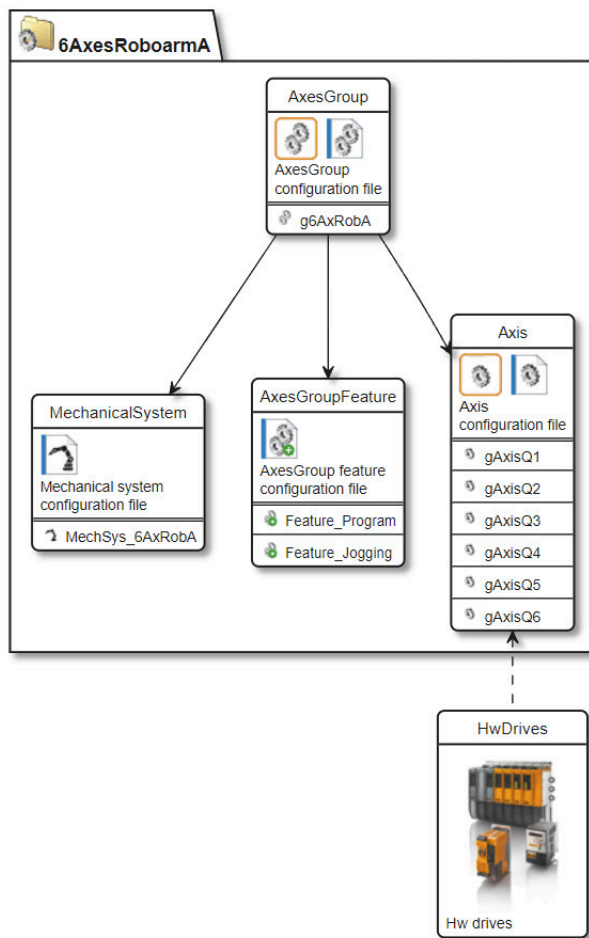


Figure 19: Generic robot - 6-axis robotic arm (A)



The following figure shows the link between the configuration elements, whereby the drive system including motors are not part of the package and, therefore, have to be added manually.

Figure 20: Links between configuration elements - 6-axis robotic arm (A)

Further information

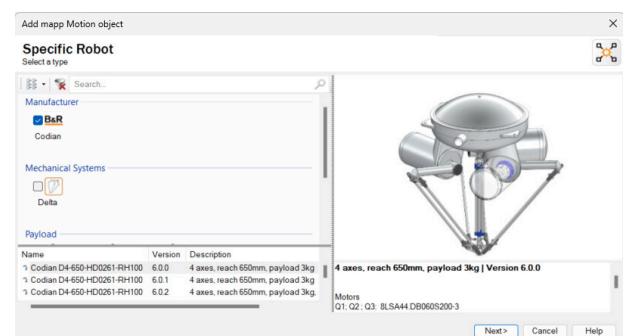


[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Mechatronic designs \ mapp Robotics](#)

3.4.2 Specific robots

B&R's Machine-Centric Robotics offers a broad range of so called "Specific robots". Adding a specific robot to the Configuration View from the Object Catalog opens the Specific robot assistant, which supports the selection of an Codian robot or robots inside open Robotics with associated ACOPOS drive hardware in just a few steps. After the assistant is completed, the drive hardware is added and the robot settings have been made. The development effort is minimized because the necessary drive hardware and the configuration parameters are added to the project automatically. The configuration of a Specific robot includes all necessary parameters for smooth operation of a robot. This includes not only the mechanical dimensions, but also a previously identified dynamic model of the robot and controller parameters adapted to the robot.

More robots can be added to Automation Studio via Upgrades sections.



Mechatronic design for Specific robots

As described, specific robots are added via the assistant which upon competition will automatically insert the following components.

- Axis group
- Axes (amount depending on robot type)
- Mechanical system (fixed)
- Axis group features
 - Programs
 - Jogging
 - Feed forward
- Axis feature
 - Shared brake signals (optional)
- ACOPOS drive system (preconfigured recommended hardware)
- Controlling elements in robot
 - Motor including motor parameters (optional if EnDat)

All these elements together with the mechatronic design, are inserted by the MCR assistant in the Automation Studio project.

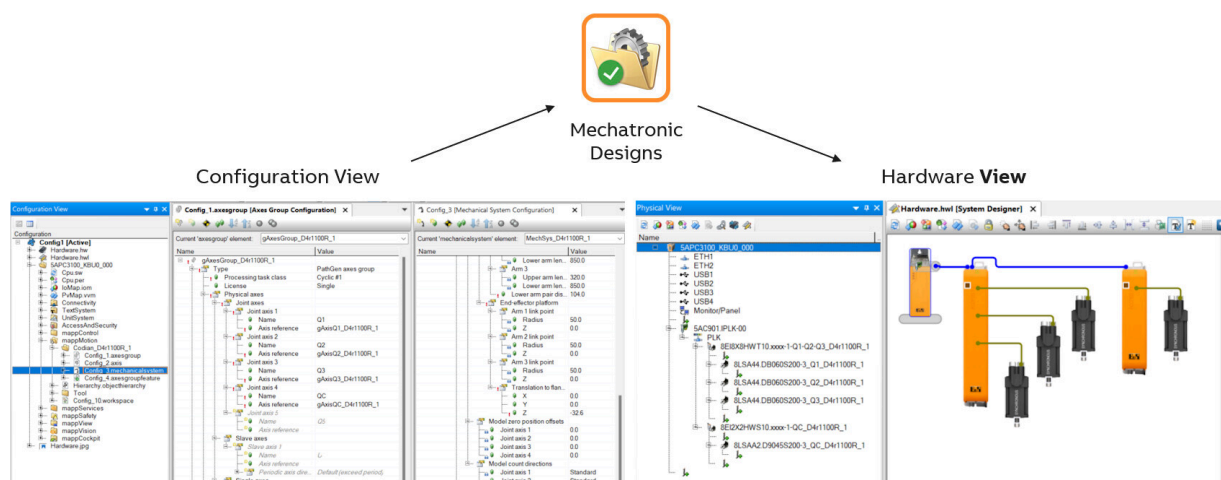


Figure 21: Mechatronic design

Links between configuration elements

Specific robots also include the drive system and motors to control the robot as stated in the figure. The package provides a previously identified dynamic model, which is set via the axis group feature Feed forward. Additionally, with some robot mechanics, to control the brakes of the robot, the axis feature "Shared brake signals" is created, which is mapped to each axis channel automatically. This depends if brake control is shared on the robot (one output control mores than one brake of the motors at once).

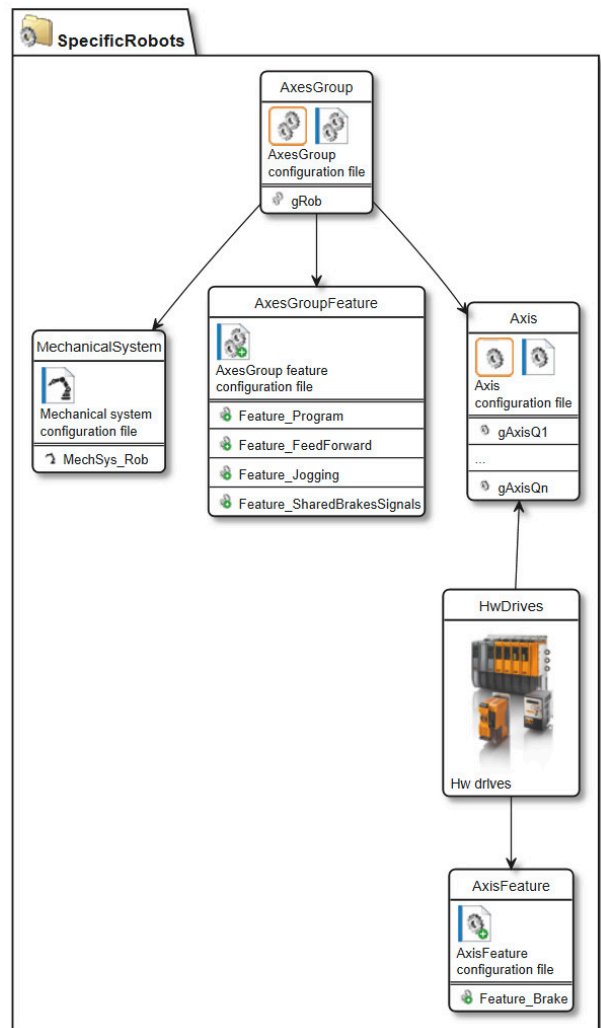


Figure 22: Specific robots - Links between configuration elements (Brake functionality is optionally)

Further information



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Mechatronic designs \ mapp Robotics \ Specific robots](#)

3.4.2.1 Codian robots

Topology

In the topology below, you can see the basic components to control a 5-axis delta robot, in this case a Codian D5.

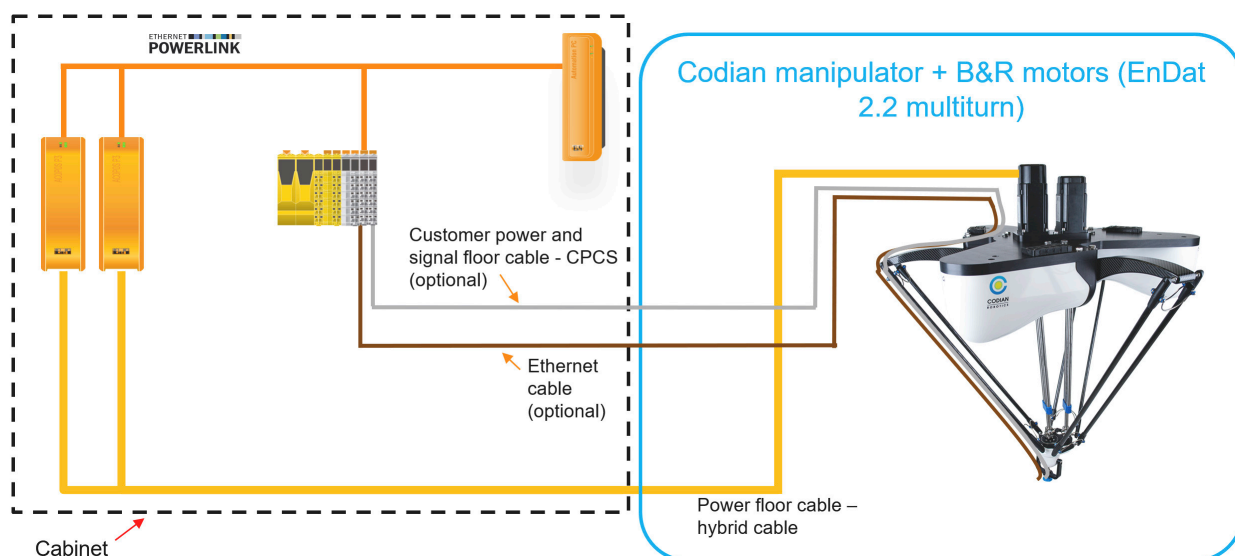


Figure 23: Topology - Codian D5

Codian and B&R components

- Codian D5 5-axis delta robot
 - B&R motors
 - Gearboxes
- B&R Automation PC
- B&R safety system
- B&R ACOPOS drive system

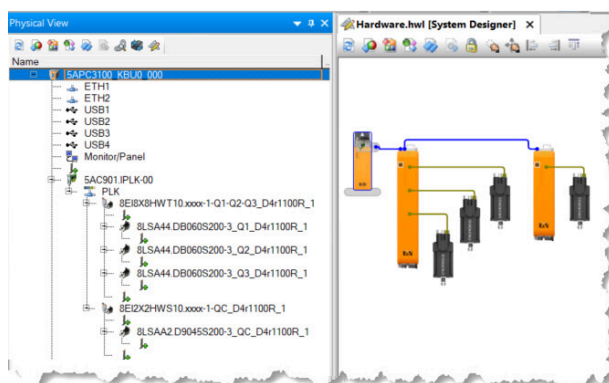


Figure 24: Hardware tree and System Designer - Codian D4

ACOPOS drive system

Predefined drive system settings

- Encoder interface
- Mechanical elements
- Controller parameters
- Homing mode (might need to be changed, see Homing modes for Codian delta robots)
- Movement error
- Jerk filter

Predefined motor configuration settings

- Motor parameters
- Brake
- Encoder
- Gearbox

Safety

As mentioned above, the motors being used allow SafeMOTION as well as the commonly known safety features like safe torque off (STO). Since the brakes are directly controlled by the ACOPOS drive, there is no need for a safety relay for the brakes.

3.4.2.2 open Robotics

With open Robotics, mapp Motion provides ready-made mechatronic designs made specifically for Comau robots. This includes not only the mechanical dimensions, but also a previously identified dynamic model of the robot and controller parameters adapted to the robot.

open ROBOTICS



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Mechatronic designs \ mapp Robotics \ Specific robots \ open Robotics](#)

3.5 License model

The table below shows the available licenses for mapp Robotics. For simulation, no license is required, but one of the licenses below is needed as soon as the axis group is running on a controller.

Seminar	Order number	Description	Status
SEM1115	1TGMPROBCNC.10-01	mapp Robotics / CNC "Premium", 4+ axes	Required
SEM1115	1TGMPROBCNC.20-01	mapp Robotics / CNC "Ultimate", 4+ axes	Optional
SEM1115	1TCMPROBCNC.10-01	mapp Robotics / CNC "Premium", 4 axes	Optional
SEM1115	1TCMPROBCNC.20-01	mapp Robotics / CNC "Ultimate", 4 axes	Optional

Table 1: Required licenses for mapp Robotics seminars

The information below helps finding the correct license. If further information is required, see the Automation Help link below.

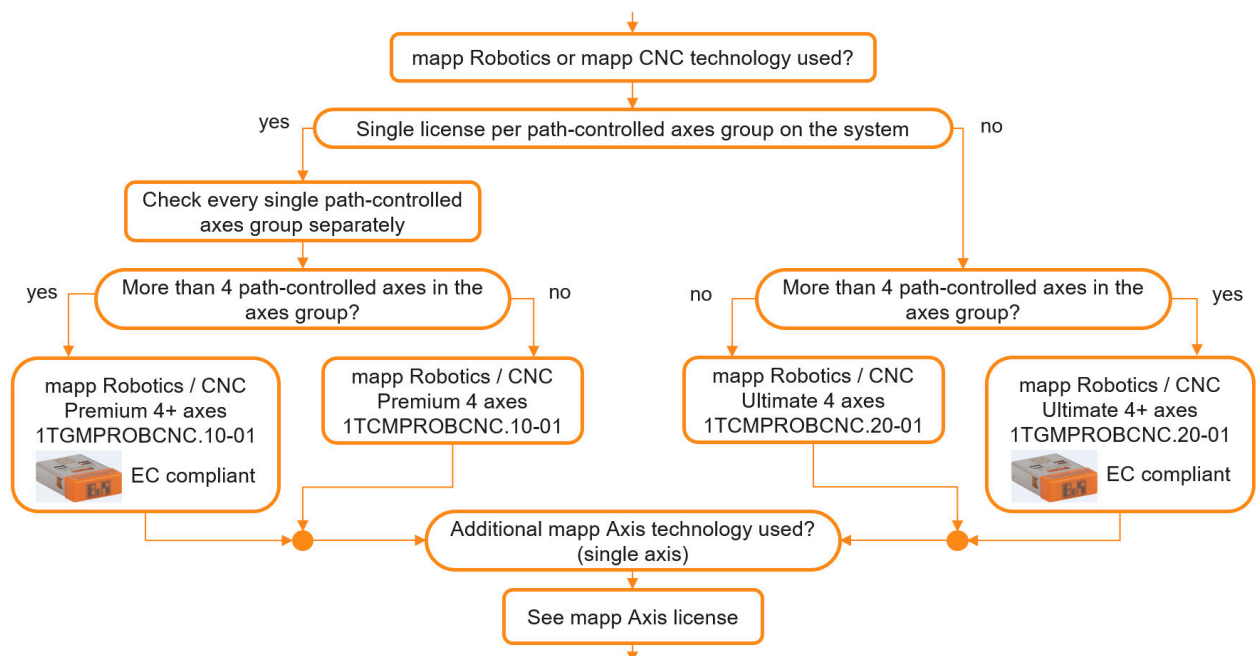


Figure 25: mapp Robotics - Licensing

Further information



[Motion control \ mapp Motion \ General information \ Licensing \ Licensing \ mapp Robotics examples](#)

4 First project

This chapter focuses on the concepts and tools needed to create the first Machine-Centric Robotics project. All the steps showing how to add a new robot to a project, connect it to a digital twin of the system and get it ready to perform some basic movements using mapp Cockpit are described in the next sections.



On the [B&R GitHub repository](#) you will find an Automation Studio project that provides a complete solution for all exercises of this training manual:
[MCR solution project](#)

4.1 Getting started

As a starting point, all the necessary configurations regarding how to start a project can be found in the "Getting started" page of Automation Help. This is a step-by-step guide to add a robot into an empty project using the framework of Machine-Centric Robotics.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Getting started \ MCR Codian Delta robots](#)

4.2 Machine-Centric Robotics assistant

As previously mentioned, with Machine-Centric Robotics it is possible to work with two types of robots, generic robots and specific robots. This guide will focus on how to use the assistant to configure specific robots, those robots in the MCR portfolio. More information on the generic robots and how to handle them can be found in Automation Help.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Mechatronic designs \ mapp Robotics \ Specific robots](#)

To start the MCR assistant, it is necessary to select the element "Specific robots" in the Configuration View Object Catalog. The assistant is started by double clicking on it or dragging it into the mapp Motion folder in the Configuration View. Once it is finalized, it generates all the necessary configuration files and hardware elements with all the required settings to add a specific robot to the project.

To get the robots like from Codian displayed as it can be seen in the picture, a hardware upgrade via Automation Studio might be needed. This upgrade is available from B&R homepage.

If attending a training, the trainer also provides the necessary setup files.

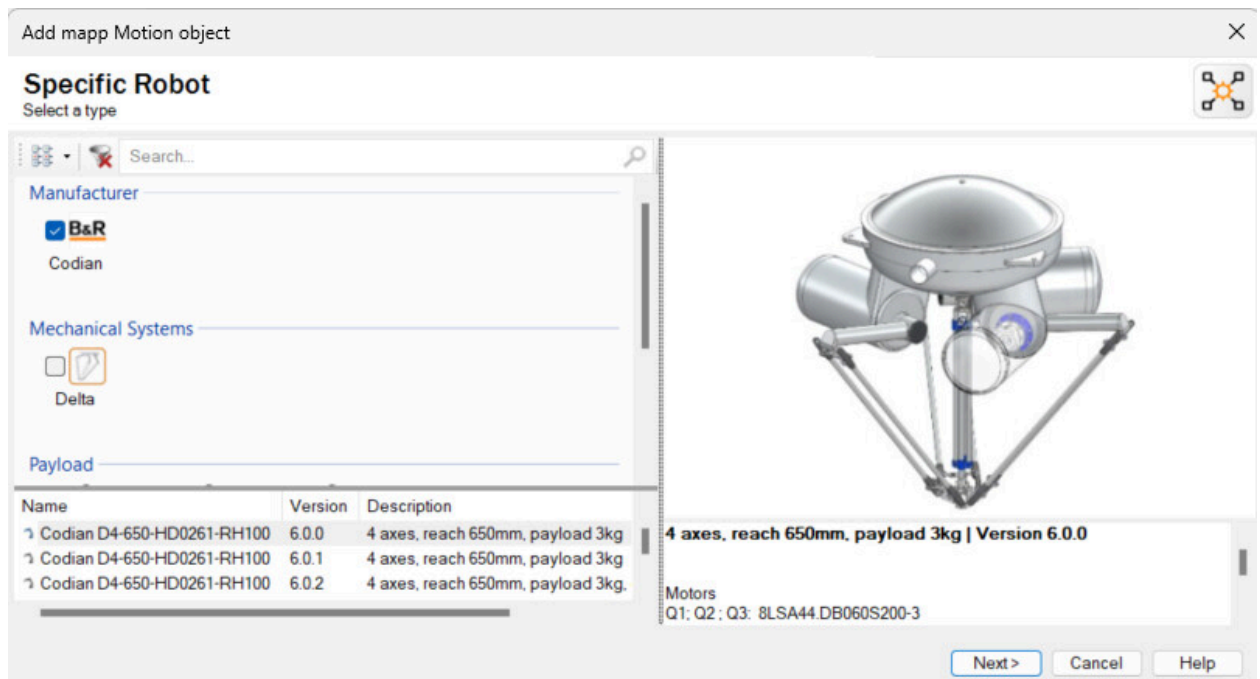


Figure 26: Machine-Centric Robotics assistant

The assistant is structured in three steps:

- 1) The first step is to select the desired robot for the project. Different filters are available to navigate through the portfolio and find the robot that fits the required specifications. All the possible elements can be filtered depending on the manufacturer, the type of kinematics, the maximum payload and the reach of the arm.
- 2) The second step is to select the right connection point for the communication bus. The assistant shows a list of all the interfaces compatible inside the hardware topology of the project and allows selection of the desired connection point for the robot.
- 3) Finally, an overview of all the elements created is displayed. This list contains the new axis group for the robot, all its joint axes, its mechanical system description and all the default axis group features that have been created. It is possible to change the name of all these elements in this step.

Once the assistant is finished, the project is configured and the robot is ready to be tested.

Additionally, the assistant also creates a new "Object hierarchy" file if the project did not have one previously. This file defines some common parameters for all the mechatronic systems such as the units, the rotation definition, the position of the global coordinate system, etc.

The next image shows the configuration files obtained after completing the assistant for a 4-axis delta robot.

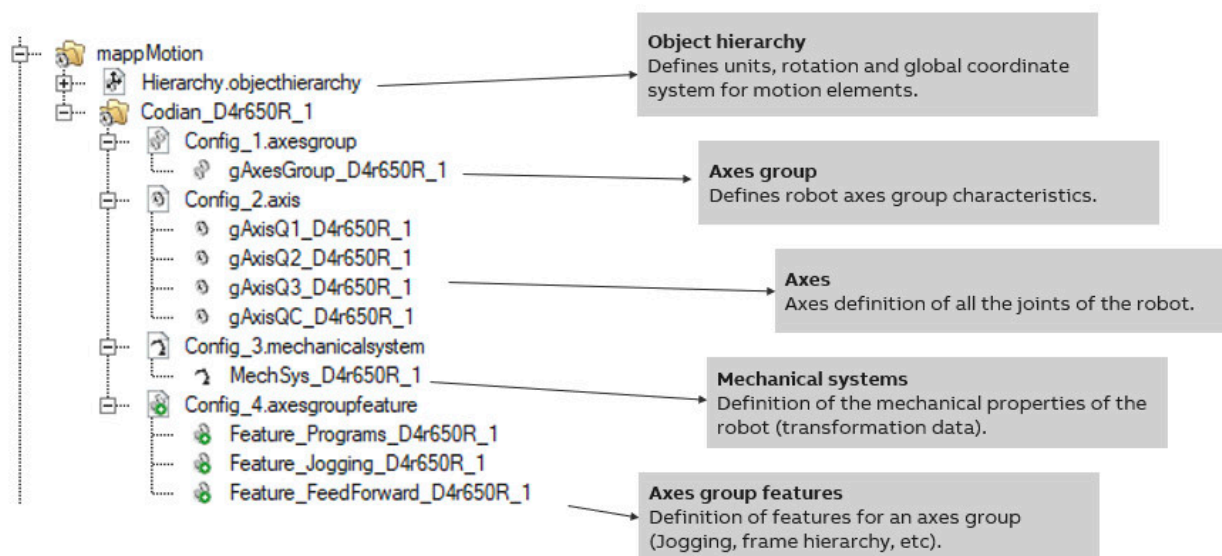


Figure 27: Files created by the MCR assistant

By default, the basic axis group features are automatically added to the project.

- **CNC and robotics programs feature:** This feature can be used to configure the behavior of CNC and robotics programs. Parameters such as the default interpolation mode between movements or the programming units can be defined using this feature.
- **Jogging feature:** This feature is used to enable the jogging function of an axis group.
- **Feed forward feature:** This feature is used to enable the function for feed-forward control with centralized set-point generation of an axis group based on a dynamic model.
- **(Shared brake signals feature):** This feature is optional and enables split brake control functions for an axis group. This makes it possible to simultaneously operate the brakes for multiple axes via a single brake output signal.

Modal data behavior

Most axis group features share a configuration parameter, the "Modal data behavior". This parameter defines the modal data behavior of each feature, meaning how the feature behaves after a robotics program has finished and a new one is launched. The three possible configurations are listed next:

- **"Use axis group settings":** The settings from the axis group are used.
- **"Reset to default":** The values are reset to the configured default values when the program ends.
- **"Keep current values":** The values at program end are used for the next program.

When using the first option, the value of the parameter "Type/Basic settings/Modal data behavior" in the axis group configuration is used. This is the default value when adding a new feature.

More information on these axis group features and all the others can be found in Automation Help.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Basic elements \ AxesGroup feature](#)

Exercise: Create a project

In the following exercise, participants will learn how to create a project for their first robot application. First, an empty project is created, then the desired robot is added and configured using the MCR assistant. For commissioning, map-pCockpit is added and the system is prepared.

- 1) Create a new project with any suitable controller (e.g. X20CP3685, APCXX with PLK interface).
- 2) Add the element "Object hierarchy" to the mapp Motion folder.
- 3) Add the robot (e.g. D4-ST21-1100-R11) by dragging the element "Specific robots" into the mapp Motion folder in the Configuration View or by double clicking on it. If the Robot Installer is not yet available, it may need to be installed using the Automation Studio Upgrade Service.

- 4) Follow the instructions in the assistant.
- 5) Add the file "mapp Cockpit Settings" in the mapp Cockpit configuration settings.
- 6) Add a User Definition in AccessAndSecurity. Define a User "Engineer" and assign the Role "BR_Engineer".
- 7) Prepare the system for a motion application (Use PLK-Interface as System timer and configure Cycle#1: set Duration to 2000 μ s and Tolerance to 0 μ s).
- 8) Activate the simulation. For the rest of this training course, it will stay active.
- 9) Transfer the project.

Further information



[Motion control \ mapp Motion \ mapp Robotics/CNC Getting started](#)
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Mechatronic designs \ mapp Robotics \ Specific robots](#)



Result:

If, after transferring the project, the controller status is "RUN", it means that the system has been configured and initialized correctly. The assistant should be used to add all the mentioned software components and hardware elements to the project. The next image shows an example of the elements used for an 4 axis delta.

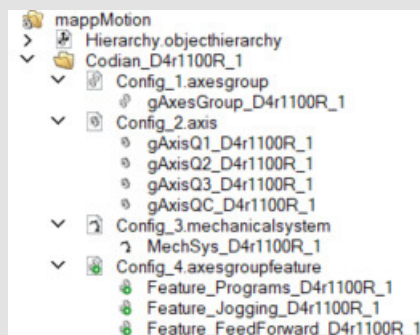


Figure 28: mapp Motion components

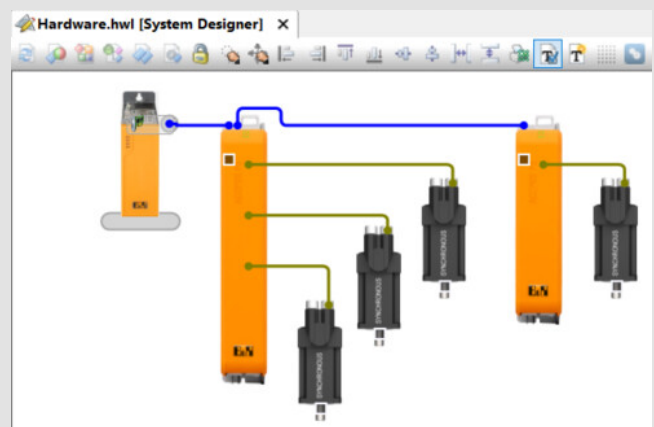


Figure 29: System Designer view

If this exercise is run on real hardware and not in a simulated environment, it is important to remember to correctly configure the broadcast channel for the POWERLINK interface. The following Automation Help link details how to do it for each different type of robot.



[Motion control \ mapp Motion \ mapp Axis \ Programming \ Core libraries \ McAcpAx \ Technical information \ Broadcast with POWERLINK](#)

4.3 Testing the robot with mapp Cockpit

mapp Cockpit can be used to perform the first tests with a robot. Its interface allows triggering some essential commands and performing some basic movements. It also provides information about the status of the axis groups and shows error messages to assist while commissioning a robotic system.



mapp
Cockpit

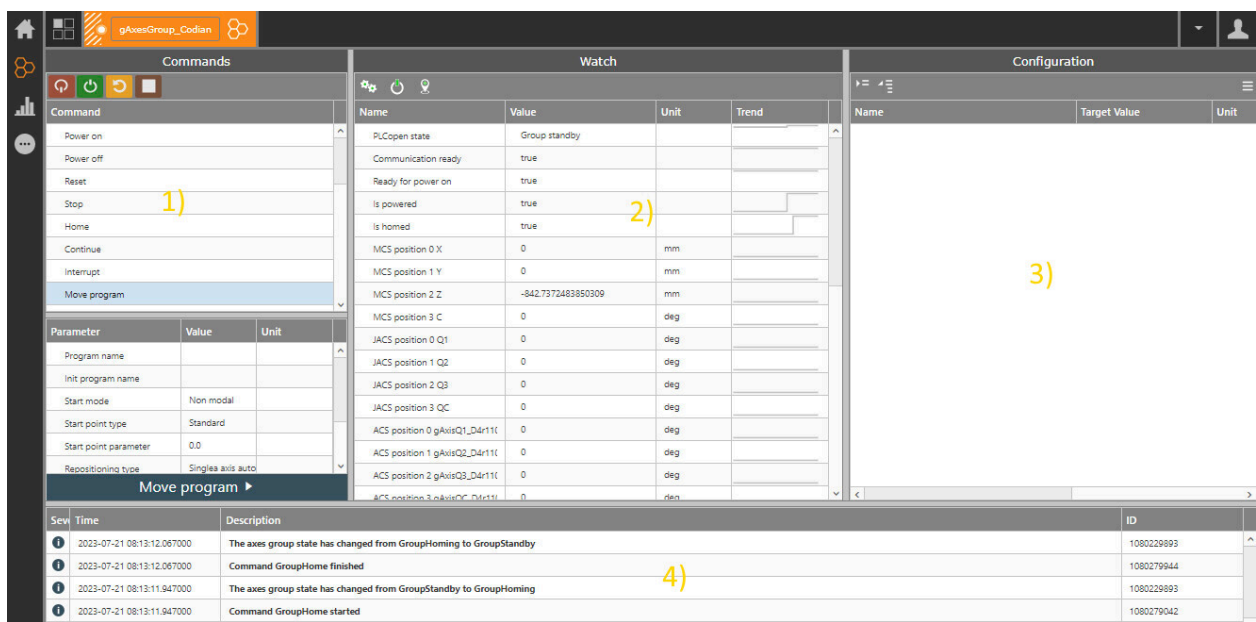


Figure 30: mapp Cockpit axis group control window

The axis group interface of mapp Cockpit is very similar to the one used for a single axis. It consists of four main sections:

- 1) The **command area**. This area allows executing all the possible axis group commands. Some of these commands can be used, for example, to power on the system, home all its axes and perform some basic movements.
- 2) The **live value or watch area**. This area shows the status of the axis group. It shows status values like the position of the TCP, the position of the joints, the PLC state, etc.
- 3) The **configuration area**. This area offers the possibility to change some configuration parameters for the axis group while the system is running.
- 4) The **message area**. This area displays messages regarding the execution of commands given to the axis group. It shows information regarding the success of the commands or possible errors that occurred while attempting to perform them.

To use mapp Cockpit, it is necessary to configure it correctly in the project. For more information about the steps necessary to do this, see Automation Help.



[Diagnostics and service \ mapp Cockpit \ Getting started with mapp Cockpit](#)

The first steps to get an axis group to be ready to move are to power it up and home it. To do that, it is necessary to trigger the commands "Power on" and "Home". The values in the watch windows will provide information about the status of the system. After powering it on and homing it, "Is power" and "Is home" are "true" and the system should be in the PLCopen state "Group standby". Once the robot is in this state, it is possible to trigger movement commands and to perform different tasks.

Commands		Watch			
   		  			
Command		Name	Value	Unit	Trend
Power on	^	PLCopen state	Group standby		^
Power off		Communication ready	true		
Reset		Ready for power on	true		
Stop		Is powered	true		
Home		Is homed	true		
Continue		MCS position 0 X	364.0000	mm	
Interrupt		MCS position 1 Y	0.000000	mm	
Move program		MCS position 2 Z	617.0000	mm	

Figure 31: Powered on and homed robot with mapp Cockpit

More information about all the commands, parameters, watch window values, traceable variables, etc, that can be used for an axis group can be found in Automation Help.



[Motion control \ mapp Motion \ General information \ Diagnostics \ mapp Cockpit for mapp Motion components](#)

Exercise: Using mapp Cockpit for operation

The objective of this exercise is to prepare the axis group for motion and execute some test movements. The axis group must first be switched on and homed. An absolute movement in the coordinate system for the joints (JACS) is then executed.

- 1) Open mapp Cockpit, login as Engineer and select the axis group component.
- 2) Prepare the axis group (power on and home).
- 3) Select the "Move direct absolute" command and enter the necessary parameters to perform a movement in the joint coordinate system (JACS).
- 4) Start the movement.
- 5) Observe the movement in the Watch window.

Further information



[Diagnostics and service \ mapp Cockpit](#)

**Result:**

To access mapp Cockpit, it is necessary to enable the OPC UA system in the configuration of the controller.

Access mapp Cockpit with the following URL or via Tools - mapp Cockpit in Automation Studio:

<http://localhost:8192/mappCockpit>

The following table shows some possible position and movement parameters that can be tested to check if the system has been configured correctly.

Delta robot

Parameter	Position 1	Position 2	Position 3
Velocity [mm/s]	100		
Acceleration/Deceleration [mm/s ²]	1000		
Cordinate system	JACS		
Q1 [°]	30	30	60
Q2 [°]	30	60	15
Q3 [°]	30	0	-15
QC [°]	0	45	180

4.4 Digital twin

A digital twin is a simulated model of a system that has been assigned all the characteristics and functions of the real machine. This includes the physical and mechanical properties as well as the position of all its moving parts. By analyzing the simulated model, it is possible to identify inefficiencies and malfunctions and correctly solve them and optimize the performance of the system.

In robotics, having a digital twin of the robot and its environment can be enormously beneficial. It will not only help to determine the positions necessary to perform a certain task, like setting the picking and placing positions in a pick and place application, but it will also help to prevent collisions and understand the trajectories while running a certain task. Although there are different possibilities for creating a digital twin, in this training course the focus will be on using the B&R simulation tool **Scene Viewer**.

4.4.1 Scene Viewer

Scene Viewer can be used to show the movement of some predefined robot kinematics. The position setpoints for each individual joint axis are transferred to the simulation software and the movement is displayed visually like it would occur with a real system.

Setting up a digital twin is very simple using Scene Viewer. When the system starts and a File Device is configured in the mapp Motion configuration, the file will automatically be created.

Also on new mechanics the manual procedure by adding or creating a suitable equivalent mechanical structure of the machine in the simulation environment and linking all the moving elements to the set positions created by the application, a simulated model of the machine is obtained. As mentioned previously, this model can be a huge help when commissioning and developing an application.



This simulation tool includes a help system that documents all its features. It describes how to create a scene and how to add different elements and edit their characteristics and positions. It also describes how to link moving elements to application variables to simulate the behavior of the system. Specifically for robotics systems, two guides are included. The first one shows how to include and use a robot model in Scene Viewer. The second one explains how to create a new custom robot with any desired mechanical structure and use it in a simulation.



[Motion control \ mapp Motion \ General Information \ Concept \ Simulation \ Scene Viewer](#)

The next image shows a view of the proposed scenario with a 4-axis robot:

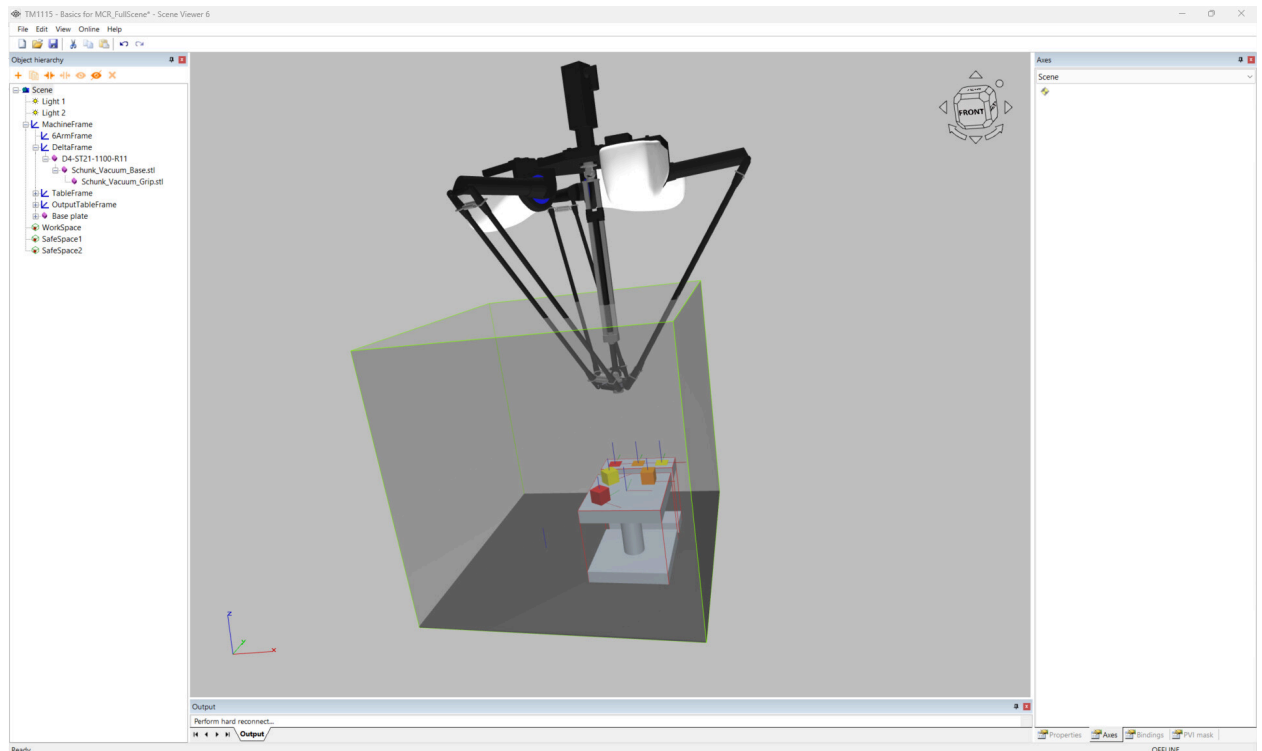


Figure 32: View of the proposed scenario with a 4-axis robot in Scene Viewer



Download from www.br-automation.com (Downloads → Software → Simulation)

4.4.2 Connecting the application to the digital twin

Depending on the robot that is used, and if it's already supported by mapp Motion, the configuration setting of creating a SceneViewer file at startup can be activated. The SceneViewer file will then automatically insert the robot and all coordinate systems (frames) and after establishing the connection to the PLC the robot will move according to the started program or movement.

Therefore the path to the FileDevice in the Objecthierarchy and the Scene Viewer Object in the mechanical systems settings configuration file need to be checked for the right settings. Otherwise a generic robot will be inserted in the generated Scene Viewer file on the specified FileDevice location.

If the robot is not supported yet, and waiting for the update is not an option, the link to the robot can be established manually. Therefore following steps are required, which are described in more details later on:

- Create a task on the PLC and use the FB MpDeltaXAxis (X stands for number of axes)
- Share the JointAxesPositions from the Info output structure of the Mp-FB in OPC UA
- Insert the used robot in Scene Viewer after installing the robot update package (ask B&R support to get the installation package if not included)
- Bind the axis position with the OPC UA variables previously added

Manual connection of robot in Scene Viewer

As previously mentioned, the digital twin needs the position setpoints of the individual joint axis to properly display the movements of a robot. For that, it is necessary to communicate these values from the application to the simulated model in Scene Viewer.

The function block MpDeltaXAxis (X stands for the number of axes) offers an output structure, where the position of the joints are provided. These values can be used to transfer via OPC UA, so the SceneViewer is able to calculate the robots movement based on the joint axes positions.



To use the correct transformation and see the correct movement, always set joint positions are recommended to be used. The cartesian positions X, Y, Z as a source for Scene Viewer can lead to problems displaying the correct movement depending on the executed command.

Name	Type	Value
MpDelta4Axis_0	MpDelta4Axis	
MpLink	McAxesGroup Type	
Enable	BOOL	
ErrorReset	BOOL	
Parameters	MpDelta4AxisPar Type	
Update	BOOL	
X	LREAL	
Y	LREAL	
Z	LREAL	
C	LREAL	
PathVelocity	REAL	
CommandBusy	BOOL	
CommandAborted	BOOL	
PowerOn	BOOL	
IsHommed	BOOL	
MoveActive	BOOL	
MoveInterrupted	BOOL	
MoveDone	BOOL	
Stopped	BOOL	
InMotion	BOOL	
SingleStepActive	BOOL	
Info	MpDelta4AxisInfo Type	
CommunicationReady	BOOL	
ReadyToPowerOn	BOOL	
WaitForContinue	BOOL	
JointAxisPosition	LREAL[0..3]	
JointAxisPosition[0]	LREAL	
JointAxisPosition[1]	LREAL	
JointAxisPosition[2]	LREAL	
JointAxisPosition[3]	LREAL	
Program	McProgInfo Type	

Figure 33: Output structure of FB MpDelta4Axis

The PV variables containing the information regarding all these set positions must then be linked with the digital twin. The two possibilities to complete this connection are using OPC UA or PVI connections. In this guide, the focus will be on using OPC UA.

To share a variable via OPC UA, the first necessary step is to add an "OPC UA default view" file to the connectivity folder in the Configuration View. This file gathers all the variables in the project and allows them to be enabled on the OPC UA server, which means that they will be shared via this bus and other clients will be able to read them.

The second step is to enable the variable containing all the joint positions in this configuration file. This should be the output structure just introduced. Once enabled, it will be accessible from Scene Viewer and it will be possible to link the joints of the simulated robot to these setpoints, thus creating the digital twin. It is very important to activate the property "Automatic Enable" to "Recursive" so Scene Viewer can access each one of the set positions separately. The next image shows all the configuration settings needed to enable the OPC UA server connection.

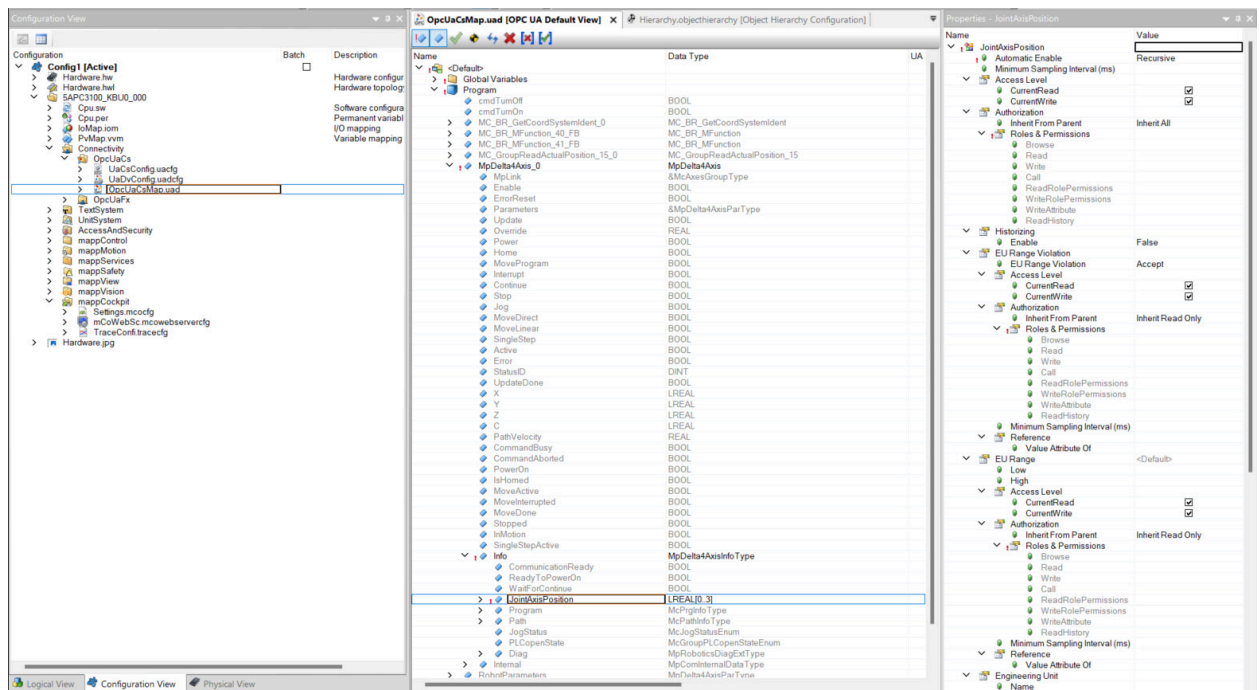


Figure 34: Configuration of the OPC UA server

As a final step, the OPC UA system must be active on the controller (if not already done). To do that, it is necessary to access the configuration file for the controller and set the parameter "OPC-UA System/Activate OPC-UA System" to on. The next exercise shows details about how to complete all the necessary steps to connect the digital twin in Scene Viewer and enable this OPC UA connection. For more information about how to establish this connection, see Automation Help and Scene Viewer Help.



[Motion control \ mapp Motion \ General Information \ Concept \ Simulation \ Scene Viewer](#)

Exercise: Creating and operating a digital twin

The following exercise shows how to use the Scene Viewer simulation tool to create a digital twin of your robot, operate it and connect the model to the application. With the FB MpDeltaXAxis, joint axis positions for robot are made available cyclically. The target positions are then transferred to the Scene Viewer model via OPC UA. After successful connection, some test movements are suggested to check the behavior of the robot.

Automation Studio:

- 1) Create a new ST program (task).
- 2) Add the FB "MpDelta4Axis" (when 4-Axis robot).
- 3) Create variables for the task with type "MpDelta4AxisParType" (when 4-Axis robot).
- 4) Call the FB in the ST program (see [Getting Started MCR Codian Delta robots](#) for help)
- 5) Add the OPC UA Default View file to the Connectivity folder in the Configuration View.
- 6) Enable array "Info / JointAxisPosition[]" in the OPC UA Default View.
- 7) Enable attribute "Automatic Enable - Recursive" for this array in the OPC UA Default View.
- 8) Compile and transfer the project.

Scene Viewer:

- 1) Create a new empty Scene Viewer scene or use an existing one.
- 2) Place a suitable robot in the scene. (Maybe it's necessary to install a codian plugin for the scene viewer available [here on the B&R homepage](#))

- 3) Test the robot movements using the sliders in the "Axes" window (bottom right corner).
- 4) Connect Scene Viewer to the OPC UA server.
- 5) Go to "Online / OPC UA / Connect" and select the default server.
- 6) Bind each "Real axis" of the robot to the array position that contains its set position in the "Bindings" window (bottom right corner).
- 7) Optional: Use automatic generate scene (see configuration for "Scene Viewer" in [Object hierarchy](#))

mapp Cockpit:

- 1) Perform a movement with mapp Cockpit and check simulation behavior.

Further information

[Motion control \ mapp Motion \ General Information \ Concept \ Simulation \ Scene Viewer](#)



Result:

If the connection between the digital twin and the application is running correctly, no red messages should appear in the "Output" window of Scene Viewer and the status showed in the bottom right corner of the screen should read "RUN". The next image shows an example of a Scene Viewer model binded to the application via OPC UA.

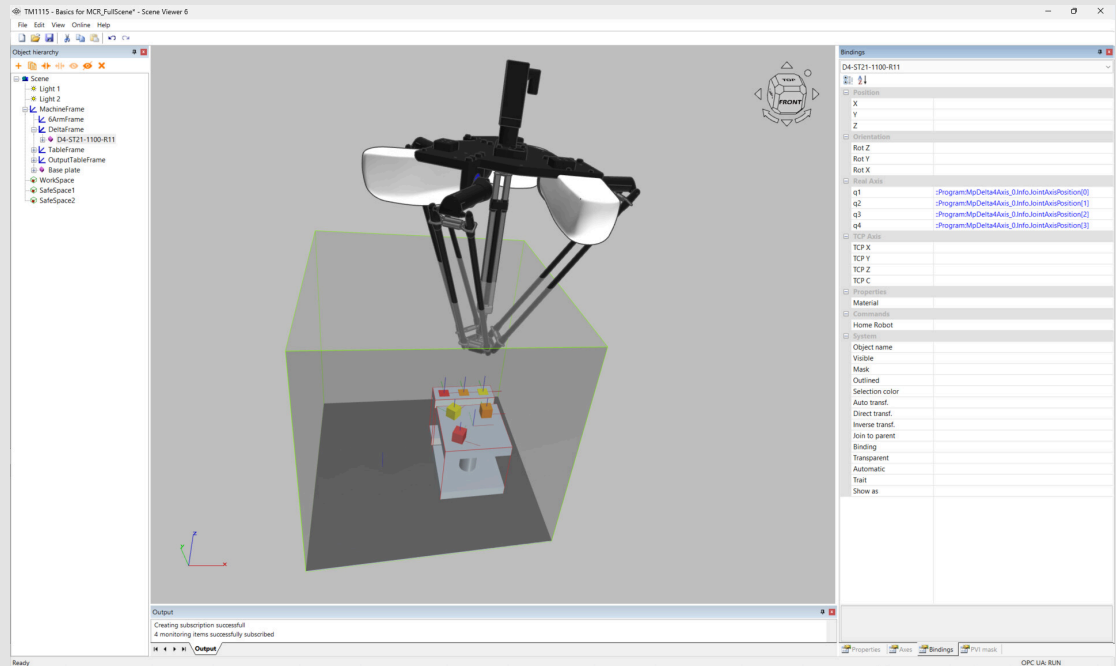


Figure 35: Scene Viewer simulation of a robot with proper binding through OPC UA

As can be seen in the image, each of the real axes on the robot must be bound to the respective setpoints shared on the OPC UA server. Be sure to bind them in the correct order so the digital twin really represents the real system.

When testing movements in mapp Cockpit, the robot should move accordingly in the simulation. The position of the TCP can be checked in the "Axes" window.

The automatic Scene Viewer generation can optionally be used to simplify setup. To enable this feature, the AxesGroup must be properly configured in the Object Hierarchy, and an existing File Device must be selected in the Scene Viewer – File Device setting.

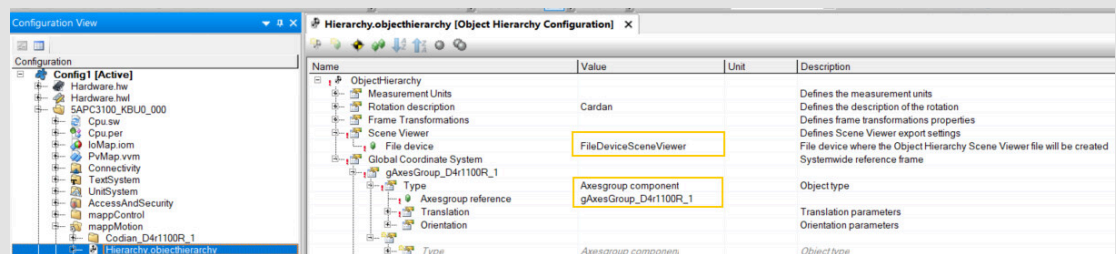


Figure 36: ObjectHierarchy configuration for automatic Scene Viewer generation

5 Basic movements and parameters

This chapter defines some of the basic movements and basic parameters used when working with robotic systems. The goal is to introduce basic robotic concepts in order to be ready to test the system created in the previous chapter and understand how robotic movements are defined and performed.

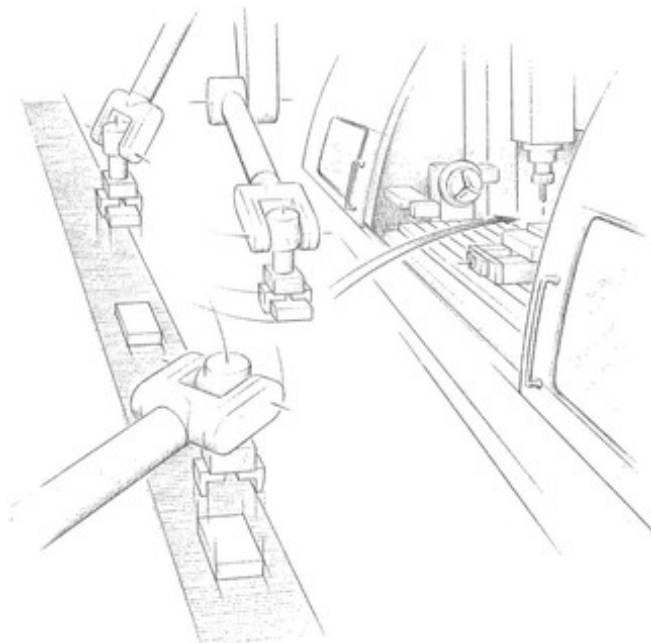


Figure 37: Machines such as grippers for retrieving products can be implemented using robotics systems.

5.1 Path-controlled movements

Path-controlled movements are essential for robotic applications. The core idea behind path-controlled movements is to generate set positions for each individual axis in the axis group in order to reach a certain position in space or follow a certain trajectory. This is accomplished using the concept of path calculation. This functionality generates cyclic position setpoints for the axis group's path-controlled axes from movement commands. It is the basis for handling precise and complex robotic tasks.

Each movement command defines different options on how the robot can move from its current position to a new one. These possibilities generate different movement types that have different advantages and restrictions.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Concept \ Technical Information \ Path-controlled movements](#)

The next sections present some of these concepts and describe both the advantages and the potential problems that each one has.

5.1.1 Basic coordinate systems

But before discussing the basic types of movements, it is necessary to describe how positions in space are defined. Robots are, as mentioned, mechanical systems that by moving their individual joints or axes can reach different positions in space. But, how are these positions described?

A key part of a mechanical system is the tool center point or TCP. This point is usually placed in the central point of the tool mounted on the robot or, if there is no tool, its flange. It defines the position and orientation of the robot in Cartesian space. So if, for example, a robot is in a position of 3 m in the X axis and 1 m in the Y axis, it refers to the position of the robot TCP with respect to a reference point, for example, the base of the robot. The joints of the robot can be in different positions depending on the mechanics of the robot, but it is ensured that the TCP is in that position with respect to its base.



Figure 38: Examples of different robot mechanics and the position of their TCP coordinate systems.

So to conclude, the primary task of a robot is to control the movement of the tool center point. The focus is on the path that the TCP traverses through space, rather than the positions of the individual joints required to achieve that movement. A given movement can usually be achieved by different combinations of joint positions.

The position of a robot can therefore be defined in different ways, either using the TCP or defining the position of each joint. Next, the three basic possibilities are described:

- **Axis (motor) coordinate systems (ACS):** Positions are defined with respect to the robot axes or motors. The mechanical system is not taken into consideration when defining positions using this system. The motors of the robot are commanded to move to a certain position. Any possible couplings between axes or mechanical restrictions are not considered.
- **Joint axis coordinate systems (JACS):** The position of the robot is determined by the rotation angle of its joints. This position can also be specified in the configuration of the robot. Couplings and other mechanical restrictions of the system are taken into account.
- **Cartesian coordinate systems (CCS):** The position of the robot is determined by the distance and orientation of the TCP with respect to a specific coordinate system. Depending on the degrees of freedom of the robot, Cartesian space can have from one to six dimensions. Robots that only move along a plane with no rotation of the TCP will require two coordinates (XY), while a 6-axis robot that can move along all possible coordinates in the physical space will require 6 coordinates (XYZ ABC).

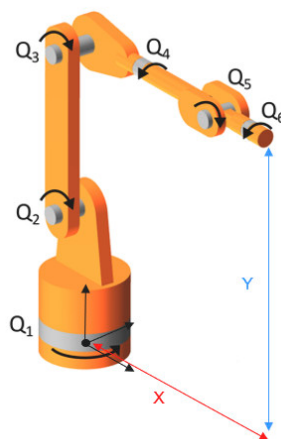


Figure 39: Same position defined using JACS and CCS

The basic coordinate system in B&R robotics when moving based on Cartesian coordinates is the machine coordinate system (MCS). This is the core coordinate system and all positions and elements are defined with respect to it. By default, this system is placed at the base of the robot.

5.1.2 Direct and inverse transformations

As just mentioned, there are two main options for commanding the robot to move to a certain position: Using Cartesian coordinates for the TCP with respect to a specific coordinate system or directly using joint positions to move each axis on the robot. But, either way, the robot can only move the motors in their system to achieve different positions. For this reason, it is necessary to use transformations to go from Cartesian coordinates to joints and back.

The transformations are mathematical operations that perform the conversion between Cartesian positions of the TCP in space to joint configurations and vice versa. When a movement command is executed, it is always necessary to obtain the motor positions of the system. The control system of the robot is capable of using these transformations to compute the position of each of the joints to reach various positions. It takes into account the mechanics and the couplings between axes of the robot to generate the necessary set positions for each motor. All these calculations are done internally and are transparent, so the user doesn't have to worry about them or handle implementation.

Each transformation has a different name:

- **Direct or forward transformation** converts the joint configuration of a robot to the position of the TCP with respect to its base.
- **Inverse or backward transformation** converts the Cartesian position of the TCP with respect to the base link of the robot to the joint configuration.

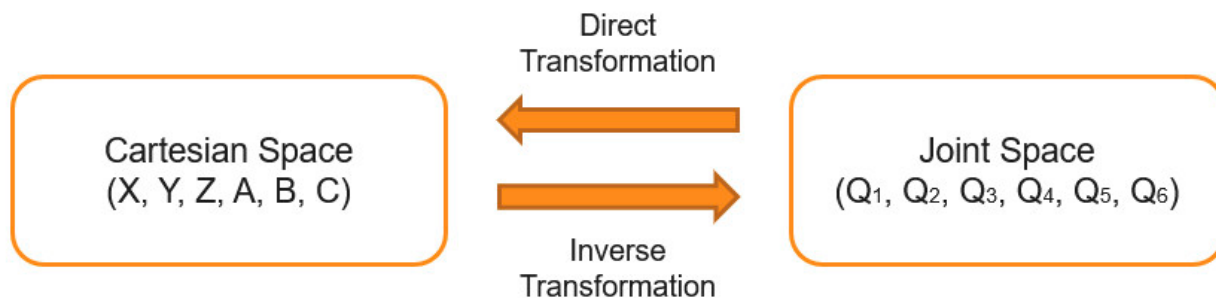


Figure 40: Transformations for a 6-axis robot

5.1.3 Singularity

There exists a recurrent problem that occurs when transforming positions from Cartesian space to joint space. The issue is a consequence of the fact that certain TCP positions can be reached by an infinite number of joint configurations. These positions are known as **singularities**.

A singularity is a condition caused by the collinear alignment of two or more robot axes. Clear examples of singularities occur in 6-axis robotic arms. Two clear examples of this problem are detailed in the following:

- The first configuration is called the **zero crossing of axis 5**. This singularity occurs when axes 4 and 6 are aligned. There is an infinite combination of values for Q_4 and Q_6 that result in the same position of the TCP.
- The second configuration is called the **inversion of the tool**. This singularity occurs when axes 1 and 6 are aligned. The tool is aligned with the base of the robot. An infinite combination of values for Q_1 and Q_6 result again in a fixed position of the TCP aligned with the base of the robot.

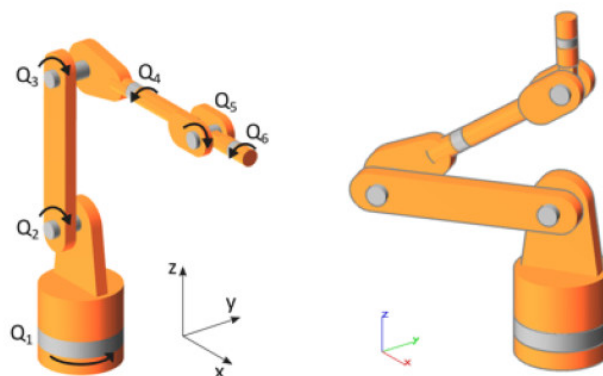


Figure 41: Examples of singularity positions (left: axes 4 & 6 aligned, right: axes 1 & 6)

As mentioned in both cases, there is an infinite number of possible axes configurations that would result in the same tool position or movement path. With these positions, it would be necessary for multiple axes to move in opposing directions at infinite speed to complete a movement through the singularity points, which is not possible. Therefore, when a singularity is reached, the result is usually an error and a cutoff.

These singularity positions can be exited by moving the individual joints or moving the robot in joint space.

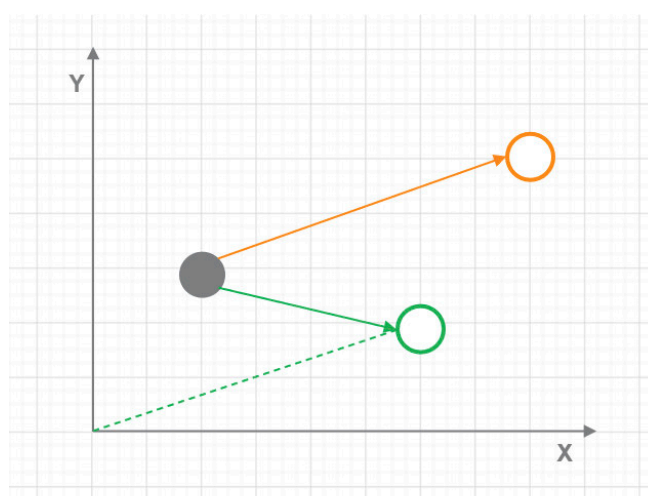
5.2 Movement types

After defining some basic concepts for robot positions and movements, this section focuses on presenting different types of movements. The main focus is to differentiate between movements depending on how the target position has been defined or on how the trajectory to go from the current position to the target position is computed.

5.2.1 Absolute vs relative movements

Depending on how the target position is defined, movements can be classified as two main types:

- **Absolute movements** are those movements in which the target or final position is defined with respect to a fixed point in space or joint configuration.
- **Relative movements** are those movements in which the target or final position is defined with respect to the current position or state of the robot.



- Move Absolute X= 6 and Y = 2
- Move Relative X= 6 and Y = 2

Figure 42: Example of absolute and relative movements in the XY plane.

From the previous definition, it is clear that for absolute movements it is necessary to have a zero or base position. When moving in joint space, this base position is defined as the homing position of each of the axes. When moving in Cartesian space, however, it is defined by a coordinate system selected by the user when starting a movement. One example is the Machine Coordinate System introduced previously. Chapter "Coordinate systems" provides more details about all the possible coordinate systems.

5.2.2 Linear vs P2P movements

Depending on the trajectory used by the TCP to move from its current position to the target position, movements can be classified into two main types: linear movements and point to point (P2P) or direct movements.

- **Linear movements** are those in which the TCP moves along a straight line in Cartesian space connecting the current position and the target position.
- **P2P movements** or **direct movements** are those in which the movements of all robot axes are optimized. The axes move synchronously from the current configuration to the target configuration using the most efficient trajectory. This usually results in curved trajectories of the TCP in Cartesian space.

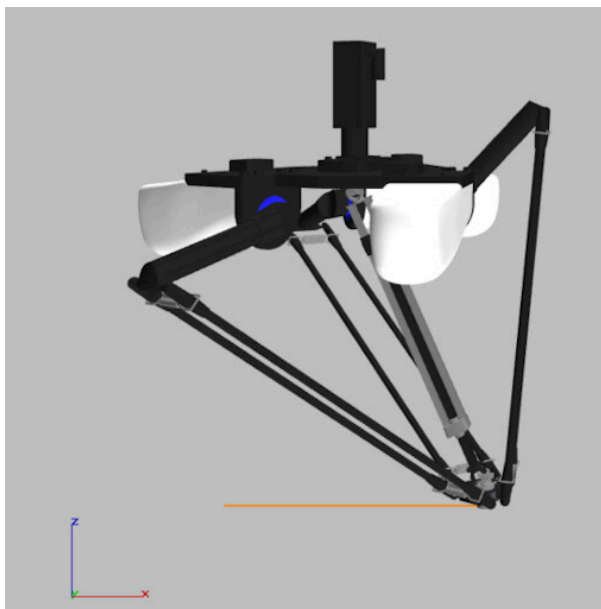


Figure 43: Linear movement

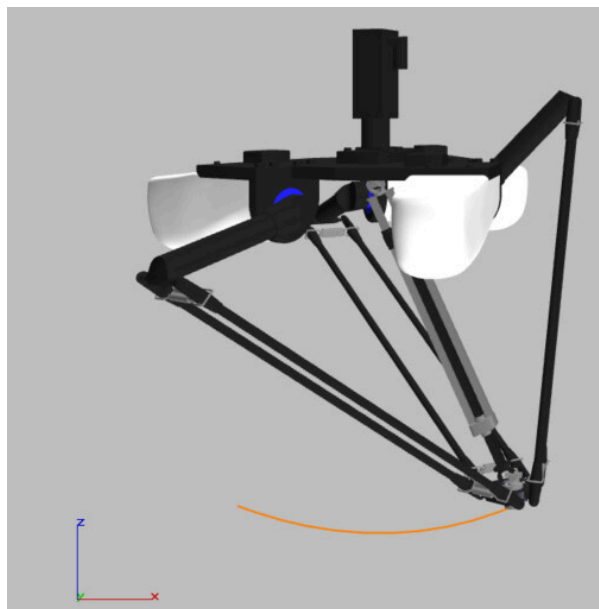


Figure 44: P2P movement

In a typical robotics application, P2P movements are used to move the TCP to distant positions, for example to move from one working area to another or to reach an approach point before switching a tool. Linear movements, on the other hand, are used to perform specific tasks in the application, for example picking a product, drilling a hole, etc.

5.3 Feed rate

The feed rate is one of the most important process parameters involved in a path-controlled movement. It defines the speed of the path followed by the TCP in units per minute.

The feed rate of a path-controlled movement is the vector sum of the speeds of the individual axes. By default, all the path axes are involved in the feed rate calculation.

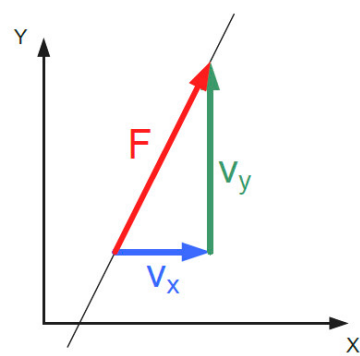


Figure 45: Feed rate F - Sum of V_X and V_Y

For example, if a robot that can only move in the XY plane, the feed rate is the magnitude sum of the vectors that define the TCP velocity in the X and Y directions.

So, depending on the trajectory, the system will move faster or slower in each one of the coordinate directions but its movements with respect to the plane can have the same speed.

5.4 Override

Override is a movement factor that is multiplied by a certain system parameter to modify the behavior of a movement. By applying this factor, parameters like the speed or accelerations of movements can be modified on the fly. For example, a velocity override would be a factor between 0 and 1 that is multiplied by the velocity of the movement to change it. Depending on its value, it would increase or decrease the speed at which the path is completed. The override factor may impact both path-controlled and non-path-controlled axes within the axis group. The following table presents a list of possible overrides that can be applied to different parameters in a system.

Type	Override	Affect
Feed rate	0 to 1	Path-controlled movements; modifies the feed rate of the programmed path.
Rapid	0 to 1	Path-controlled movements; modifies rapid movements.
Time stretch	-1* to 1	Stretches the path in time and thus affects the velocity, acceleration and jerk simultaneously. (* Only if reverse movements are enabled.)
Velocity	0 to 1	All path-controlled movements.
Acceleration	0 to 1	All path-controlled movements.

Examples of different types of override factors.

The most commonly used type is the time stretch override. This factor affects the whole path and so it modifies all its parameters making it the most suitable for a lot of common use cases.

More than one factor can be applied to a certain movement. However, not all of them can be used at the same time. Depending on the type of movement, different factors can be multiplied to change the behavior of path-controlled movements. Using these combinations, it is possible to affect some other movement parameters, like path acceleration or jerk. The following image shows three examples of these combinations:

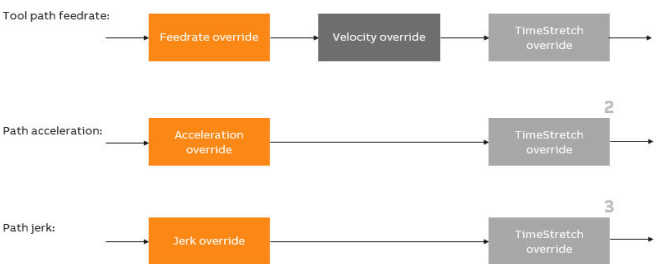


Figure 46: Override interaction examples

For more information about all these possibilities, see AS Help.

Exercise: Linear vs P2P

In this exercise, course participants will be instructed to test linear and direct or P2P movements with mapp Cockpit and look at the differences between them. First, it is necessary to prepare the axis group for movement by switching it on and homing it. A linear and a direct (P2P) movement are then performed. The path followed by the TCP can be tracked using the Scene Viewer tool "Path tracker".

- 1) Add Scene Viewer tool "Path Tracker" to the TCP.
- 2) Open mapp Cockpit and select the axis group.
- 3) Switch on the axis group and home it.
- 4) Define a start and target position for both movements and switch the coordinate system to MCS.
- 5) Move the TCP to the start position with the command "Move direct absolute".
- 6) Move the TCP to the target position with "Move linear absolute".
- 7) Repeat the movement from one corner to the other with "Move direct absolute".

Watch the path in Path tracker as it moves. Look at the differences and see if the trajectories match the description of each type of movement.

Further information



[Motion Control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Libraries \ Core libraries \ McAx-Group \ Function blocks \ MC_BR_GroupSetOverride](#)



Result:

Try different positions and vary more than one coordinate to check how the trajectory changes. The next suggested positions can be tested to help understand the behavior of each movement:

- For the a delta robot scene, the points could be $(X,Y,Z,C) = (-300, 0, -750, 0)$ and $(X,Y,Z,C) = (300, 0, -750, 0)$

To track the movements properly in Scene Viewer using the tool "Path tracker", it is necessary to properly enable and disable it.

By enabling and disabling the command "Record" for this object, it is possible to start and stop tracking the path. By triggering the command "Clear", on the other hand, the path can be reset and started again.

It is also possible to change the color of the path and to increase the number of points recorded in order to track longer trajectories.

More information on the object can be found in Scene Viewer Help.

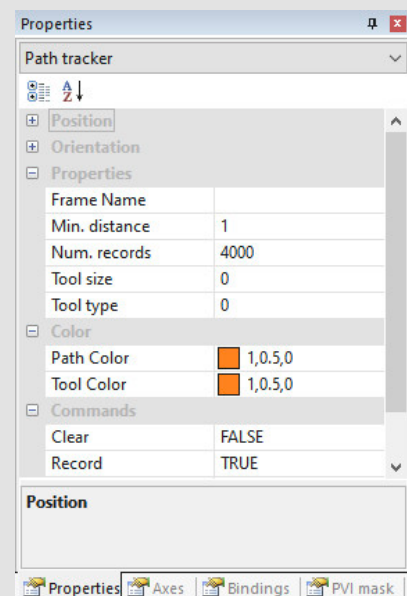


Figure 47: Path tracker properties

Optional exercise: Record a trace

A trace can be recorded by carrying out the following steps. First, data points are selected, then the trigger is configured and then a movement is started. This process is repeated to draw a comparison between the two types of movements, linear and direct. The same start and target positions from the previous exercise can be used.

- 1) Open mapp Cockpit and select the axis group component.
- 2) Prepare the axis group (power on and home).
- 3) Move the robot to the starting position.
- 4) Navigate to the configuration for the trace.
- 5) Add the positions of all individual axes to the data points of the trace recording.
- 6) Configure the trigger: Select the condition so that the trace recording is triggered when the movement starts (path velocity not equal to 0).
- 7) Activate the Trace function.
- 8) Trigger a direct movement to the target position.
- 9) Repeat the trace recording of a linear movement.
- 10) Compare the positions of the individual axes, especially Q2 and Q3.

Further information

[Diagnostics and Service \ mapp Cockpit \ Web-based HMI application \ Trace](#)

**Result:**

After performing and comparing both movements, the biggest differences should be noticed in axes Q2 and Q3. The next image shows a possible result for these traces using a 3-axis delta robot.

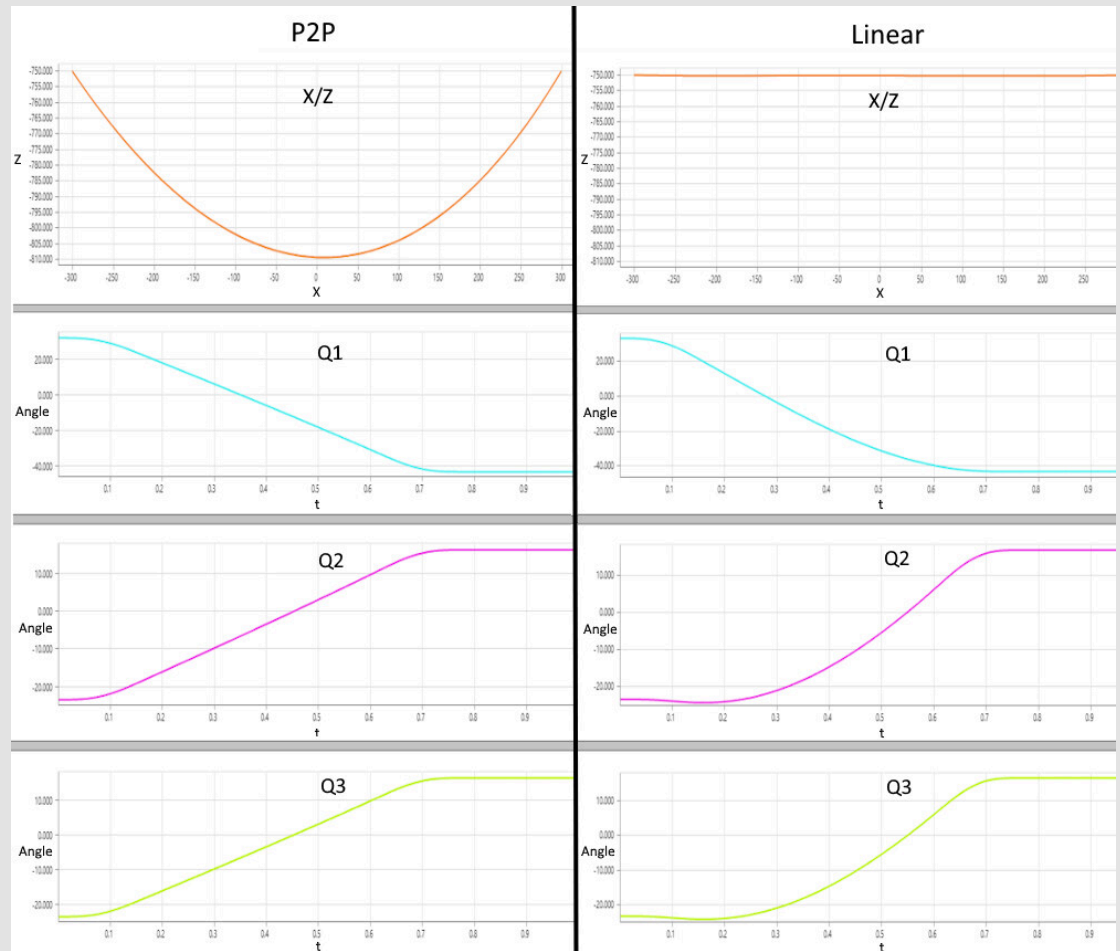


Figure 48: Sample traces

The next image shows a possible trace configuration for a delta robot with the datapoints to obtain the position of all axes and the X,Y,Z position of the TCP. It also describes a possible configuration for the trigger signal.

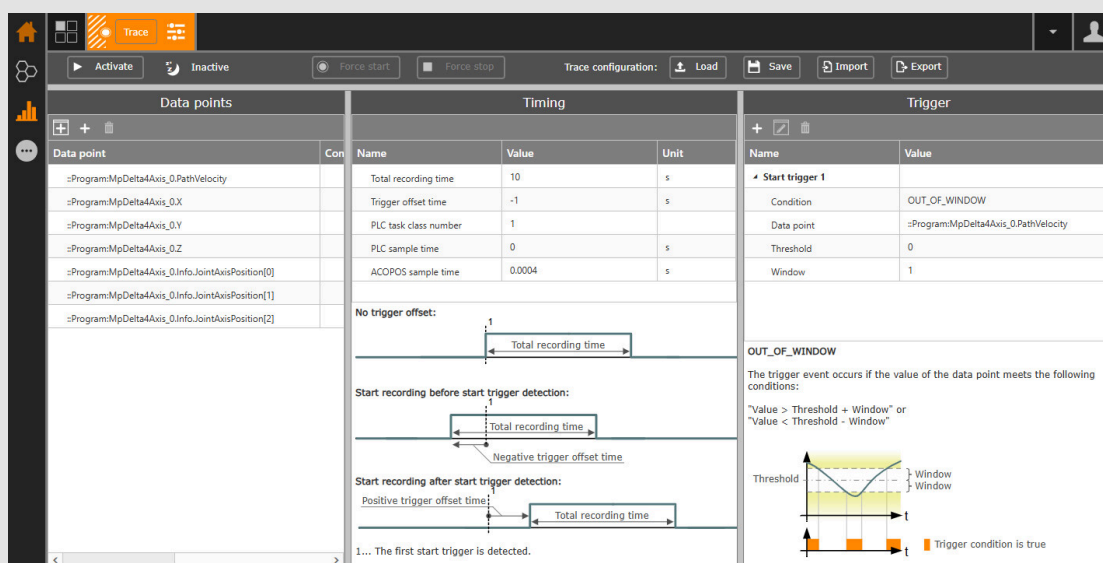


Figure 49: Sample configuration for the traces

Optional exercise: Reading the logger - Part 1

In this exercise, an error is deliberately triggered in the prepared axis group and diagnostics are performed using the Logger. Various errors, which either exceed a limit of a single axis or the range of the TCP, can be triggered here. The "Motion" module must be enabled in the Logger to be able to analyze all information on the errors.

- 1) Prepare the axis group.
- 2) Trigger an error, such as:
 - Position in the JACS coordinate system greater / less than limit value.
 - Position in the MCS coordinate system outside the range of the robot.
- 3) Open the Logger and activate the "Motion" module.
- 4) Download the Logger entries.
- 5) Analyze the Logger entries.

Further information



[Diagnostics and service \ Diagnostics tools \ Logger](#)

**Result:**

After triggering each error, it should be possible to find the corresponding entries in the Logger and analyze their information. Then after resetting via the "Reset" command in mapp Cockpit and correcting the error, the program should be ready to be executed again.

The screenshot shows the 'Logger' application window. The top status bar indicates 32 Errors, 0 of 5 Warnings, 0 of 186 Informations, and 1 Success. The left sidebar shows a tree view of modules, with 'Motion' selected. The main area displays a list of error entries:

Severity	Time	ID	Entered by	Description
1 Error	2025-03-26 09:32:49.584...	-1067245568	gAxesGroup_D4r1100R_1	Command: MoveDirect failed
2 Error	2025-03-26 09:32:49.484...	-1066860046	gAxesGroup_D4r1100R_1	Intended TCP position cannot be reached - Am1 (Upper arm length + Lower arm length) is not long enough; Me...
3 Error	2025-03-26 09:32:49.430...	-1067317248	gAxesQ3_D4r1100R_1	Command: MoveCyclicPositionStartSyncMotion failed
4 Error	2025-03-26 09:32:49.430...	-1067186160	gAxesQ3_D4r1100R_1	During execution of the command an error occurred. See previously entered message for further information
5 Error	2025-03-26 09:32:49.430...	-1067379826	gAxesQ3_D4r1100R_1	Start of movement not possible: Axis is not homed. ParID: 778

The 'Details' pane for the selected error (ID: -1066860046) shows the following information:

Name	Value
Severity	Error
Date	26.03.2025
Time	09:32:49.484000
ID	-1066860046
Area	B&R
Facility	105
Entered by	gAxesGroup_D4r1100R_1
Logbook	Motion
Record ID	210
Format	EventId
Location	Online
Description	Intended TCP position cannot be reached - Am1 (Upper arm length + Lower arm length) is not long enough; Mechanical system: 4-axes delta (A); Method: inverse
ASCII Data	44 19 00 00 00 44 15 00 00 00 5E 00 00 00 00 00 69 40 5E 00 00 00 00 00 24 40 5E AA FF 16 AA 29 19 B8 40 5E 00 00 00 00 00 00 00 00 00 00 00 00 00 00 74 40 5E 00 00 00 00 90 8A 40 5E 00 00 00 00 00 00 74 40 5E 00 00 00 00 90 8A 40 5E 00 00 00 00 00 00 74 40 5E 00 00 00 00 90 8A 40
Binary Data	

Figure 50: Example of Logger entries

6 Create and use robotics programs

Motion programs are usually the basic element used to perform robotic tasks. They establish a modular and reusable approach to perform complex motion trajectories.

Each program can be structured to tackle a certain task and the same system can run different programs to perform different processes. They can be easily modified and adapted for different robots and environments. Therefore, they are a very efficient way to implement robotic tasks.

ST robotic programs use the syntax of Structured Text to define a motion path via motion instructions or commands. These programs can include the declaration of variables and constants, logical decisions, motion commands and so on.

In this chapter, an introduction to ST robotics programs is presented. The basis of how to create a program and run it are discussed. Robotic programs must be stored on a local folder and can be executed from there.

Robotic program

```
VAR
    InitialPos : McAxisTargetType := ...
    Position1 : McPointType := ...
    Position2 : McPointType := ...
    Position3 : McPointType := ...
    Position4 : McPointType := ...
END_VAR

PROGRAM _MAIN
    Feedrate(5000);
    MoveAR(InitialPos);
    WaitTime(1);
    MoveJ(Position1);
END_PROGRAM
```

This training course will focus on robotics programs developed using Structured Text. G-code is also quite commonly used for this application. All the concepts and commands showed in this guide have their equivalents in G-code and can also be used. The following page in Automation Help connects both languages and shows these equivalences.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming\ Programming languages \ G-code and ST overview](#)

6.1 Variables

Variables are a basic element in all robotics programs. They can be used to save information, modify it or even communicate with other programs or the machine application. They can be defined above the motion logic, at the beginning of the ST program or in separate files.

When creating variables inside a program, the variables must be declared inside a specific section delimited by the statements "VAR" and "END_VAR". The rules are the same as when declaring variables in ST tasks. All must have a name and a type. An initial value can also be specified.

It is possible to have more than one variable declaration section in a program. Using specific modifiers for the "VAR" statements, it is possible to change the properties of the variables declared inside them. They can become constants, they can have a different scope (for example global variables for all programs) or they can even be linked with PLC application variables. One example is the statement "VAR CONSTANT". All variables inside the variable declaration created using this statement are constants, and therefore they can be read in the program, but not modified. The next example shows how to create different types of variables and constants.

```
VAR
    NumberOfTools: UINT := 5;
    Pos: ARRAY[1..3] OF DINT;
    GripperSensor: BOOL;
END_VAR
```

```
VAR CONSTANT
    MAX_NUMBER: UINT := 100;
    LIMITS: ARRAY[1..3] OF
        DINT:= [2(1), 2];
END_VAR
```

The other possibility is to declare variables using ".var" files, as done in ST application programs. When doing this, it is first necessary to create a ".var" file that will contain all the variables and constants needed in the program. Using the keyword "#include", this file will then be added to the beginning of the program. One big benefit of using this approach

is that variables can be modified inside of Automation Studio using the "Table" view. This tool is very helpful because it includes a lot of important parameters and also simplifies creation of new variables, reducing the possibility of having syntax errors in these files.

When including the file in the program, it is important to use the correct path from the program location to the variables file. A good practice is to have them in the same folder so there is no need to define the path. The next example shows a possible approach for this method.

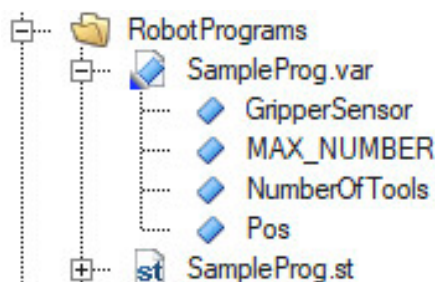


Figure 51: Variable file for robotics programs

```
#include 'SampleProg.var'

PROGRAM _MAIN

    /* Program Logic */

END_PROGRAM
```

As mentioned previously, variables can belong to three different scopes depending on where they can be accessed from. They are specified using the parameter "Storage". The following parameters can be used as a modifier for the "VAR" statement to set the scope of different variables, as previously mentioned.

- **NC_GLOBAL** variables are those shared across all interpreter instances, for example axis group variables
- **IP_GLOBAL** variables are those shared within the same instance group, for example a particular axis group variable
- **LOCAL** variables are those shared within the program file

```
VAR {IP_GLOBAL}
    gGripperSensor: BOOL;
END_VAR
```

One important thing to mention is that variables declared in files and included using "#include" are IP_GLOBAL. Therefore, they are shared between all the programs in the interpreter instance.

More information on ST motion programming can be found in Automation Help.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming\ Programming languages \ Structured Text \(ST\) \ Program content \ Variables \(IEC ST\)](#)

6.2 Commands

The motion instructions or commands are a core part of robotics programs. These elements are defined to perform all necessary basic tasks in robotic applications. They can be used to provide a wide range of functionality, including setting a feed rate for the entire program or obtaining the position of the system at a certain point in the path.

Basic motion tasks, like the one described in the previous chapter, have specific commands to perform them. Commands to perform linear and P2P movements are some of the most commonly used to define a path in the robot application. Other commands define other types of movements that can be very useful for specific applications, like circular movements or velocity movements. The following table describes some of the most commonly used.

Command	Description
MoveL	Linear movement to a specified target position
MoveLR	Rapid linear movement to a specified target position
MoveJ	Point-to-point movement to a specified target position
MoveJR	Rapid point-to-point movement to a specified target position
MoveC	Circular movement performed at any position in space

Table 2: Commonly used ST motion commands

Command	Description
MoveA	Point-to-point movement to an axis target position

Table 2: Commonly used ST motion commands

Apart from movements, many other basic functionalities commonly used in robotics are covered using commands. A lot of them are related to the use of axis group features and mapp functionalities. Some of them will be presented in upcoming chapters when different features and functions are described. The following table describes a few of them.

Commands	Description
Feed rate	Specifies the feed rate in units per minute.
WaitTime	Wait for a specified time (path-synchronous).
GetPoint	The current position in a defined coordinate system as well as joint or axes positions can be obtained during program execution.
SetTool	Selects a tool data set from the tool table.

Table 3: Commonly used ST commands

In Automation Help, it is possible to review all of these commands and find more information on their purposes and how to use them.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming\ Programming languages \ Structured Text \(ST\)](#)

6.3 Data types

All these commands require different inputs to work correctly, like for example a target position is required to trigger linear or P2P movements. mapp Robotics includes several data types that gather all these inputs and allow all these commands to be used. There are data types included to describe Cartesian positions, axes configurations, frame coordinates, orientations, transformation matrices, etc. In the following table, a sample of some of the most commonly used ones is presented.

Data type	Description
McPointType	Describes the position and orientation of the TCP with respect to the Cartesian coordinate system and the absolute positions of slave axes.
McAxisTargetType	Describes the target position of all real axes in the mechanical system.
McFrameType	Describes the transformation of a coordinate system.
McPosType	Defines the X, Y and Z coordinates for a position in Cartesian space.
McOrientType	Describes the orientation in A, B and C rotation angles and the type of rotation used to obtain them.

Table 4: Data types commonly used in robotics programs

In the help description for each command, it is possible to find which data type is required and samples showing how to properly define them.

For more information about these data types as well as a complete collection of them, see Automation Help.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming\ Programming languages \ Structured Text \(ST\) \ Predefined data types](#)

6.4 Actions

Simple and readable motion programs are important so everyone can understand their purpose and the process. As it happens in ST tasks, sometimes it is necessary to reuse some code blocks or separate them from the main logic to keep programs simple and readable. When doing this, actions can be very useful.

Actions are blocks of code that can be called from other code structures, like the main routine or external functions, to perform a specific task. For example, if an action is defined to move the robot to a certain resting position, every time we finish a process or complete a program it is possible to just use this action and the robot will go to the resting position. If this occurs several times in the same program, the number of lines will be reduced significantly.

The next two examples show how to implement actions and use them in a local and global context.

Actions in local context

```
//Action declaration
ACTION init:
    state := State#wait;
END_ACTION

PROGRAM _MAIN
    init;
END_PROGRAM
```

Actions in global context

```
#pragma SCOPE 'IP_GLOBAL'
ACTION init:
    state := State#wait;
END_ACTION

PROGRAM _MAIN
    init;
END_PROGRAM
```

Other possibilities to simplify code and reuse it are functions and function blocks. For more information about how to use them and their differences and utilities, see Automation Help.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming\ Programming languages \ Structured Text \(ST\) \ Program content](#)

Exercise: Creating robotics programs

In this exercise, participants will learn how to create, edit, process, and execute a Robotic ST program. A robotic ST program defines the movements and actions of the robot and is essential for controlling its behavior in a structured and programmable way.

Before creating the program, a file device must be set up. A file device is a predefined storage location where the ST program file will be saved—such as a user partition or an USB stick.

Once the file device is created, the ST robotics program can be saved to it. After saving, the program is executed using specific commands in mapp Cockpit.

Use the predefine Scene Viewer for this exercise. Before starting the programming task, open this scene and link all necessary variables to the robot mechanics. This step is required to correctly visualize the robot's behavior and recreate a digital twin of the system.

The steps that must be followed to execute an ST program are presented next:

- 1) Create a suitable file device on the controller. (See [Getting Started MCR Codian Delta robots](#))
- 2) Create an ST robotics program and save it in the file device (optional: add new file and change the file extension to .st).
- 3) Compile and transfer the project.
- 4) Open mapp Cockpit
- 5) Make the axis group ready for movement (power on and home).
- 6) Start the program with "MoveProgram".
- 7) While the program is running, interrupt and start it with the command "Interrupt" and "Continue".

Edit the program to perform other movements and execute the operation again.

Further information

[Motion control \ mapp Motion \ mapp Robotics/CNC \ Getting Started \ MCR Codian Delta robots \ Creating a Structured Text motion commands program](#)
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming\ Programming languages \ Structured Text \(ST\)](#)



Result:

The robot should move according to the program that has been developed. Using a "Path tracker" tool can help with analyzing the trajectory and its behavior. The image on the right shows the path performed by the robot if the provided sample programs is executed. This sample can be found in the appendix of this guide, "Exercise 7: Create robotics programs".

While running in a simulation, it is possible to use a local folder on the PC to load and edit programs. Any folder can be used, but it must be properly specified using its path when defining the file device. A very helpful approach is to use a relative path inside the project structure. Using this method, programs can be created, handled and edited using Automation Studio.

To do that and have an independent path regardless of the position of the project, a relative path from the simulation to the folder in the logical structure of the project must be used.

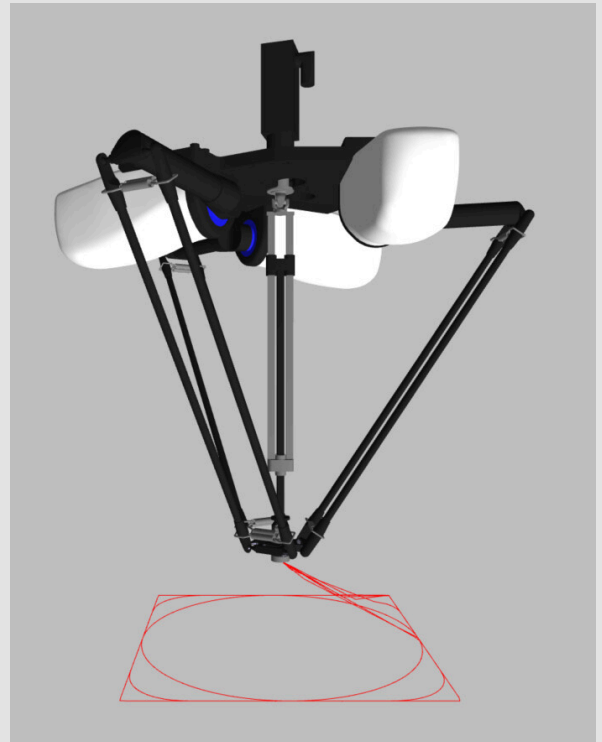


Figure 52: Sample program path trajectory

The USER_PATH can be specified as a file device. It can be used both in the simulation and on the actual hardware.

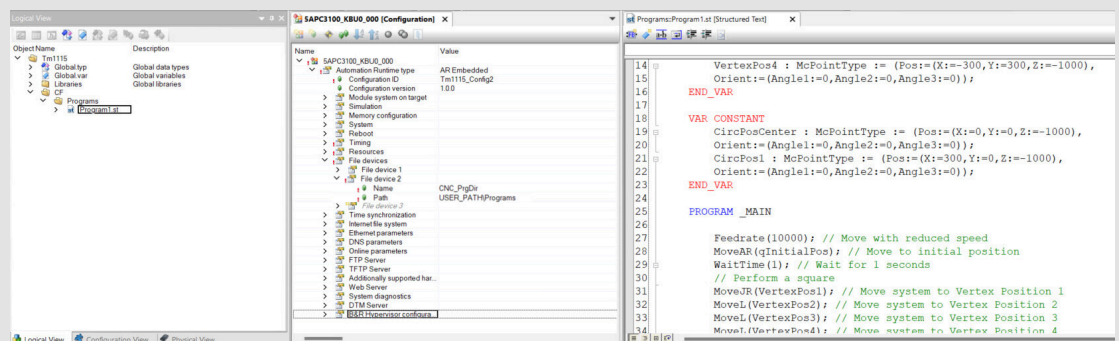


Figure 53: File Device

When transferring the application, it is possible to specify that the CF folder should be copied to the user partition.

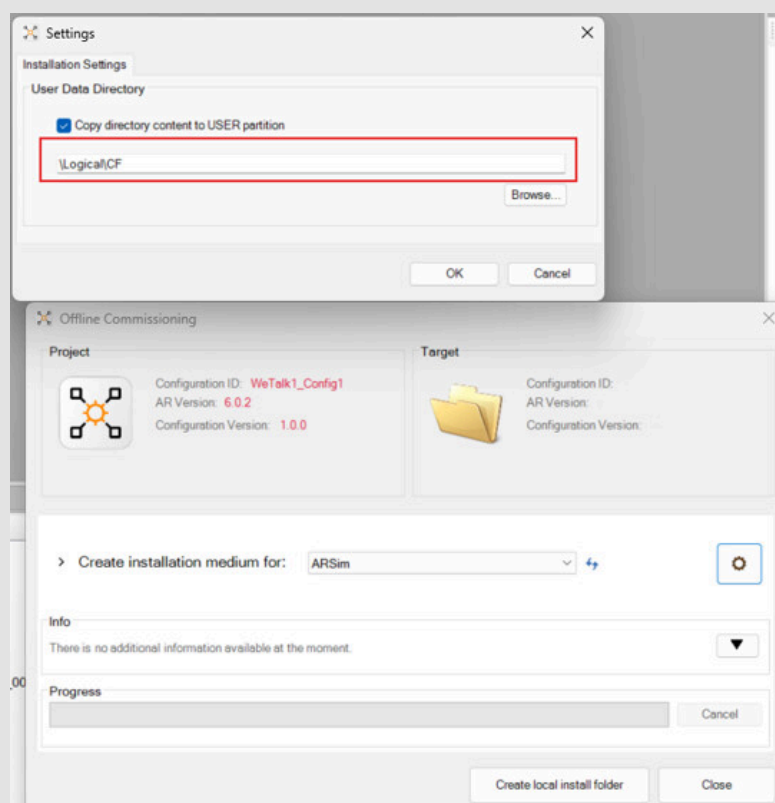


Figure 54: Transfer option for copying CF folder to USER partition

In ARSim, the user partition is created in the Temp folder located within the directory where the Automation Studio project resides: `..\<WorkingPath>\USER`.



[Automation Runtime \ Method of operation \ Module/Data security \ User partition](#)

Optional exercise: Reading the logger - Part 2

Errors in the robotics program are displayed in detail and as information in the Logger. Various error scenarios can also be simulated again here. Two examples that can be tested are syntax errors and non-executable movement commands in the ST program.

- 1) Prepare the axis group.
- 2) Create an error in the robot program, such as:
 - Position in the MCS coordinate system outside the range of the robot.
 - Syntax error in the movement command.
- 3) Open the Logger and activate the "Motion" module.
- 4) Download the Logger entries.
- 5) Analyze the Logger entries.

Further information



[Diagnostics and service \ Diagnostics tools \ Logger](#)

**Result:**

After triggering each error, it should be possible to find the corresponding entries in the Logger and analyze their information. Then after resetting via the "Reset" command and correcting the error, the program should be ready to be executed again.

The screenshot shows the 'Logger' application window. The top status bar indicates 32 Errors, 0 of 23 Warnings, 0 of 979 Informations, and 1 Success. The main table lists error entries with columns for Severity, Time, ID, Entered by, and Description. The fourth entry is selected, and its details are shown in the 'Details' pane below.

Severity	Time	ID	Entered by	Description
1 Error	2025-03-26 12:19:20.513...	-1067245568	gAxesGroup_D4r1100R_1	Command: MoveProgram failed
2 Error	2025-03-26 12:19:20.371...	-1067245459	gAxesGroup_D4r1100R_1	Program start aborted
3 Error	2025-03-26 12:19:20.371...	-1066910773	gAxesGroup_D4r1100R_1	Program failed to load Status: 15176
4 Error	2025-03-26 12:19:20.371...	-1066910932	gAxesGroup_D4r1100R_1	Failed to open file Name: Program1.st Device Name: CNC_PrgDir Status: 20708

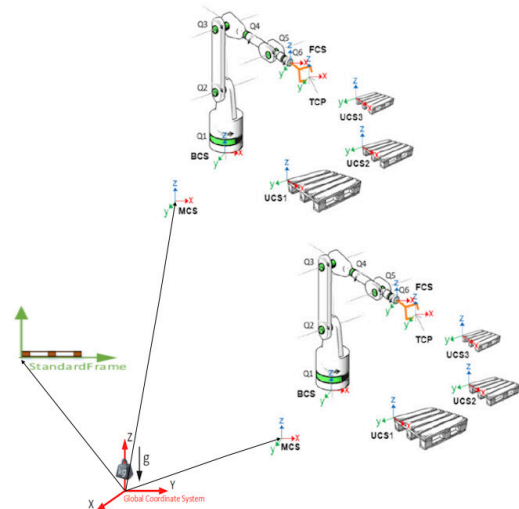
Name	Value
Severity	Error
Date	26.03.2025
Time	12:19:20.371000
ID	-1066910932
Area	B&R
Facility	104
Entered by	gAxesGroup_D4r1100R_1
Logbook	Motion
Record ID	1010
Format	EventId
Location	Online
Description	Failed to open file Name: Program1.st Device Name: CNC_PrgDir Status: 20708
ASCII Data	Program1.st ICNC_PrgDir EPP

Figure 55: Example of Logger entries

7 Coordinate systems

Coordinate systems or frames greatly simplify programming robotic systems. They are arbitrary points in space used to uniquely determine the position and the orientation of any point in Cartesian space with respect to them. In a 3D space, any coordinate system is determined by X, Y and Z positions or translation coordinates and A, B and C orientation or rotation angles.

Being able to describe a point with respect to different reference positions in space is very important in robotics. For example, when handling a product on a table, it might be interesting to define its position with respect to the center of the table.



On the other hand, when moving to other positions or moving the robot to a resting position, it might be better to define these positions with respect to the base of the robot. Defining proper coordinate systems simplifies performing complex robotics tasks.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Concept \ Range of functions \ Basic functions \ Coordinate systems](#)

7.1 Global coordinate system

When introducing the software components necessary for MCR, the configuration file "Object hierarchy" was presented. This file contains a very important definition regarding coordinate systems: the Global Coordinate System (GCS). This frame is the zero position or base frame for all the coordinate systems in the application. Any machine, robot or other mechanical system in the project will be referenced directly or indirectly with respect to this frame.

To configure this frame, it is necessary to define the position of each mechanical system in object hierarchy attribute "Global Coordinate System". Each system has to be added as a new object with the proper axis group reference and its position and orientation with respect to the GCS.

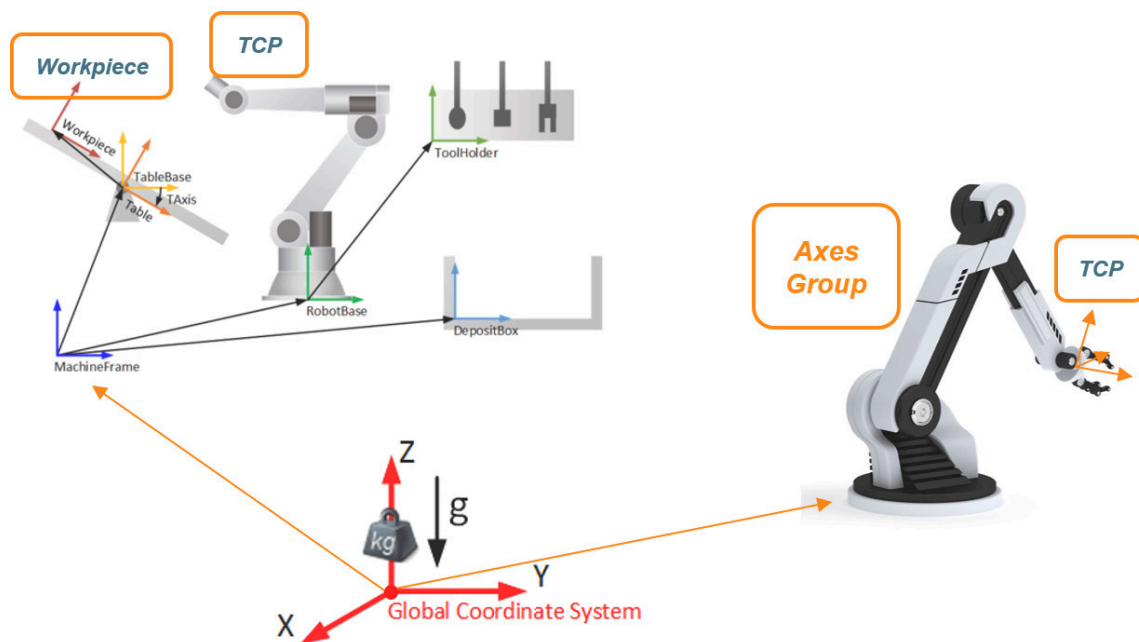


Figure 56: Example of a machine with different systems referenced with respect to the GCS

7.2 Frame hierarchy

Once the robotic system has been positioned with respect to the GCS, it might be necessary to define other frames in the system to perform various tasks. For example, in a pick and place application, it would be necessary to define the position of the products that must be picked and the position where each product should be placed. It might be also important to define the position of a tool holder so the robot can change its tool depending on the product being handled. For all of these positions, it is necessary to create different frames with a defined relationship between them, i.e. a frame hierarchy.

A **frame hierarchy** is an axis group feature that generates a structure of frames that can be used to define positions in Cartesian space when commanding robotic systems. Frames are defined relative to other coordinate systems. The first frame in the structure and its base is the Machine Coordinate System (MCS), which is introduced in the basic co-ordinate systems section. From this frame, it is possible to define different types of frames and hierarchies to include all the necessary reference points to easily perform all robotic processes.

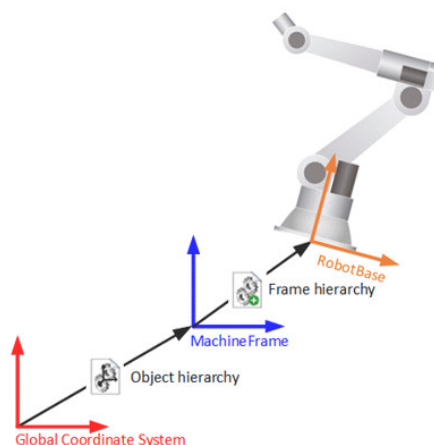


Figure 57: Relationship between the GCS, the MCS and the robot position.

There are 3 types of frames that can be configured in the frame hierarchy.

- **Standard frames** are those defined by a translation and orientation with respect to the previous frame. These are fixed frames that define new static coordinate systems in Cartesian space.
- **Frame table frames** are those selected from a table of frames with multiple translation and orientation values. By selecting different indexes, it is possible to change the position of the coordinate system.
- **Programmed moving frames** are those frames that can move according to the movement of a slave axis³. One of the translation or orientation values is linked to the position of the axis. Then the frame can either translate along the X, Y or Z coordinates or rotate around A, B or C.

There are two main types of frame hierarchies, the standard frame hierarchy and the customized frame hierarchy. They offer different possibilities for how the frames can be created and the types of frames that can be included in them.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Basic Elements \ AxesGroup feature \ Frame hierarchy](#)

7.2.1 Standard frame hierarchy

The standard frame hierarchy is a predefined structure of 8 frames. The relationship between each frame is fixed and cannot be changed. Translation and orientation values for each coordinate system with respect to its parent frame are the only parameters that can be configured in this structure. The following image shows this fixed structure:

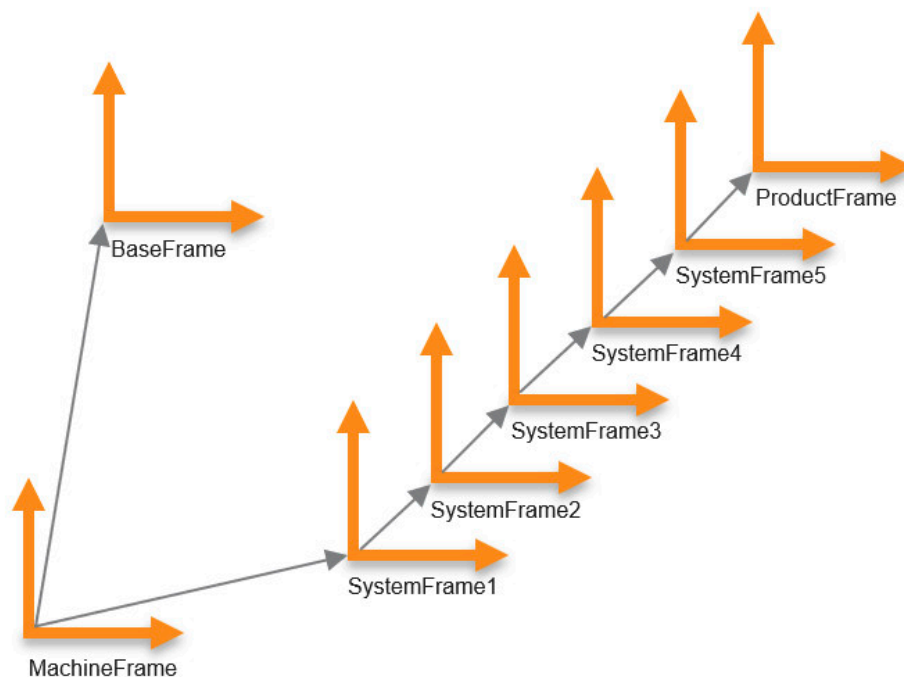


Figure 58: Standard frame hierarchy structure

This frame hierarchy represents the legacy approach to coordinate systems in previous motion systems. All coordinate systems in this structure are standard frames except for "SystemFrame3", which is a "frame table" type frame. This means that when using the standard frame hierarchy, it is only possible to use one table frame and the rest must be standard frames. There are no programmed moving frames in this structure.

³ A typical example of a slave axis is a drilling or milling head that moves in coordination with the robot's motion.

7.2.2 Customized frame hierarchy

The customized frame hierarchy is an open structure that allows configuring any possible set of frames. This structure is organized in levels and each level can have as many frames as required. A new level can then be created after each one of them as its child with any number of new coordinate systems. This structure can be increased, layer to layer, until it covers all the necessary reference systems in the application. Following this process, it is possible to generate any desired coordinate system hierarchy.

The frames can be of any type: standard, table frames or programmed moving frames. Depending on their type, different parameters will be needed to determine the position and orientation with respect to their parent frame. The following image shows a sample of a customized frame hierarchy for a specific application:

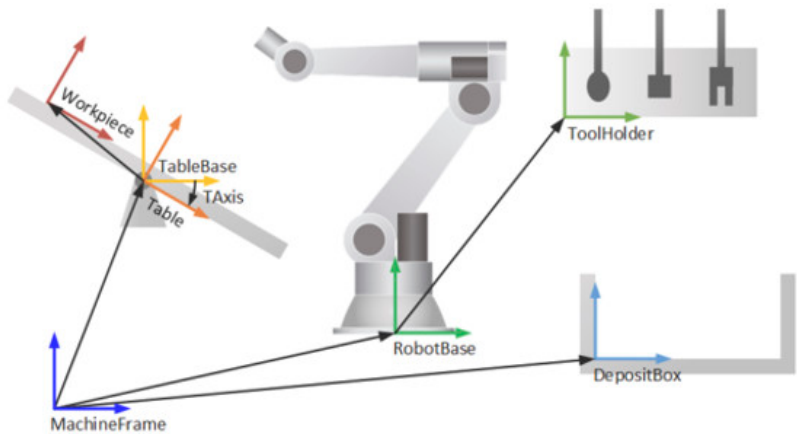


Figure 59: Customized frame hierarchy for a pick-and-place application with a robot and different tools.

The customized frame hierarchy type also includes a frame property mapping table. In this table, it is possible to set seven frames as predefined coordinate systems.

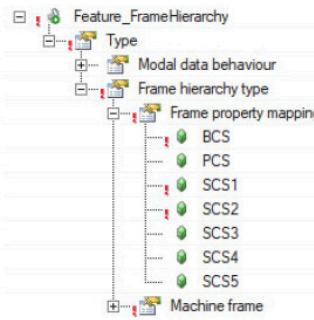
	Feature_Hierarchy		
	Type	Frame hierarchy	
	Modal data behaviour	Use axes group settings	Defines the modal data behaviour of the feature.
	Frame hierarchy type	Custom	Defines the local frame-hierarchy of an axes group
	Frame property mapping		
	BCS	RobotFrame	Name of the frame that should be mapped to BCS (mandatory)
	PCS	MachineFrame	Name of the frame that should be mapped to PCS (mandatory)
	SCS1	TableFrame	Name of the frame that should be mapped to SCS1
	SCS2	Output TableFrame	Name of the frame that should be mapped to SCS2
	SCS3		Name of the frame that should be mapped to SCS3
	SCS4		Name of the frame that should be mapped to SCS4
	SCS5		Name of the frame that should be mapped to SCS5
	Machine frame		Base of all other coordinate systems of the axes group.

Figure 60: Frame property table example.

- From these frames, two must always be defined:
- The first one is the **base coordinate system (BCS)**. It defines the position of the base of the robot. It must be linked to the frame in the hierarchy located at its base.
 - The second mandatory frame is the **product coordinate system (PCS)**. It defines the position of the product by default. This means that all default positions, if they do not specify any frame, are defined in relation to this coordinate system.
- The rest of the frames are **system coordinate systems (SCS1 to SCS5)**. They can be linked to any frame on the hierarchy and can be used to simplify programming or testing. These systems are linked to a set of constants in the application and can simplify programming movements and obtaining positions with respect to different coordinate systems. They can also be selected when defining a position while commissioning the robot with mapp Cockpit.

Parameter	Value	Unit
Position[0]	ACS	mm
Position[1]	MCS	mm
Position[2]	PCS	mm
Position[3]	SCS1	deg
Position[4]	SCS2	deg
Position[5]	SCS3	deg
Velocity	SCS4	mm/s
Acceleration	SCS5	mm/s ²
Deceleration	TCS	mm/s ²
Jerk	JACS	mm/s ³
Coordinate system	ACS	
[DEV]Buffer mode	[Dev]Aborting	
[DEV]Transition mode	[Dev]None	
[DEV]Transition parameter	0.000000	

Move direct absolute ►

Figure 61: Using coordinate systems in mapp Cockpit commands



[Motion control \ mapp Motion \ General information \ Programming \ Application program \ Libraries \ Core libraries \ McBase \ Data types and constants \ Enumerators \ McCoordinateSystemEnum](#)

7.3 Coordinate systems in Scene Viewer

Scene Viewer can also be used to display the frame hierarchy in the digital twin. Using the measurement object "Frame", all coordinate systems can be included in the simulation. The frame hierarchy can be properly included in the simulated scenario by defining its relative position and orientation with respect to the parent frames. This simulation can be helpful to understand the system and simplify programming of certain robotic tasks.

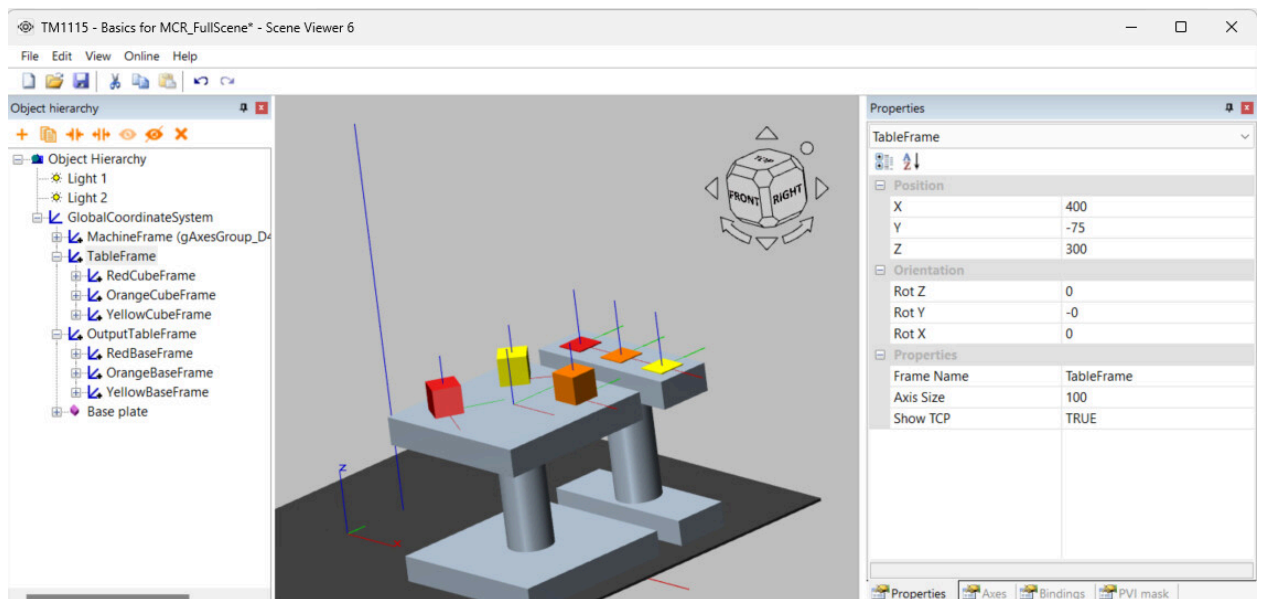


Figure 62: Frame properties in Scene Viewer

There is another measurement object, the "Positioner", that is very useful to check the position of the robot with respect to different frames in the hierarchy. This tool can be placed at the TCP of the robot and by changing its property "FrameName" to the name of the different "Frame" objects in the scene, it is possible to know the position of the robot with respect to each one of them.

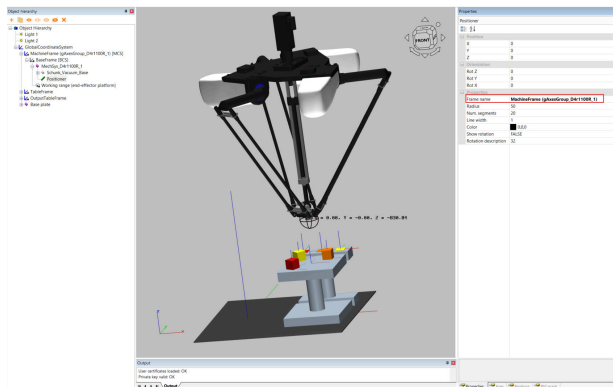


Figure 63: Position of the TCP with respect to the MCS

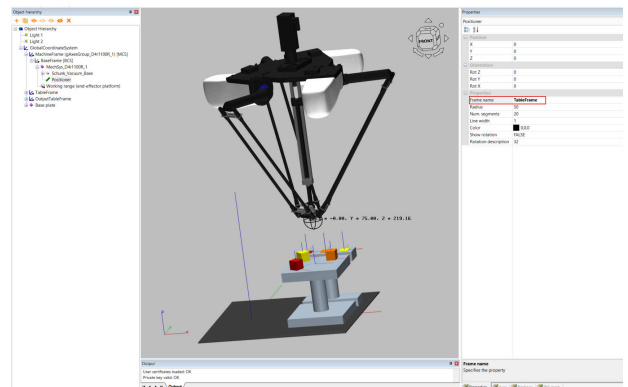


Figure 64: Position of the TCP with respect to the TableFrame

For the particular case of programmed moving frames, it is possible to bind the position or orientation of a frame to a certain application variable, making it possible to simulate rotating tables or conveyors. Not only the position of the frame itself will change but also the position of all its child frames and objects. This can be a very powerful tool to simulate complex robotic processes.

Exercise: Adapting software to the digital twin

This exercise shows how to select and configure the object hierarchy for a specific task. The first step is to identify the different frames in the scene. Next, it is necessary to find the relationship between these frames to define the correct hierarchy between them in the object hierarchy. After that, it is possible to move the robot to any position in space with respect to any of the frames.

- 1) Identify all frames in the given scene
- 2) Define the relationship between all frames in a tree of coordinate systems in the object hierarchy.
- 3) Create a new File device (like the CNC_PrgDir) and use it for Scene Viewer- File device in the object hierarchy.
- 4) Compile and transfer projects.
- 5) Open the automatic generated Scene Viewer and compare it to the existing one.

Perform tests with individual movement commands for each of the coordinate systems and check whether the tree frame is set correctly.

Further information



[Motion control \ mapp Motion \ General information \ Configuration \ Basic elements \ Object hierarchy](#)

**Result:**

The object hierarchy for the proposed scene in the figures shown above this exercise looks like following:

Hierarchy:objecthierarchy [Object Hierarchy Configuration]				
Name	Value	Unit	Description	
Object-Hierarchy				
Measurement Units	Cardan		Defines the measurement units	
Rotation description			Defines the description of the rotation	
Frame Transformations			Defines frame transformations properties	
Scene Viewer			Defines Scene Viewer export settings	
File device	FileDeviceSceneViewer		File device where the Object-Hierarchy Scene Viewer file will be created	
Global Coordinate System			Systemwide reference frame	
gAxesGroup_D4r1100R_1				
Type	Axesgroup component		Object type	
Axesgroup reference	gAxesGroup_D4r1100R_1			
Translation			Translation parameters	
X	400	measurement...	Translation in X direction	
Y	0.0	measurement...	Translation in Y direction	
Z	1350	measurement...	Translation in Z direction	
Orientation			Orientation parameters	
TableFrame				
Type	Standard frame		Object type	
Frame name	TableFrame		Standard frame name	
Translation			Translation parameters	
X	400	measurement...	Translation in X direction	
Y	-75	measurement...	Translation in Y direction	
Z	300	measurement...	Translation in Z direction	
Orientation			Orientation parameters	
RedCubeFrame				
OrangeCubeFrame				
YellowCubeFrame				
Type	Standard frame		Object type	
Frame name	YellowCubeFrame		Standard frame name	
Translation			Translation parameters	
X	-80	measurement...	Translation in X direction	
Y	80	measurement...	Translation in Y direction	
Z	0.0	measurement...	Translation in Z direction	
Orientation			Orientation parameters	
Angle 1	0	measurement...	Rotation around the first coordinate axis of the rotation order	
Angle 2	0.0	measurement...	Rotation around the second coordinate axis of the rotation order	
Angle 3	45	measurement...	Rotation around the third coordinate axis of the rotation order	
OutputTableFrame				
Type	Standard frame		Object type	
Frame name	OutputTableFrame		Standard frame name	
Translation			Translation parameters	
X	400	measurement...	Translation in X direction	
Y	175	measurement...	Translation in Y direction	
Z	300	measurement...	Translation in Z direction	
Orientation			Orientation parameters	
RedBaseFrame				
Type	Standard frame		Object type	
Frame name	RedBaseFrame		Standard frame name	
Translation			Translation parameters	
X	-100	measurement...	Translation in X direction	
Y	0.0	measurement...	Translation in Y direction	
Z	0.0	measurement...	Translation in Z direction	
Orientation			Orientation parameters	
OrangeBaseFrame				
YellowBaseFrame				

Figure 65: Object hierarchy for the proposed scene

In detail the Translation and Orientation will be configured as follows:

gAxesGroup_D4r1100R_1

- Type = Axesgroup component
- Axesgroup reference: gAxesGroup_D4r1100R_1
- Translation = (X=400mm, Y=0mm, Z=1350mm)
- Orientation = (Angle1=0°, Angle2=0°, Angle3=0°)

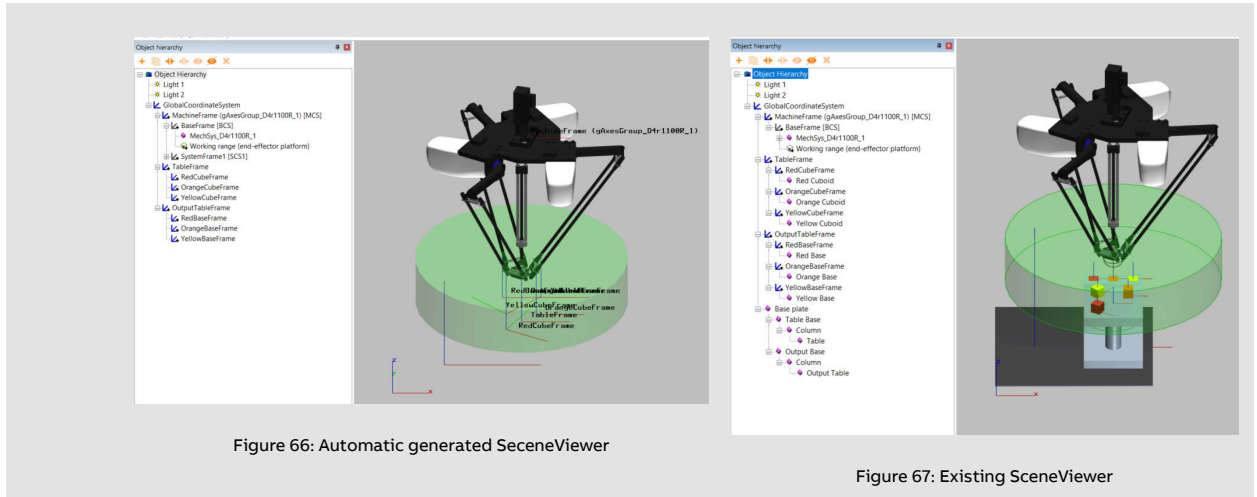
Table Frame

- Type = Standard frame
- Frame name: TableFrame
- Translation = (X=400mm, Y=-75mm, Z=300mm)
- Orientation = (Angle1=0°, Angle2=0°, Angle3=0°)
- **RedCubeFrame**
 - Type = Standard frame
 - Frame name: RedCubeFrame
 - Translation = (X=-80mm, Y=-80mm, Z=0mm)
 - Orientation = (Angle1=0°, Angle2=0°, Angle3=-30°)
- **OrangeCubeFrame**
 - Type = Standard frame
 - Frame name: OrangeCubeFrame
 - Translation = (X=75mm, Y=75mm, Z=0mm)
 - Orientation = (Angle1=0°, Angle2=0°, Angle3=0°)
- **YellowCubeFrame**
 - Type = Standard frame
 - Frame name: YellowCubeFrame
 - Translation = (X=-80mm, Y=80mm, Z=0mm)
 - Orientation = (Angle1=0°, Angle2=0°, Angle3=45°)

OutputTableFrame

- Type = Standard frame
- Frame name: OutputTableFrame
- Translation = (X=400mm, Y=175mm, Z=300mm)
- Orientation = (Angle1=0°, Angle2=0°, Angle3=0°)
- **RedBaseFrame**
 - Type = Standard frame
 - Frame name: RedBaseFrame
 - Translation = (X=-100mm, Y=0mm, Z=0mm)
 - Orientation = (Angle1=0°, Angle2=0°, Angle3=0°)
- **OrangeBaseFrame**
 - Type = Standard frame
 - Frame name: OrangeBaseFrame
 - Translation = (X=0mm, Y=0mm, Z=0mm)
 - Orientation = (Angle1=0°, Angle2=0°, Angle3=0°)
- **YellowBaseFrame**
 - Type = Standard frame
 - Frame name: YellowBaseFrame
 - Translation = (X=100mm, Y=0mm, Z=0mm)
 - Orientation = (Angle1=0°, Angle2=0°, Angle3=0°)

The following picture shows on the left side the automatic generated SceneViewer and on the right the already existing one. The missing components, like the Table or the Cubes, can be added manually. The Frames, which are important for the programming are identically.



If the automatic Scene Viewer generation is used, the generated scene file must be copied to a different location. Otherwise, the file will be overwritten the next time the simulation is restarted, and all changes will be lost.

7.4 Using coordinate systems in robotics programs

Coordinate systems can also be used and modified in ST robotic programs. Some ST commands can change the coordinate system that will be used to perform movements in the program.

These commands can become very handy when the application requires performing the same exact process for different products or at different positions on the product. For example, if it is necessary to drill a set of holes in several products placed on a table, it is only necessary to program one task with respect to the center of the product and then switching between product frames with these commands allows all products to be drilled identically. This feature really simplifies the number of positions required and therefore the development effort.

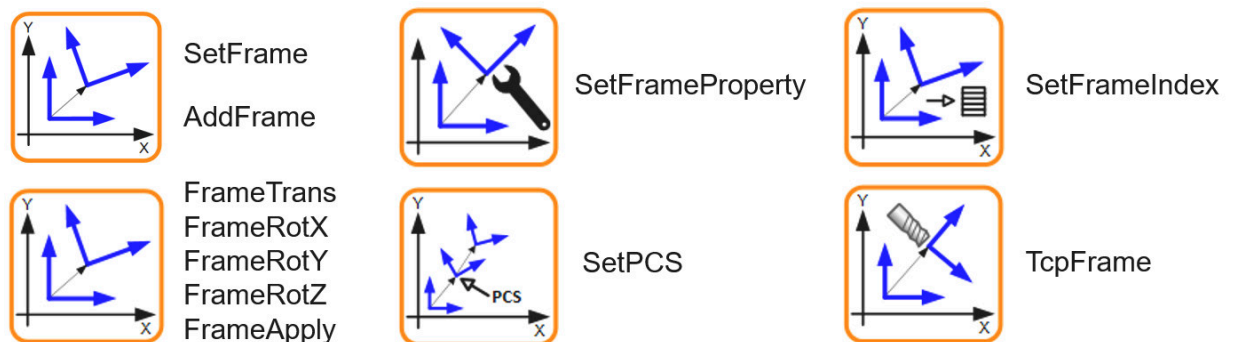


Figure 68: ST commands to handle coordinate systems

Aside from these commands, there are many other instructions that can help when generating new frames inside an ST program. Using them, it is possible to translate and rotate existing frames as well as to create new ones inside the frame hierarchy.

For more information about these instructions and how they can be used, see AS Help.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Programming language \ Structured Text \(ST\) \ Conversion and transformation](#)

Exercise: PCS programming

This exercise shows how to change the product coordinate system when running a robotics program. It depicts how the frames declared in the frame hierarchy can be used to perform the same task on different targets. By changing the PCS during operation, the same movement commands are executed with respect to different coordinate systems.

- 1) Define a new empty ST robotics program and add it to the file device.
- 2) Define two positions above each product. One just on top of them and the other one a safe distance above it (e.g. 100 units).

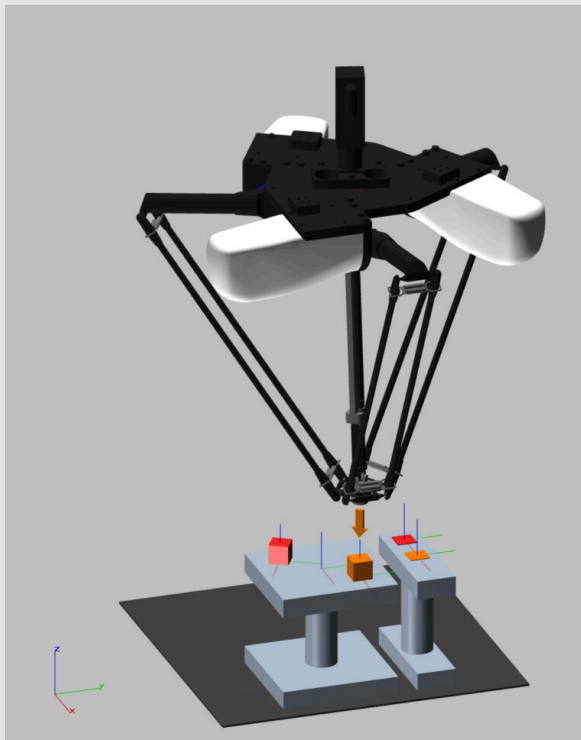


Figure 69: Pick movement for the Orange Cube

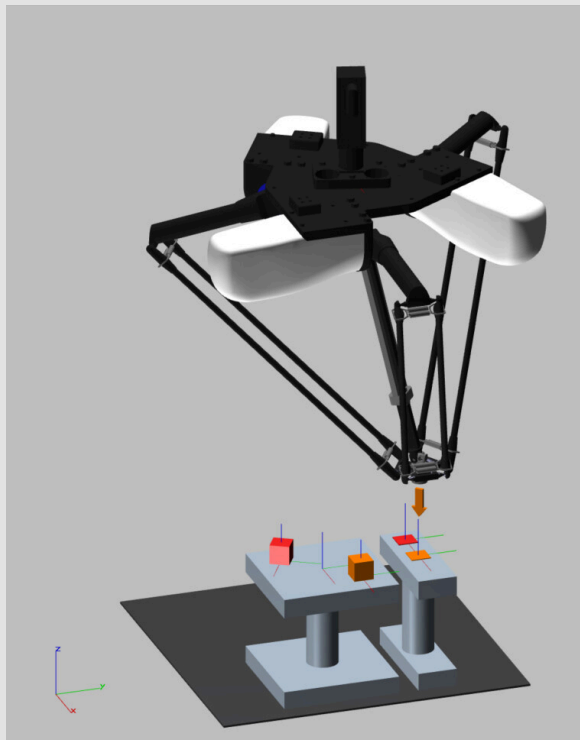


Figure 70: Place movement for the Orange Cube

- 3) Develop a program to pick the cubes and place them in the base that has the same color. Use rapid movements to move between the safe positions and linear movements to approach the picking and placing positions.
- 4) Execute the program using mapp Cockpit.

Further information



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Programming language \ Structured Text \(ST\) \ Conversion and transformation](#)
[Motion control \ mapp Motion \ General information \ Configuration \ Basic elements \ Object hierarchy](#)

**Result:**

The following code is a suggested solution to perform a pick-and-place process for each cube.

VAR

```
InitialPos : McAxisTargetType
            := (JointAxis:=[0, 0, 0, 0, 0, 0]);
Position : McPointType := (Pos:=(X:=0,Y:=0,Z:=50),
                          Orient:=(Angle1:=0, Angle2:=90, Angle3:=0));
PrePosition : McPointType := (Pos:=(X:=0,Y:=0,Z:=150),
                              Orient:=(Angle1:=0, Angle2:=90, Angle3:=0));
```

END_VAR

PROGRAM _MAIN

```
Feedrate(2000); // Move with reduced speed

MoveAR(InitialPos); // Move to initial position

// Get Ready --> Go to Table Frame PrePosition
SetPCS(OH::TableFrame); // Set "TableFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition

(* Pick Orange Cube *)
SetPCS(OH::OrangeCubeFrame); // Set "OrangeCubeFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
WaitTime(0.5); // Simulate gripper closing
MoveL(PrePosition); // Move system to point PrePosition

(* Place Orange Cube *)
SetPCS(OH::OrangeBaseFrame); // Set "OrangeBaseFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
WaitTime(0.2); // Simulate gripper opening
MoveL(PrePosition); // Move system to point PrePosition

(* Pick Red Cube *)
SetPCS(OH::RedCubeFrame); // Set "RedCubeFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
WaitTime(0.5); // Simulate gripper closing
MoveL(PrePosition); // Move system to point PrePosition

(* Place Red Cube *)
SetPCS(OH::RedBaseFrame); // Set "RedBaseFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
WaitTime(0.2); // Simulate gripper opening
MoveL(PrePosition); // Move system to point PrePosition

WaitTime(1); // Wait for 1 second

MoveAR(InitialPos); // Move back to initial position
```

END_PROGRAM

8 Path planning

Path calculation is the core functionality for a path-controlled movement. This takes place in the motion chain and cyclically generates position setpoints for the path-controlled axes of the axis group from movement commands. Path calculation is performed in steps and can be broken down into several functional units, which will be dealt with in more detail later.



Figure 71: Programs, commands or FB cause an axis movement

The path calculation is used to generate movements performed simultaneously by multiple axes. Path calculation is executed centrally on the controller.

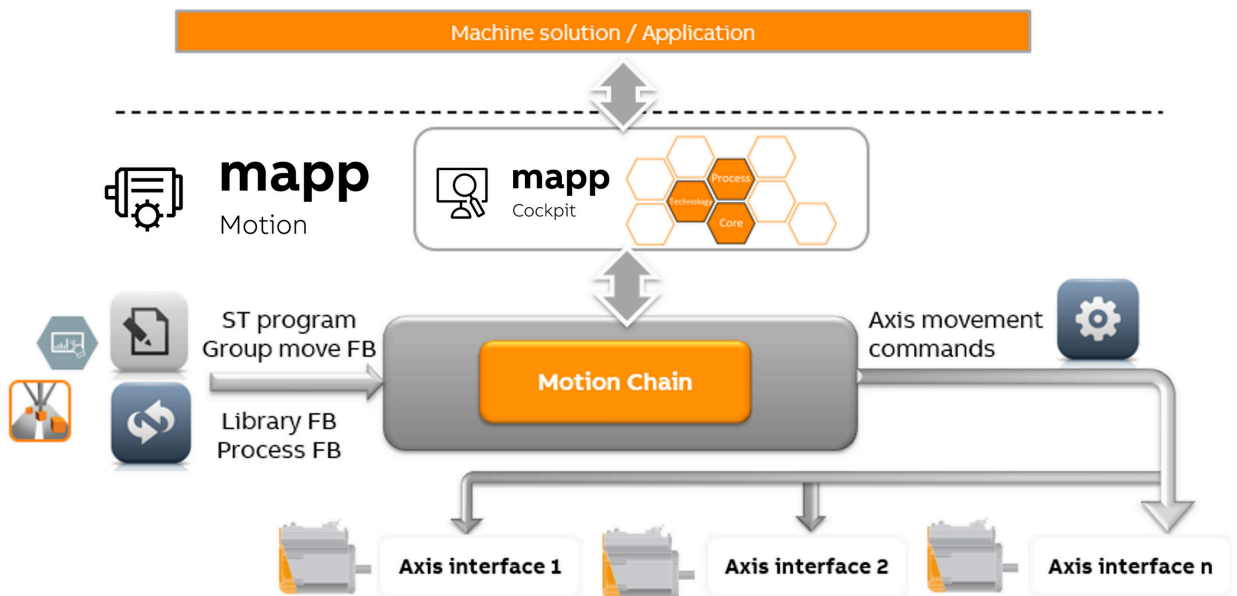


Figure 72: Figure 19: Overview of possible interfaces for motion control

Generation of the path is based on function blocks that can be configured to issue motion commands with or without a robot program. These motion commands are processed in the background and then position setpoints are sent via the network to the individual drives at defined intervals. The path calculation controls movements in Cartesian space⁴, which execute a robot movement.

8.1 Function units

The path calculation is predictive and runs in the background (idle time); only the set value sampler runs cyclically and generates the setpoints for the axes.

4 Orthogonal coordinate system with X, Y and Z coordinates

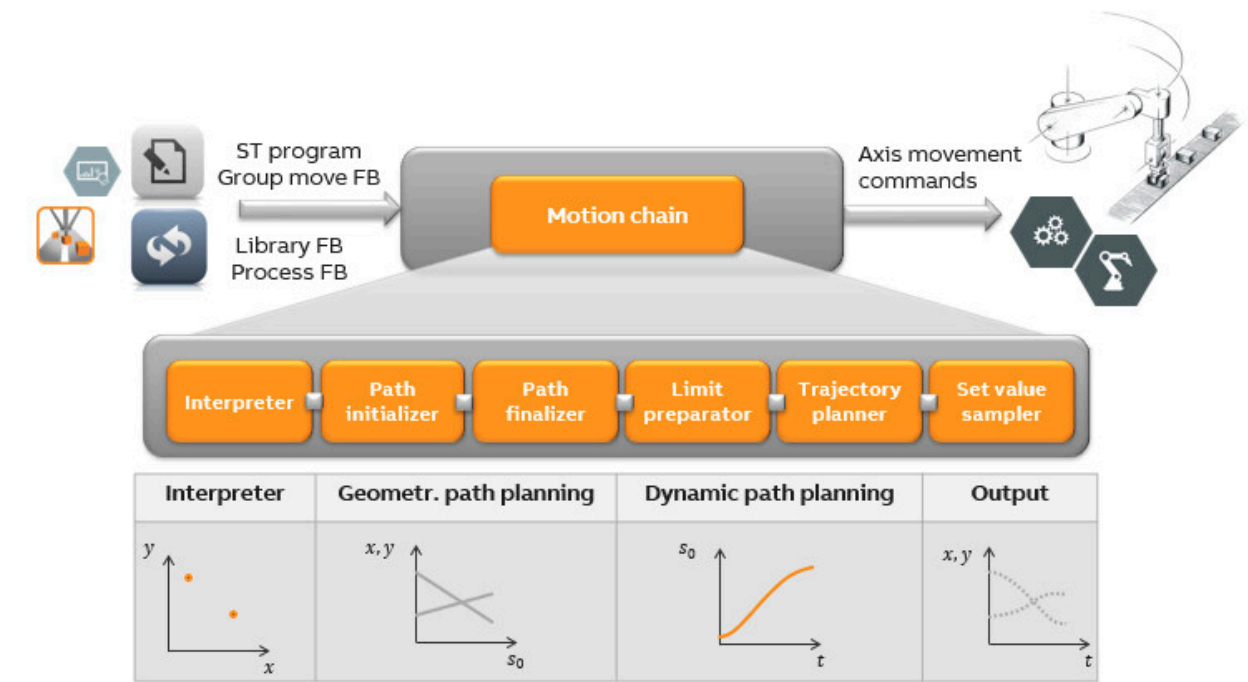


Figure 73: Division of the motion chain into the individual modules

Function unit	Task
Interpreter	Interprets the robot program and generates motion commands from it
Geometric path planning - Path initializer - Path finalizer - Limit preparator	The geometric path and the associated path information are generated from the movement commands
Trajectory planning	Calculation of the path speed profile
Set value sampler	Generation of cyclic setpoints for individual axes

[Motion control \ mapp Motion \ mapp Robotics/CNC \ Concept \ Technical Information \ Path-controlled movements](#)

All function units that process data leading up to the set value sampler are referred to as "background processing" for the path calculation. Their task is to prepare the path. They only run during the CPU idle time (no cyclic tasks are active). The speed of path preparation therefore depends on how heavily the CPU resources are being utilized by cyclic tasks.

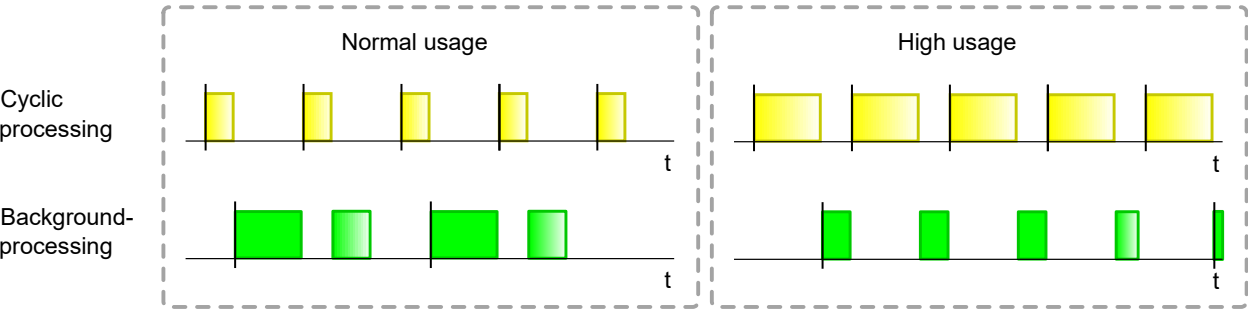


Figure 74: Effect of CPU usage on background processing



If the CPU load on the PLC is too high, unwanted path movement standstills may occur!

Functionality of the motion chain

Motion packets transport data through the motion chain and contain movement and non-movement related information.

The necessary modules are inserted into the motion chain depending on which axis group features are enabled. They are used to process and extend the motion packets.

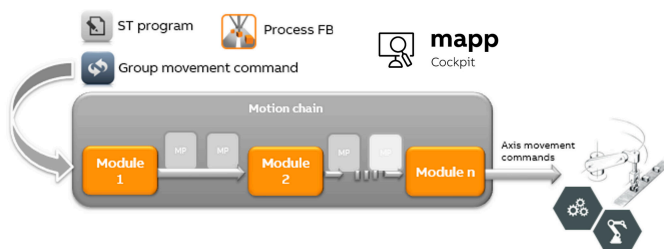


Figure 75: Information from the motion chain is constantly expanded, but not changed.

Interpreter

The interpreter reads robot programs and compiles the ST code they contain or the movement commands in the motion packets for further processing in the system. The language used (e.g. Structured Text motion commands or G-code dialect) is configurable and can be adapted to different program dialects.

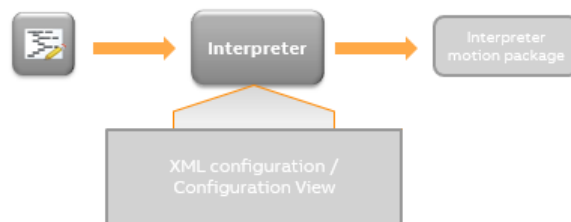


Figure 76: Interpreter

Path initializer

A path with its own master axis reference (sigma axis) is now calculated from the movements created by the interpreter. At this point in time, no axis movements and dependencies have been defined.

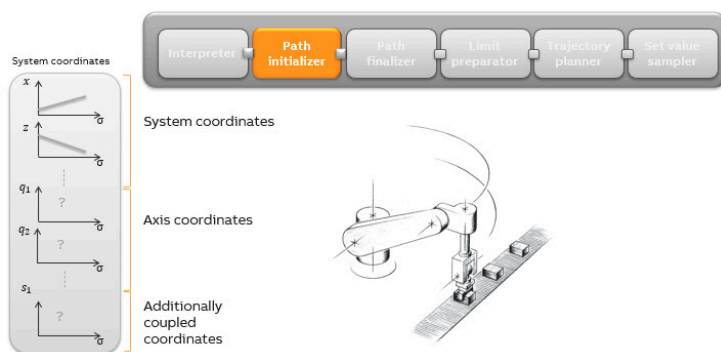


Figure 77: First the path is calculated; no axis movements yet

Path finalizer

The corresponding axis movements and path progress are calculated based on the path information. Tool dimensions and kinematics are taken into account here.

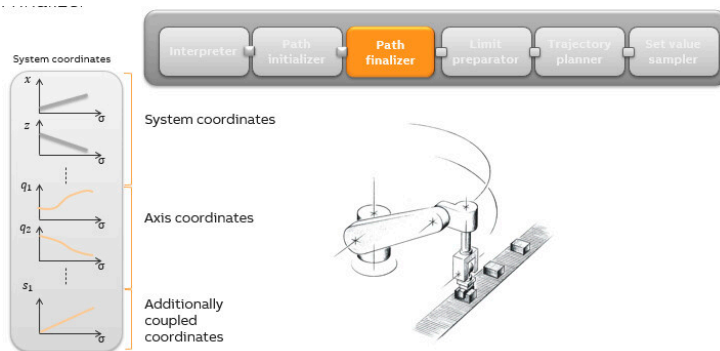


Figure 78: From this point on, the axis values are also calculated

Limit preparator

All configured and program-dependent limits are taken into account:

- Speed limit values (axes, path velocity)
- Acceleration limit values (axes, path acceleration)
- Jerk limit values (axes, path acceleration)
- Torque limit values (axes, motor parameters, dynamic model)

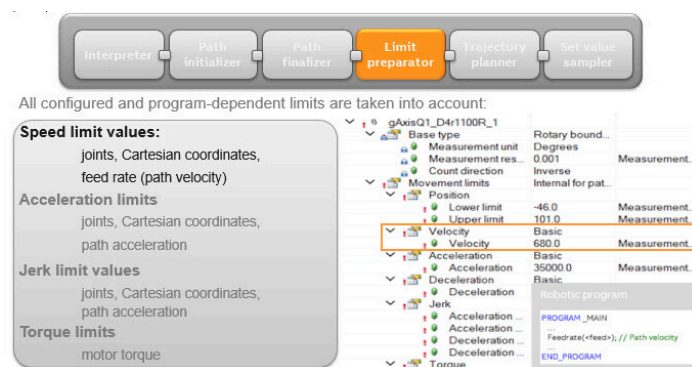


Figure 79: At this point, various limit values are taken into account

Trajectory planner

The maximum possible velocity profile is determined from the geometric path and the limit values (blue line). In addition, a braking trajectory can be calculated at any time in order to be able to stop on the geometric path at any time while observing all limit values (red lines).

Stopping can occur due to excessive CPU load and the missing motion packets caused by this.

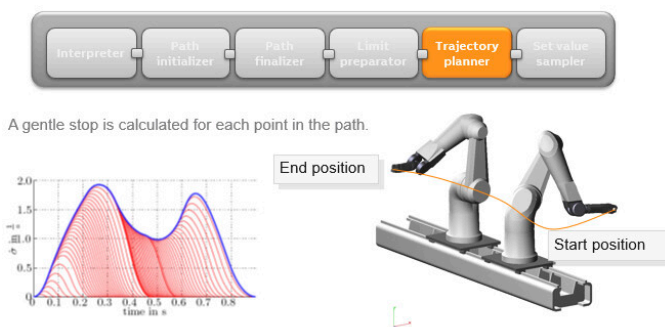


Figure 80: Planning of the complete path incl. controlled stop profiles



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Concept \ Technical Information \ Maximum stop trajectory length](#)

Set value sampler

This module evaluates the prepared motion profile cyclically and transfers the position setpoints to the axes. It also provides a wide variety of monitor data.

To do this, it uses the data it receives from the "background processing" functional units of path generation.

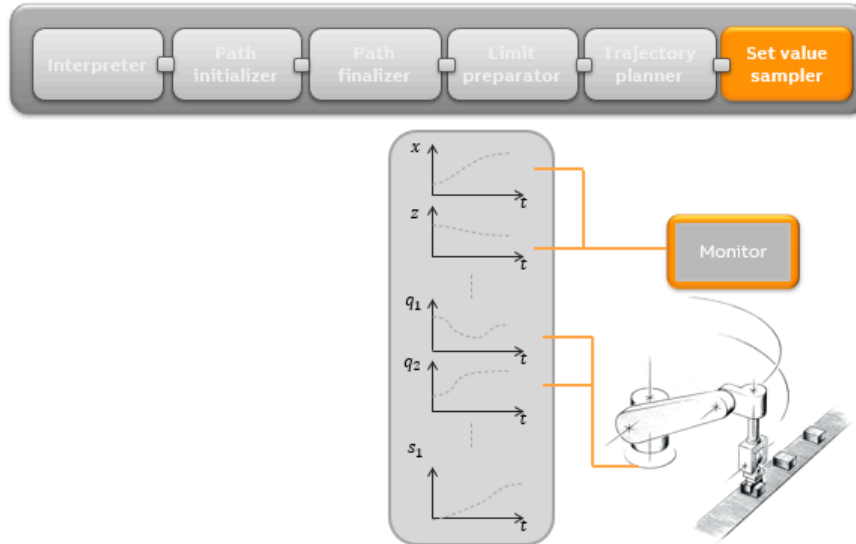


Figure 81: Last module in the motion chain



[Motion control \ mapp Robotics/CNC \ Configuration \ Basic elements \ Axes Group \ PathGen Axes-Group](#)

8.2 Closed-loop control concepts

Setpoint generation

For a path-controlled movement, however, the position setpoints are generated by the controller's set value sampler and sent to the drive as cyclic position setpoints.

To traverse the path as precisely as possible and avoid errors in controlling the axes, a tolerance time of 0 must be set for task class #1. This is the task class in which the positions are transferred.

Path-controlled movements involve the connection of several axes in a common movement, defined by one path. The complexity of the application does not increase in proportion to the number of axes.

The performance requirements of the PLC do increase, however, because it is there that the path must be planned and calculated. The faster you set task class #1, the more position setpoints will be sent to the drives and the more precisely the path will be traversed. This also increases the CPU load on the PLC, however.

The ACOPOS generally operates at 400 μ s (different settings are possible), so it does not make much sense to calculate the path any faster than this. If calculations are performed slower, interpolation occurs automatically.

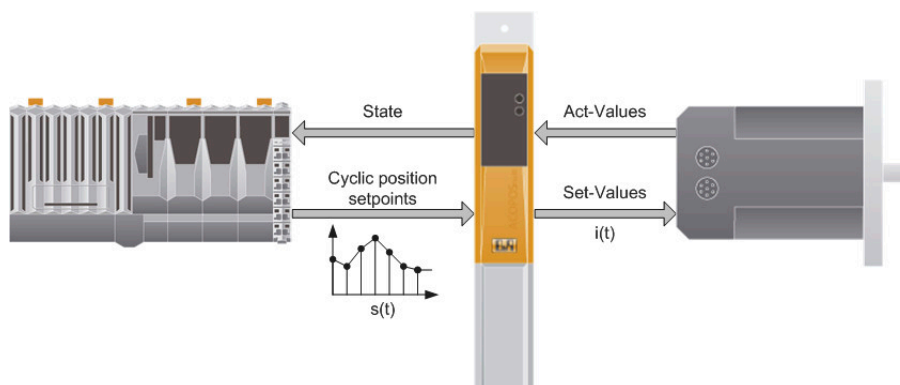


Figure 82: Path-controlled movement – Cyclic setpoint transfer



The shorter the interval selected between position setpoints, the greater the precision with which the drive can execute the path-controlled movement. However, this places high demands on the controller's necessary computing power and the network speed.



[Motion control \ mapp Motion \ mapp Axis \ Concept \ Motion control \ control concepts \ Setpoint generation](#)

Feed-forward control

To reduce lag errors and thus ensure greater path precision, torque setpoints can also be calculated for the position setpoints during centralized setpoint generation. These torque setpoints are then used for feed-forward control on the different drives in a path-controlled axis group.

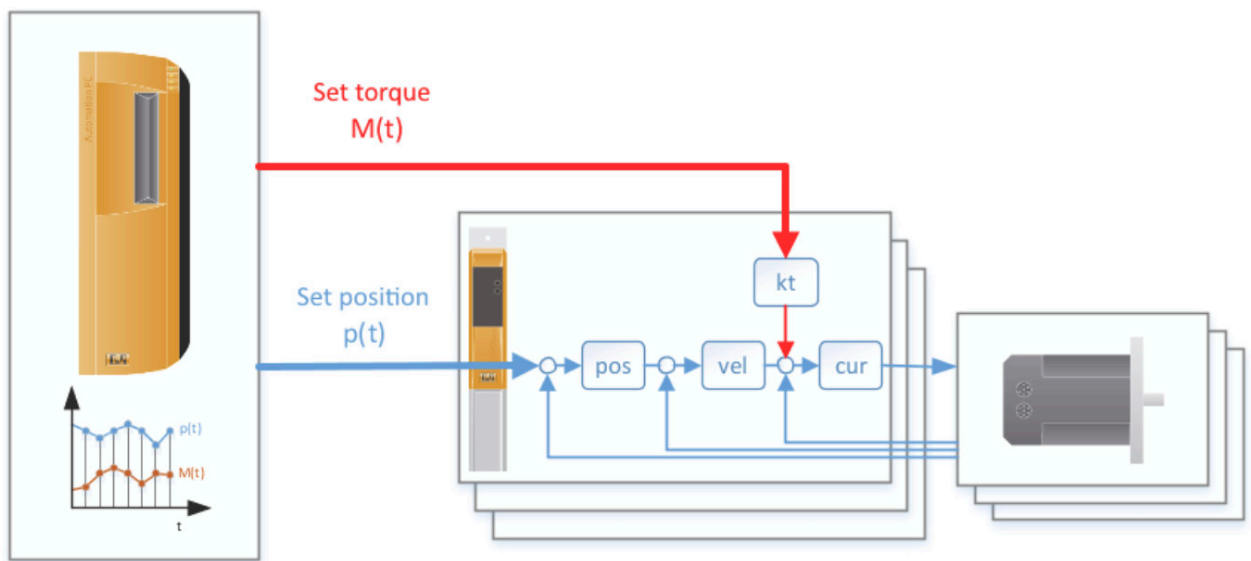


Figure 83: Feed-forward control

Unlike feed-forward control for individual axes, the reciprocal influence of the individual axes in the axis group must be calculated using a suitable dynamic model for the mechanical system. Dynamic models are provided for specific mechanical systems and can be configured using application-specific dynamic parameters such as dimensions, weights and friction coefficients. A key component of MCR are pre-configured dynamic models that enable precise feedforward control torques.



[Motion control \ mapp Motion \ mapp Axis \ Concept \ Motion control \ control concepts \ Feed-forward control](#)
[Motion control \ mapp Robotics/CNC \ Configuration \ Basic elements \ AxesGroup feature \ Feed forward](#)

9 Interaction with tools and loads

Tools allow robots to perform different tasks: drilling, grasping, injecting, etc. Integrating them into the mechanics of the robot is a very important feature in robotic systems. The tools used to perform each task can vary greatly in shape and size. This would make working with robotic programs extremely unpractical because every time a different tool needs to be used, the program should be reworked to fit the new mechanics of the system. By using the feature for tools, movements are automatically adapted to compensate for the different dimensions and characteristics of each tool configured.

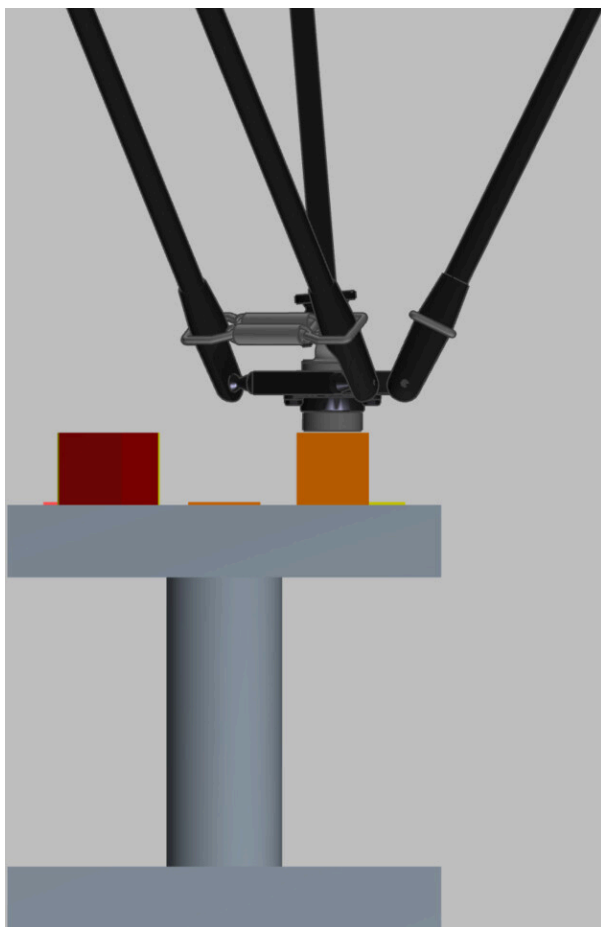


Figure 84: No tool SceneViewer model

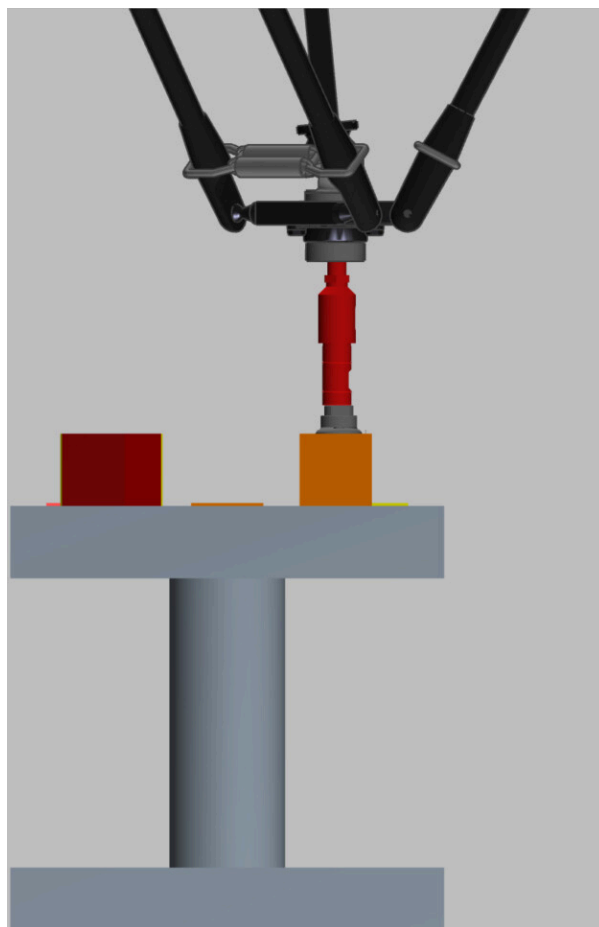


Figure 85: Schunk vacuum gripper SceneViewer model

As you can see in the images above, two different tools are used to reach the same position. Due to the differences between each tool's geometry, the robot ends up in a different configuration to reach the same position. This example helps understand how to use and the potential of the tools feature.

In this section, we will discuss how to use tools in our robotics system as well as their parameters and configurations.

9.1 Tool definitions

Adding a tool to the flange of the robot modifies some of its characteristics. The geometry of the robot is changed because a new element is included in its mechanical structure. The position of the TCP is modified and so the inverse and direct transformations of the system change. Changing the geometry of the robot also changes its working space. The range of positions that can be reached by the TCP and therefore the area of the space that the robot can move through are modified.

The dynamic behavior of the robot is also affected. Adding a tool adds a weight at the end of the mechanical structure. This changes the whole dynamic model of the system and therefore the joints of the robot need to compute different torques to move correctly. For applications that need high acceleration and deceleration, the dynamics of the tool can be a very relevant parameter.

Another relevant characteristic of a tool is its shape, i.e. its wireframe model. This property describes the 3D model of the tool in order to be able to monitor the whole robot and prevent collisions. Adding tools of course adds more complexity when detecting collisions with other objects and the robot itself. Therefore, its necessary to include the tool's model in the system. This concept is covered in the section "Workspace monitoring".

Tools are included in the axis group using the axis group feature “Tools”. This feature defines the tool or tools that can be used by each axis group. There are two possibilities when defining them:

- The **single tool configuration** only links one single tool to the axis group. The geometric and dynamic parameters of this tool are directly linked in the axis group feature. Using these configurations, it is not possible to link a wireframe model for it.

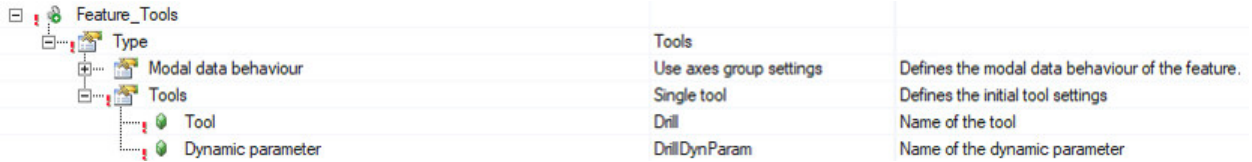


Figure 86: Single tool configuration.

- The **tool table configuration** allows linking a table of tools to the axis group. To use this possibility, it is necessary to create a "tool table" and link it to this feature. The table can contain several tool definitions, each one with its geometric, dynamic and wireframe model parameters. Using function blocks or ST commands, it is possible to switch between tools by changing the index of the tool used.

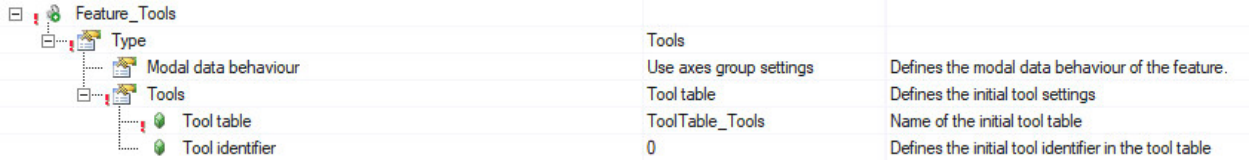


Figure 87: Tool table configuration.

9.2 Tool geometry

The tool geometry is very important as it shows the translation and rotation that a tool adds to the TCP position. "Tool geometry" is the file necessary to define all these parameters. All possible parameters that can be configured are presented next:

- The **translation** defines the new position of the TCP after adding the tool. This position is defined as the X, Y and Z translation from the tool flange of the robot (previous position of the TCP) to its tip, which is the new position of the TCP.
- The **orientation** defines the new rotation of the TCP after adding the tool. This orientation is defined as the A, B and C rotation angles from the previous orientation of the TCP to its new orientation.
- The **length** is used to configure the length of the tool. It is measured from a reference frame at the tool flange of the robot to the tool tip.
- The **radius** is used to configure the radius of the tool. It is used for cutter diameter compensation.
- The **virtual tool tip** defines the position of the tip of a rotating tool, like a drill.

Tool_Geometry			
Tool geometry description	General		Description of the tool geometry
Translation			Translation parameters
X	0.0	measurement units	Translation in X direction
Y	0.0	measurement units	Translation in Y direction
Z	0.0	measurement units	Translation in Z direction
Orientation			Orientation parameters
Angle 1	0.0	measurement units	Rotation around the first coordinateaxis of the rotation order
Angle 2	0.0	measurement units	Rotation around the second coordinateaxis of the rotation order
Angle 3	0.0	measurement units	Rotation around the third coordinateaxis of the rotation order
Radius	0.0	measurement units	Radius
Length	0.0	measurement units	Length
Virtual tool tip	Position 0		Defines the position of the virtual tool tip

Figure 88: Tool geometry configuration example.

Normally in robotic applications, the only necessary parameters to define a tool are the translation and the rotation, because the rest of the parameters suit other types of applications. It is possible to filter them out by selecting the "Tool geometry description" robot.

9.3 Using tools in robotics programs

Tools can also be changed during program execution using some ST robotic motion commands. These commands can be used to change the index of the tool used when using a table of tools or to change some parameters of the tool while the program is running.

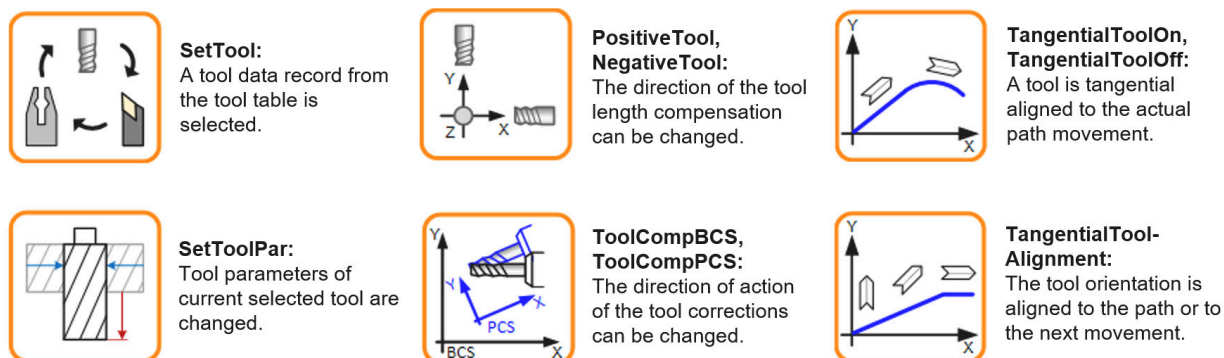


Figure 89: ST commands to handle tools

This possibility can be a very important feature in robotic applications that require the use of several tools during one process. For example, this can be a process in which a product must be grasped and moved to a particular table, then some mechanical tasks need to be performed and it has to be moved back to its original position. Being able to perform such a process without stopping the program to modify the geometric or dynamics properties of the tool is very helpful and reduces programming effort by developers.

For more information about these instructions and how they can be used, see AS Help.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Programming languages \ Structured Text \(ST\) \ Technology](#)

Exercise: Tools in the robotics program (ST)

In this exercise, two different tools are configured and defined in the project so they can be used when running a robotics program.

Each tool modifies the TCP coordinates of the robot with respect to its tool flange. A tool dimensional drawing shows the relationship between the physics of the tool and how the TCP is modified when it is placed on the tool flange of the robot. The tool data must be correctly transferred from the dimensional drawings to the tool configuration.

Tools are configured inside mapp Motion using the axis group feature "Tools". This feature requires a tool table to gather all the data describing each tool. To have two tools, it is also necessary to define a tool configuration for each one. The tool data must be correctly transferred from the dimensional drawings to the tool configuration in order to include each tool to the mechanical system.

After correctly configuring tools and using them in a robotics program, mapp Cockpit and Scene Viewer can be used to check how the system is affected by each different tool.

Automation Studio:

- 1) Add a tool configuration file ("Tool") and create two tool geometry entries. Add the geometrical information for each tool.
- 2) Add a tool table and make two entries in it.
Assign two unique tool identifiers and link each to one of the tool geometries created previously.
- 3) Add an axis group feature of type "Tools" ("Tools Feature") and link it to the tool table.
- 4) Assign the axis group feature "Tools" to the axis group.
- 5) Transfer changes and restart the target system.

Robotics program:

- 1) Create a simple program or reuse the existing program.
- 2) Enable the first tool with the command SetTool(<Identifier 1>).
- 3) Start the program.
- 4) Enable the second tool with the command SetTool(<Identifier 2>).
- 5) Start the program with the second tool.
- 6) Analyze and compare the movements.

To check the movements using the provided Automation Studio scene, it is necessary to show and hide the simulated tools in it. Be sure to select the one being used for each test.

Further information



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Programming languages \ Structured Text \(ST\) \ Technology \ Tool selection](#)

To use the tools included in the proposed scene, it is necessary to know their dimensions. This information can be found in the drawings and 3D model. The next table is used to collect the required data:

**Result:**

To use the tools included in the proposed scene, it is necessary to know their dimensions. This information can be found in the drawings and 3D model. The next table is used to collect the required data:

Parameter	Schunk mechanical gripper	Schunk vacuum gripper
X [mm]	0	0
Y [mm]	0	0
Z [mm]	-81	-120
Angle 1 [°]	0	0
Angle 2 [°]	0	0
Angle 3 [°]	0	0

To test the tools, the robotics program must be modified. The next example shows a possibility to use different tools for the program designed in previous exercises.

```
...
    Feedrate(2000);      // Move with reduced speed
    SetTool(1);          // Set Tool Gripper=1 and Vacuum=2
    MoveAR(InitialPos);  // Move to initial position
...
```

As mentioned in the exercise description, it is necessary to show and hide the 3D models of the tools in Scene Viewer to correctly understand the path followed by the robot. The next image shows an example of how to do that:

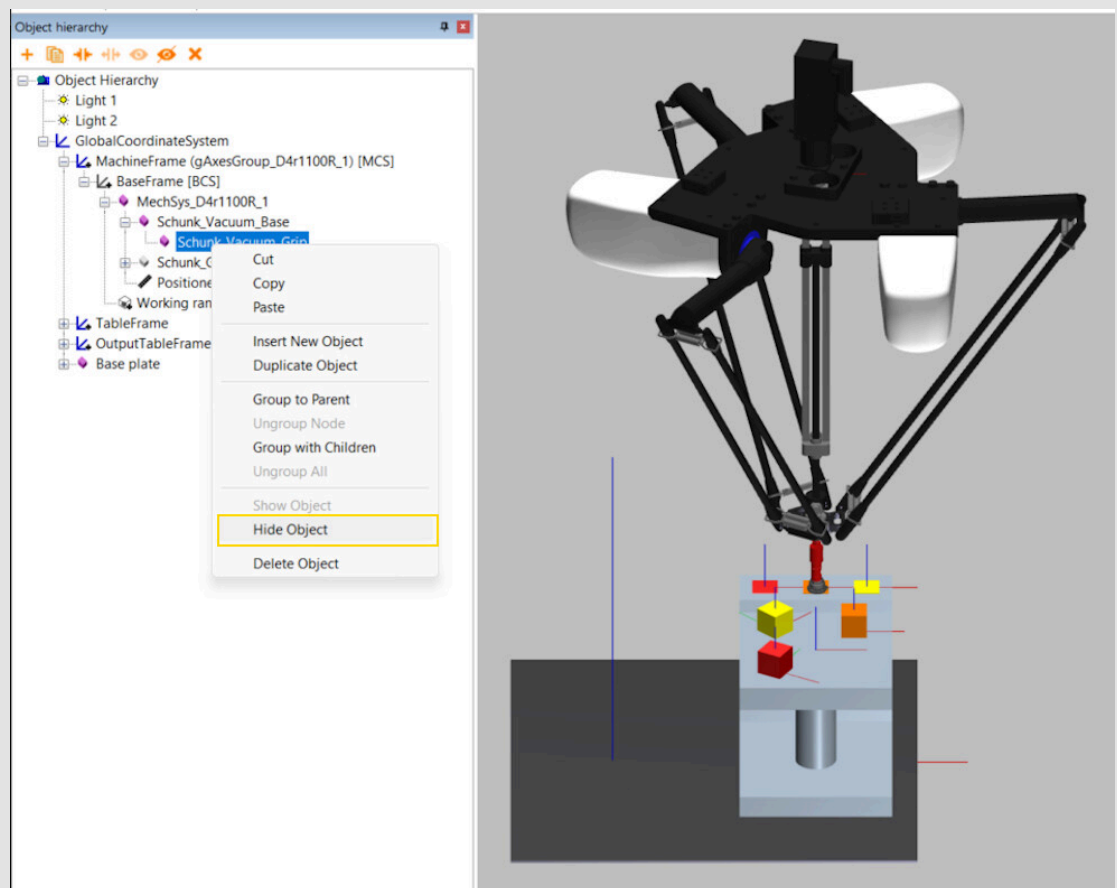


Figure 90: Hiding and showing tools in Scene Viewer

9.4 Dynamic behavior of tools

As mentioned, the dynamic behavior of the tools is very relevant in some applications. The dynamic parameters of a tool are described in the file "Tool Dynamic Parameters". These parameters allow the mechanical system to adjust all the necessary torques and forces to handle the new load added by the tool.

The parameters necessary to describe this tool are its mass, its center of gravity and its moment of inertia in all rotation directions. The format of this table is fixed. Adding or deleting parameters from the table is not permitted. If values are not required or not available, they can be entered as 0.

Base Parameter Index	Value	Unit	Description
0	10.0	kg	Mass
1	0.06	m	Center of gravity - X direction
2	0.0	m	Center of gravity - Y direction
3	0.0	m	Center of gravity - Z direction
4	0.0	kg m ²	Moment of inertia about X
5	0.0	kg m ²	Moment of inertia about Y
6	0.0	kg m ²	Moment of inertia about Z

Figure 91: Dynamic tool table example.

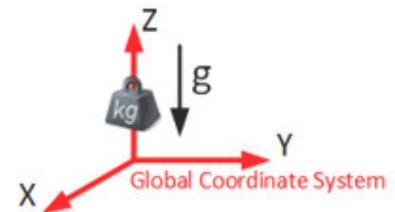


Figure 92: Gravity definition

As mentioned previously, the direction of gravity in the system is always defined as the negative direction of the Z axis in the GCS. This direction directly affects the dynamics of the system and therefore the dynamics parameters that affect the tools.



Be sure to use the correct units for each parameter.

Exercise: Dynamic parameters for tools

In this exercise, the descriptions for both tools is complemented by adding their dynamic parameters. With them, it is possible to analyze their influence on the dynamics of the whole system.

The exercises also shows that when a tool and/or product are added to the system, it automatically reduces its speed and acceleration in order to not violate the respective motor and mechanical limits.

The best way to show this is to use rapid traverse movements where the robot moves as dynamically as possible. Automation Studio:

- 1) Add a tool dynamic parameter table for each robot.
- 2) Enter the dynamics parameters for each tool (take note of the units).
- 3) Assign each tool dynamic parameter table to each tool identifier in the tool table.
- 4) Add an axesgroupfeature of type "Monitoring Elements", add a single element with the Type "Torques" and add it to the axes group.
- 5) Create a variable of type "McPathGenMonElemAxTorquesType" and bind it to the transmitted torque parameter for the single element.
- 6) Transfer the changes and restart the target system.

Robotics programs:

- 1) Create a new robotics program that performs rapid movements in space. Feel free to design you own path.
- 2) Set the tool with SetTool() to 0 (no tool).

mapp Cockpit:

- 1) Configure a trace that records the monitoring element torques, the path speed and line number.
- 2) Set the path speed as the trigger.
- 3) Start the trace.

- 4) Run the program with no tool.
- 5) Repeat the whole process for each tool.
- 6) Analyze and compare the recording.

Further information



[Motion control \ mapp Robotics/CNC \ Configuration \ Basic Elements \ Dynamic parameter table](#)



Result:

The parameters used in this exercise for each tool are detailed in the following section:

- Schunk mechanical gripper
 - Mass = 0.17 kg
 - Center of gravity - Z direction = -0.031 m
 - Moment of inertia about X = 0.000064 kgm²
 - Moment of inertia about Y = 0.000028 kgm²
 - Moment of inertia about Z = 0.000064 kgm²
- Schunk vacuum gripper
 - Mass = 0.22 kg
 - Center of gravity - Z direction = -0.06 m
 - Moment of inertia about X = 0.000251 kgm²
 - Moment of inertia about Y = 0.000251 kgm²
 - Moment of inertia about Z = 0.000045 kgm²

The following figure shows the torque curves of the three main axes (Q1, Q2, and Q3) of an Delta robot during the same movement but with different tools. Green represents the torque curve without a tool, and red with the vacuum gripper.

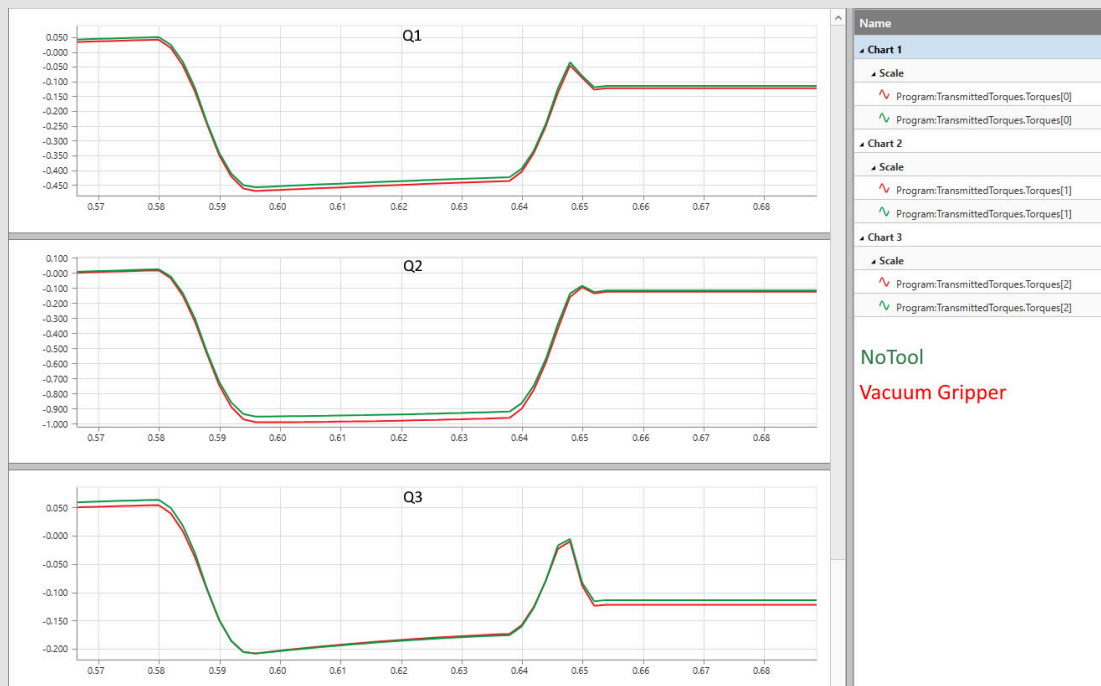


Figure 93: Traces obtained using no tool

10 Machine application and PLC synchronization

This chapter will focus on explaining some function blocks and libraries that can be very useful when creating a machine application for a robotic system. The trainees will see how to create a basic automated program to control a robot. It will also focus on the various possibilities to synchronize and communicate information between robotic programs and the machine application running on the control system. Finally, different possibilities for restarting a robotics program that has been stopped will be presented.

10.1 Machine application

Machine applications are a core part when developing a robotics project. Having an automatic application that handles the robot state and its errors is key to developing an efficient and automatic solution. In this section the basic elements to develop such an application will be discussed and analyzed.

10.1.1 Technology

The previous picture shows how mapp Robotics software solutions are organized. Like all mapp Motion elements, all the different functionalities and features regarding robotics are divided into 3 levels, from the core ones to the one focusing on entire process solutions.

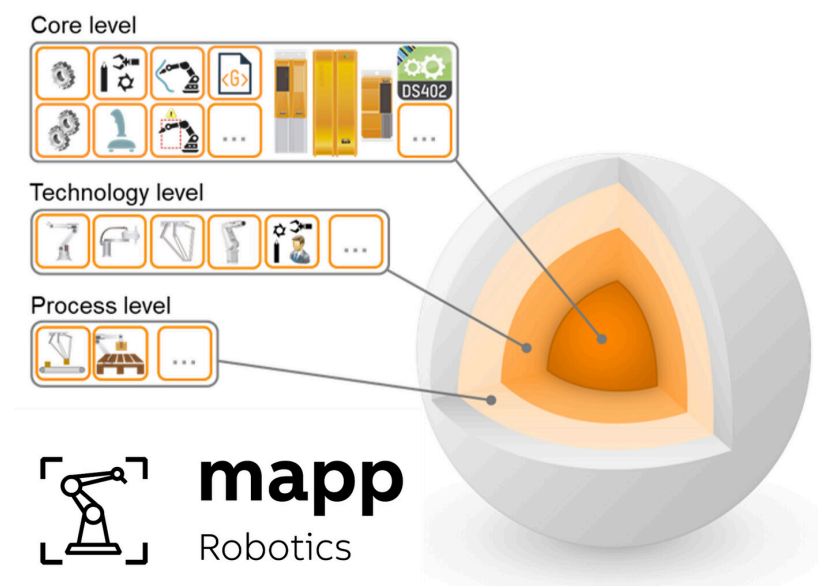


Figure 94: mapp Robotics technology levels

One of the main utilities for developing robotic machine applications is the MpRobotics library. Using function blocks from this library simplifies many development tasks for machine applications. These function blocks include all the basic functionalities of such a system: power, home, jog, error handling, etc. Internally, they use the basic PLCopen function blocks but all commands, parameters and status variables are collected together in one interface. Inside this library, there are functions blocks that cover the most common robot mechanics used in industrial environments: 6 axes robots, SCARA robots, delta robots, etc. For other types of mechanics, there is a flexible function block, MpRoboticsFlex, that can handle any mechanical structure with up to 15 axes.

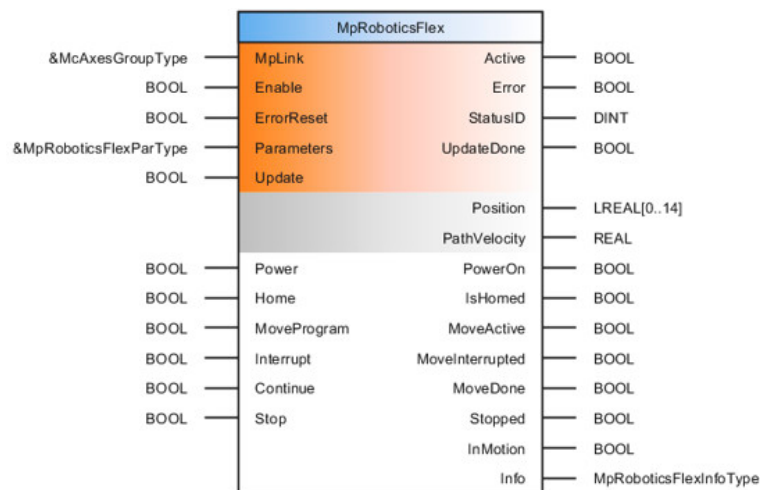


Figure 95: MpRoboticsFlex function block diagram

For additional details, see Automation Help.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Libraries \ Technology libraries \ MpRobotics \ Function blocks](#)

10.1.2 Programming tips

The application program is a way to represent an automatic sequence for controlling the robot. In this process, it is important to call the function blocks previously introduced in a clear manner. Furthermore, error events must be taken into consideration and responded to.

The function block offers inputs for controlling robotic functions. Processes can be started at a defined point in the program.

The following information is provided by status outputs and output parameters:

- Was the execution of the command successful?
- If not, what error occurred?
- Status of the process:
 - Is the axis group moving?
 - Did the axis group reach the target position?
 - Was homing performed successfully?

This information can be used for controlling the program sequence in the drive application. The program will have to respond differently depending on whether or not an error occurs.

The step switching mechanism (state machine) is a control structure that is especially well-suited for managing these types of function processes.

This type of structure allows the implementation of individual steps whose sequence can be determined by the use of a step index.

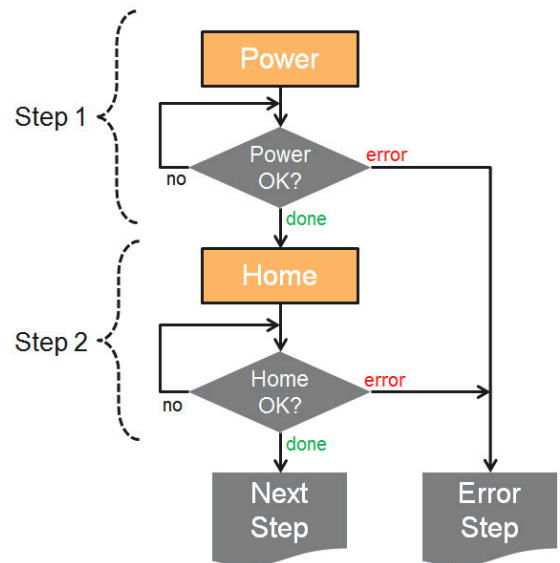


Figure 96: Sample control structure, structured programming

The necessary commands (switching on the robot, homing the axis group, etc.) can be performed in the individual steps of the control structure. Status parameters such as "Error", "StatusID" and the info structure can be used to determine the step where the application will continue. This gives the application a clear structure and opens it up for future expansion.

When developing a machine application, it is very important to correctly design the transition between PLC states. The next image shows the state diagram for an axis group.

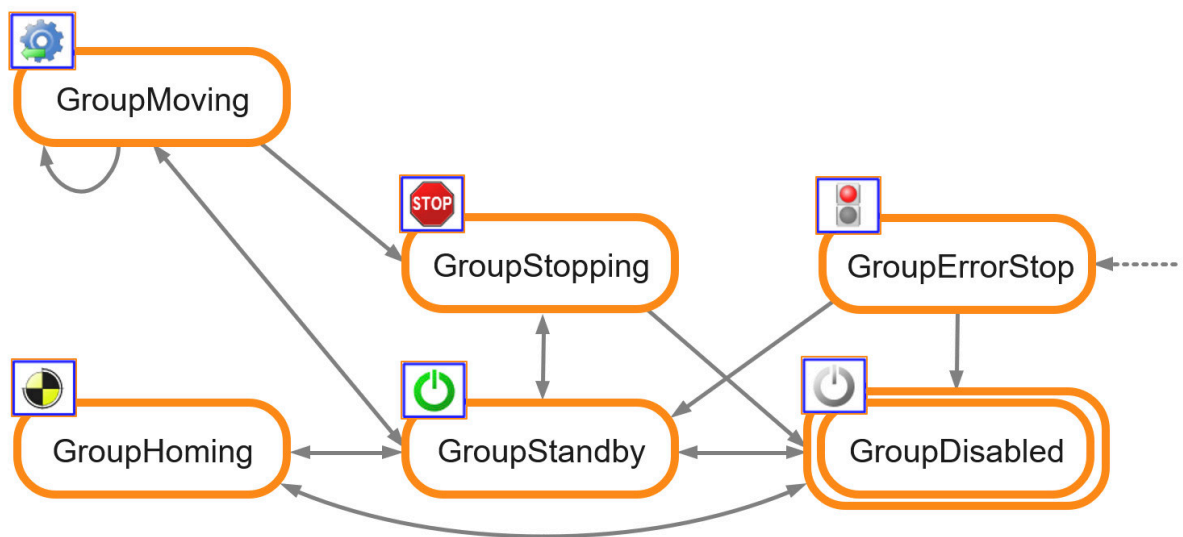


Figure 97: Axis group state diagram.

Exercise: Creating a robot application using an automatic procedure

In this exercise, a simple robot application is created using an automatic procedure. MpRobotics library function blocks are the main part of this. The function blocks in this library cover the most common robot application functions. In the second part of this exercise, the robot is automatically brought into a state in which robotics programs can be run. To do this, the robot must be switched on and homed.

Automation Studio

Programming a function block

- 1) Create a suitable MpRobotics library function block MpDelta4Axis.

- 2) Create parameter structure for the MpRobotics function block.
- 3) Link the parameter structure (address of the structure) and MpLink (address of the component) with the function block.
- 4) Enable the function block (set to Enable).
- 5) Call the function block.
- 6) Transfer the application, restart the system and test functionality.

Program an automatic procedure

- 7) Create a variable to start an automatic procedure.
- 8) Select the correct status information in the info structure for the Power command (Info.ReadyToPowerOn).
- 9) Create an automatic sequence in order to power on and home the axis group (PowerOn and IsHomed).
- 10) After the sequence has been executed, reset the start variable.
- 11) Transfer the application, restart the system and test functionality.

Further information



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Libraries \ Technology libraries \ MpRobotics \ Function blocks](#)

**Result:**

After starting the target system and the automatic procedure, the robot is switched on and homed and is thus ready for robotics programs to be run. A possible implementation is suggested next:

```

PROGRAM _INIT
// Set Initial Robot Parameters
    RobotParameters.ProgramName := 'Program1.st';
    RobotParameters.CoordSystem := 0;
    RobotParameters.Velocity := 100;
    RobotParameters.Acceleration := 1000;
    RobotParameters.Deceleration := 1000;
END_PROGRAM
PROGRAM _CYCLIC
// Error detection
    IF MpDelta4Axis_0.Error THEN
        Step := ROBOTCTRL_ERROR;
    END_IF;
// Robot Control Step Machine
CASE Step OF
    ROBOTCTRL_DISABLED:
        IF cmdTurnOn THEN
            cmdTurnOn := FALSE;
            Step := ROBOTCTRL_POWER;
        END_IF;
    ROBOTCTRL_POWER:
        IF MpDelta4Axis_0.Info.ReadyToPowerOn THEN
            MpDelta4Axis_0.Power := TRUE;
            Step := ROBOTCTRL_WAIT_POWER;
        END_IF;
    ROBOTCTRL_WAIT_POWER:
        IF MpDelta4Axis_0.PowerOn THEN
            Step := ROBOTCTRL_HOME;
        END_IF;
    ROBOTCTRL_HOME:
        MpDelta4Axis_0.Home := TRUE;
        Step := ROBOTCTRL_WAIT_HOME;
    ROBOTCTRL_WAIT_HOME:
        IF MpDelta4Axis_0.IsHomed THEN
            MpDelta4Axis_0.Home := FALSE;
            Step := ROBOTCTRL_READY;
        END_IF;
    ROBOTCTRL_READY:
        IF cmdTurnOff THEN
            cmdTurnOff := FALSE;
            MpDelta4Axis_0.Power := FALSE;
        END_IF;
        IF NOT (MpDelta4Axis_0.PowerOn) THEN
            Step := ROBOTCTRL_DISABLED;
        END_IF;
    ROBOTCTRL_ERROR:
        // Error handling
        IF MpDelta4Axis_0.Error THEN
            IF MpDelta4Axis_0.PowerOn THEN
                Step := ROBOTCTRL_READY;
            ELSE
                Step := ROBOTCTRL_DISABLED;
            END_IF;
        END_IF;
END_CASE;
// Call MpDelta4Axis FBs
MpDelta4Axis_0.MpLink := ADR(gAxesGroup_D4r1100R_1);
MpDelta4Axis_0.Parameters := ADR(RobotParameters);
MpDelta4Axis_0.Override := 100.0;
MpDelta4Axis_0.Enable := TRUE;
MpDelta4Axis_0();

```

[END_PROGRAM](#)

10.1.3 Complementary libraries

Some functionalities necessary in robotic systems are not covered by the function blocks in the MpRobotics library. These can be implemented using other libraries inside mapp Motion that include additional function blocks. Combining them with the basic ones makes it possible to develop solutions for all common robot applications in the industry. Some of these function blocks can be used to interact with axis group features such as coordinate systems, tools, etc. For additional details, see Automation Help.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Libraries \ Core libraies \ McAx-Group](#)

Exercise: Using additional function blocks

In this exercise, the necessary function blocks are programmed to display the tool center point of a robot in a previously configured user-defined coordinate system.

The necessary functions are provided with the MC_BR_GetCoordSystemIdent and MC_GroupReadActualPosition_15 function blocks from the McBase and McAxGroup libraries.

Automation Studio

- 1) Create MC_BR_GetCoordSystemIdent.
- 2) Assign the name of the coordinator system to function block MC_BR_GetCoordSystemIdent.
- 3) Enable the function block.
- 4) Call the function block.
- 5) Create MC_GroupReadActualPosition_15.
- 6) Connect MC_GroupReadActualPosition_15 to the axis group (assign the address of the component to the Axes-Group input).
- 7) Enable the function module (set the Execute input if MC_BR_GetCoordSystemIdent was executed successfully).
- 8) Call the function block.
- 9) Transfer the application, restart the system and test functionality.

Further information



[Motion control \ mapp Motion \ General information \ Programming \ Application program \ Libraries \ Core libraries \ McBase \ Function blocks \ MC_BR_GetCoordSystemIdent](#)
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Libraries \ Core libraries \ mcAx-Group \ Function blocks \ MC_GroupReadActualPosition_N](#)

Result



Using these function blocks, it should be possible to get the position of the TCP of the robot with respect to the different frames defined in the object hierarchy. By using Watch to enable and disable the function blocks and changing the coordinate system, it should be possible to perform different tests. The following code is a suggestion for how to extend the machine application to add these function blocks.

```
...
    // Call GetCoordSystemIdent FBs
    MC_BR_GetCoordSystemIdent_0();
    // Call GroupReadActualPosition FBs
    MC_GroupReadActualPosition_15_0.AxesGroup =
        ADR(gAxesGroup_D4r1100R_1);
    MC_GroupReadActualPosition_15_0();
...
```

10.2 Synchronize machine application with robotics program

Some applications require synchronizing and sharing information between the PLC application and the motion program. These processes might require changing execution of a program or simply blocking it until a certain condition is met, until a sensor is active, for example.

In this section, some of the most commonly used solutions will be explained. For additional information, see Automation Help.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Programming languages \ Structured text\(ST\) \ Synchronization](#)

10.2.1 Process variables

One of the elements that can be used for this synchronization are **process variables**. Process variables are defined in the axis group feature "CNC and robotics programs". They allow the motion program to read and write machine application variables. It is possible to read a variable linked to a digital input on the PLC, like a sensor input, and modify the behavior of the program according to it, for example.

Program elements		Program elements
Process variables		Declaration of process variables
Process variable 1		
Process variable name	::gGripperVacuum	Name of the process variable
Advanced		
Alias	gGripperVacuum	Alternative name of process variable
Synchronization	Path-synchronous	Synchronization
Reference	No	Variable is a reference
Process variable 2		
Process variable name	::gGripperVacuumOk	Name of the process variable
Advanced		
Alias	gGripperVacuumOk	Alternative name of process variable
Synchronization	Path-synchronous	Synchronization
Reference	No	Variable is a reference

Figure 98: CNC and robotics program axis group feature

These variables can be read and written in synchronization with the interpreter or the motion path. To change this behavior, it is necessary to change the parameter "Synchronization".

- **Path-synchronous:** The value of the variable is read or written in synchronization with the movement of the system. Its value is only obtained or changed when the path has reached the point in the robotic program where the variable is called.
- **Interpreter-synchronous:** The value of the variable is read or written when the interpreter is processing the robotics program. When the interpreter reads the line where the variable is called, then its value is obtained or changed.

For some applications, it can be crucial to properly configure this parameter because the behavior of the application can really change depending on it.



[Motion control \ mapp Robotics/CNC \ Configuration \ Basic elements \ AxesGroup feature \ CNC & robotics programs](#)

Exercise: Controlling the gripper

A vacuum gripper is to be controlled via path-synchronous writing of a process variable in a robotics program. To do this, the PV must be created in the control application, used in a task and configured in the "CNC & Robotics programs" feature. To display the switching state in Scene Viewer, a variable is created that changes the "Material" property of the gripper depending on the switching state.

Automation Studio:

- 1) Create a global PV variable and name it "gGripperVacuum".
- 2) Use the PV in a task so it is compiled.
- 3) Open the "CNC & Robotics programs" axis group feature in Configuration View (mapp Motion).
- 4) Create a new program element "Process variable" with this PV.
- 5) Set Advanced/Synchronization to 'Path-synchronous' and assign an optional alias name.
- 6) Transfer the project and perform a warm restart.

Robotics program:

- 1) In the robotics program, the variable can now be set at the robot's gripping position.
- 2) Add a delay time during which the vacuum is built up.
- 3) The system then moves to the storage position and resets the variable.
- 4) Add a delay time during which the vacuum is reduced.
- 5) Test the program.
- 6) Check if the times for actuating the gripper are correct (if necessary by means of a trace recording).

Scene Viewer:

- 1) Create your own task for Scene Viewer data preparation and add variable "GripperMaterial" (USINT).
- 2) Set the variable depending on the state of variable "gGripperVacuum" (for the color selection, see the "Material" property in Scene Viewer).
- 3) Add the variable to OPC UA mapping.
- 4) Transfer the project (and perform a warm restart).
- 5) Bind the variable just created to the "Material" property for both tools.

Further information

[Motion control \ mapp Robotics/CNC \ Configuration \ Basic elements \ AxesGroup feature \ CNC & robotics programs](#)

**Result:**

The result should be a clear image of when a piece is grasped or not. The tool in Scene Viewer should change its color depending on the state of the vacuum output. This should help simulate the pick-and-place solution and get a more realistic view of its behavior.

After adding the command to activate or deactivate the gripper process variable, the code used for the previous exercises could look like this.

```
...
(* Pick Orange Cube *)
SetPCS(OH::OrangeCubeFrame); // Set "OrangeCubeFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
gGripperVacuum := TRUE; // Close Gripper
WaitTime(0.5); // Simulate gripper closing
MoveL(PrePosition); // Move system to point PrePosition

(* Place Orange Cube *)
SetPCS(OH::OrangeBaseFrame); // Set "OrangeBaseFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
gGripperVacuum := FALSE; // Open Gripper
WaitTime(0.2); // Simulate gripper opening
MoveL(PrePosition); // Move system to point PrePosition
...
```

10.2.2 Path-synchronous execution of ST statements

Another option to perform path-synchronous movements is to use the statement `DO_PATH_SYNCH`. All the ST statements performed between the statements `"DO_PATH_SYNCH"` and `"END_PATH_SYNCH"` will be performed with path-synchronous execution. The commands inside will wait for the system to reach the proper points in the path to start execution of all respective ST commands.

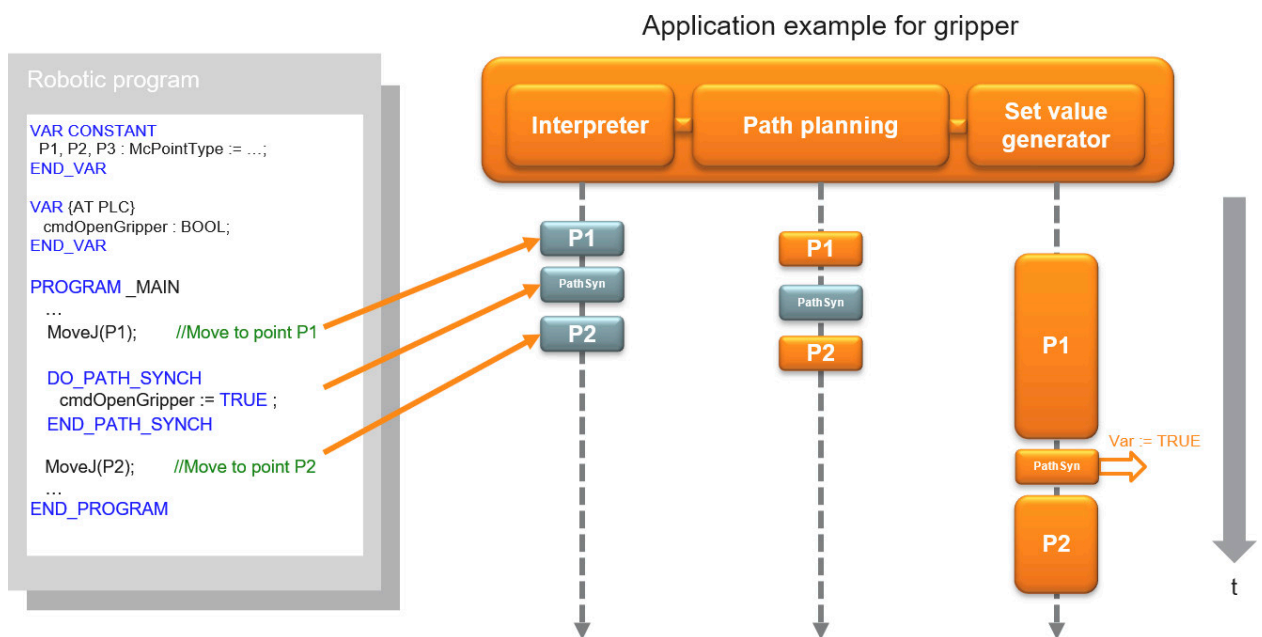


Figure 99: Example `DO_PATH_SYNCH` statement

More information on these statements can be found in Automation Help.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Programming languages \ Structured text\(ST\) \ Statements \ Path-synchronous execution](#)

10.2.3 Waiting - ST commands

Another option to synchronize the machine application and the robotic system is to use some specific ST commands. These commands stop the interpreter and wait until certain conditions are met. Some of the most commonly used are presented next:

- **WaitIp():** Program execution waits for a user input signal. The interpreter stops processing ST commands until the user issues a command that execution should resume. The machine application is notified that the interpreter has stopped. To read that information, function block MC_BR_ProgramInfo can be used. To resume the movement and start the interpreter again, it is necessary to execute the function block MC_GroupContinue or any equivalent signal (MpRobotics function block continue input or mapp Cockpit continue command).
- **WaitEndMove():** The interpreter stops processing new commands and waits until all previous movements have ended. As soon as path movement reaches the synchronization point in the robotics program, the interpreter automatically starts processing all the next commands.
- **WaitALAP():** The interpreter is paused as long as possible to wait for any machine application trigger. The interpreter processes several motion blocks for the path in advance. Using this command, the interpreter is stopped and resumed automatically in order to only execute the minimum necessary to perform the next movement. If an application trigger is received, the path can be modified on time, but if not, there is no need to pause the movement and the original path can continue as planned.

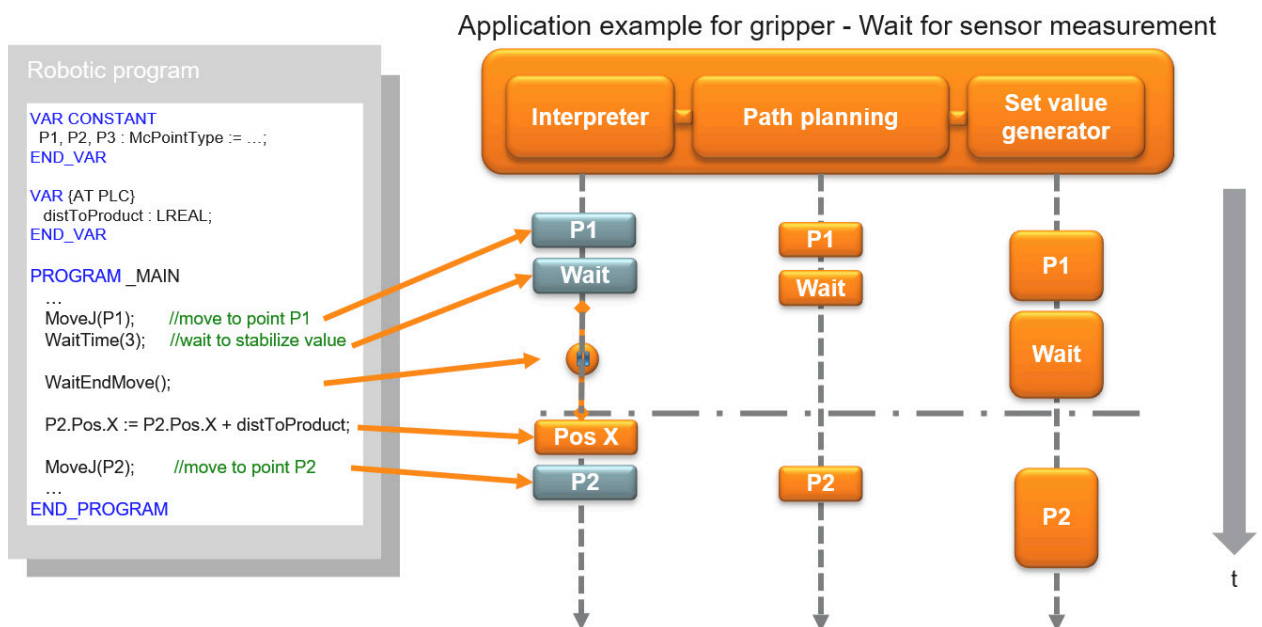


Figure 100: Example WaitEndMove command

For additional information about these commands and others, see Automation Help.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Programming languages \ Structured text\(ST\) \ Statements \ Path-synchronous execution](#)

10.2.4 Cyclic actions

Finally, **cyclic actions** can also be used in motion programs for synchronization. The statements between **CYCLIC** (or **CYCLIC_BL**) and **CYCLIC_END** define sections that are cyclically executed in the context of a robotic program. Cyclic actions end when an **EXIT** statement is executed.

There are two types of cyclic actions:

- **Blocking (CYCLIC_BL):** The interpreter is blocked while the commands inside this action are executed. No other commands will be executed until the action ends.
- **Non-blocking (CYCLIC):** The interpreter is not blocked while the commands inside this action are executed. They are executed in parallel with the rest of the commands in the robotics program.

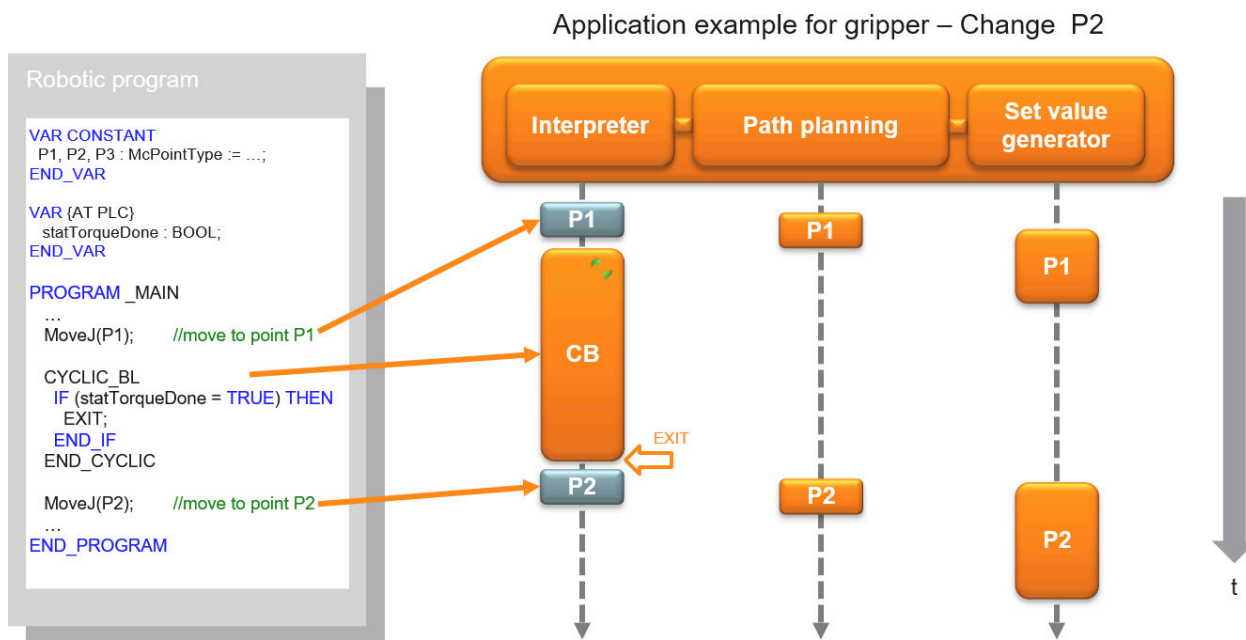


Figure 101: Example CYCLIC_BL action

Two additional cyclic execution options for motion programs are possible: **CYCLIC_PS** and **CYCLIC_PS_BL**. They use the same concept as the ones previously presented but are path-synchronous. So before starting cyclic execution, the system waits for the robot to finish all previous instructions.

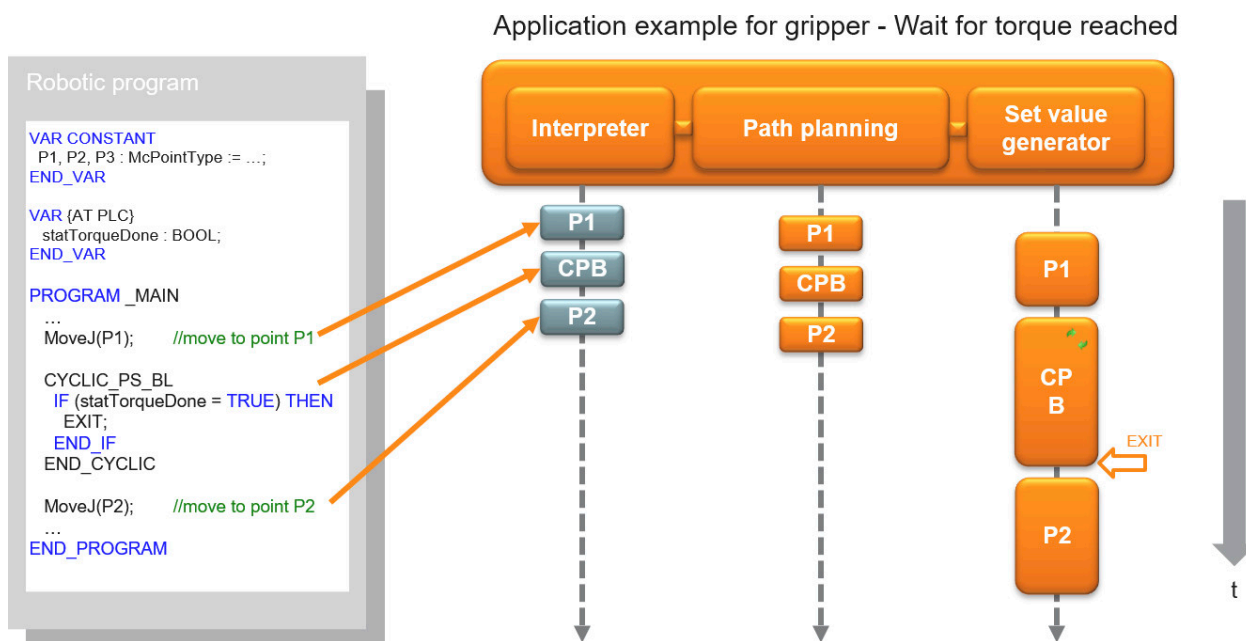


Figure 102: Example CYCLIC_PS_BL action

For additional information, see Automation Help.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Programming languages \ Structured text\(ST\) \ Statements \ Cyclic execution](#)

Exercise: Waiting for sensor feedback

By using program instruction "CYCLIC_PS_BL", the system should wait until a process variable provides the information that the vacuum has been built up or released when picking up or placing a product.

Variable creation and configuration

- 1) Create a new global process variable "gGripperVacuumOk" and use it in the task.
- 2) Create a new program element "Process variable" with this PV.
- 3) Transfer the project and perform a warm restart.

Robotics program

- 1) Use instruction "CYCLIC_PS_BL" in the robotics program when picking up and placing the product.
- 2) "EXIT" after corresponding feedback by process variable "gGripperVacuumOk".
- 3) Test the robotics program. Use the Watch window to set and reset the sensor variable "gGripperVacuumOk".

Further information

[Motion control \ mapp Robotics/CNC \ Configuration \ Basic elements \ AxesGroup feature \ CNC & robotics programs](#)

[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Programming languages \ Structured text\(ST\) \ Statements \ Cyclic execution](#)

**Result:**

Using the same program from the previous exercises, picking and placing movements can be modified as follows to include the simulation of the sensor input:

```
...
(* Pick Orange Cube *)
SetPCS(OH::OrangeCubeFrame); // Set "OrangeCubeFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
gGripperVacuum := TRUE; // Close Gripper
CYCLIC_PS_BL
IF gGripperVacuumOk THEN
    EXIT; // Wait for sensor feedback
END_IF
END_CYCLIC
MoveL(PrePosition); // Move system to point PrePosition

(* Place Orange Cube *)
SetPCS(OH::OrangeBaseFrame); // Set "OrangeBaseFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
gGripperVacuum := FALSE; // Open Gripper
CYCLIC_PS_BL
IF NOT(gGripperVacuumOk) THEN
    EXIT; // Wait for sensor feedback
END_IF
END_CYCLIC
MoveL(PrePosition); // Move system to point PrePosition
...
```

The next image shows how to bind an application variable to change the color of the gripper depending on its state:

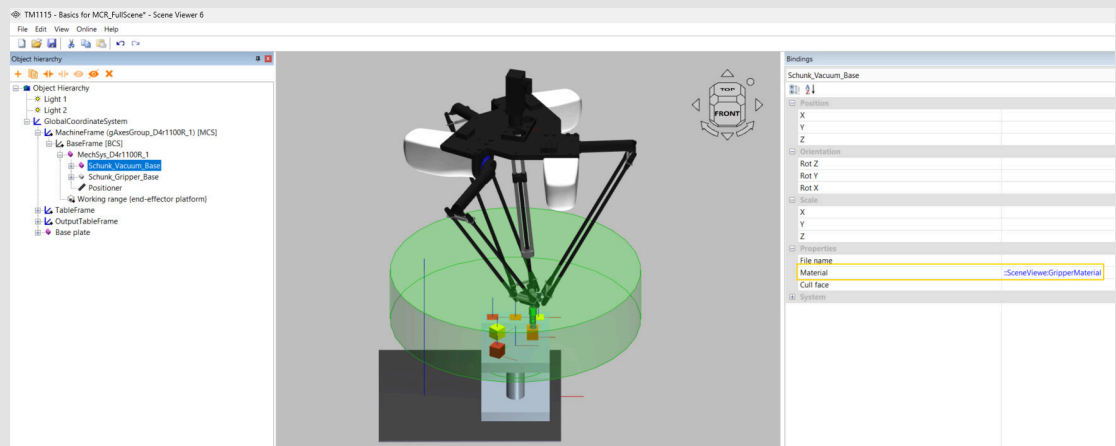


Figure 103: Binding a variable with the property "material"

10.2.5 M-functions

M-Functions are another possibility for communication between robotic programs and the machine application. When calling an M-function inside a motion program, a signal is triggered inside the PLC that can be directly linked to a variable or obtained using certain functions blocks. These signals can be used to control digital outputs, trigger specific functionalities, etc. The signal must be then reset by the application to be ready to be triggered again.

M-functions are configured inside the axis group feature "M-functions". Each one must have a unique index that must be in the range between 0 and 1023. Some of these codes are already predefined and cannot be assigned to new functionalities. The following table shows an overview:

M-function codes	Description
M0	Used as a programmed stop inside motion programs
M01	Used as an optional stop inside motion programs
M02, M29 and M30	Used as statements to close out the main program or to jump out of subroutines
M03, M04 and M05	Used to control the main spindle in the system

Predefined M-functions

All the rest of the codes can be freely used and linked to any functionality desired.

M-functions can be classified as two main types:

- **Blocking:** Stops the motion program when called until the application resets it.
- **Non-blocking:** Triggers the signal and continues the motion program without waiting for any signal to reset it.

Other behaviors for an M-function can be configured in the axis group feature. Parameters like whether the function is triggered at the beginning or the end of a movement or if it is set while running a motion program in simulation or not, can be set for each one of the configured functions.

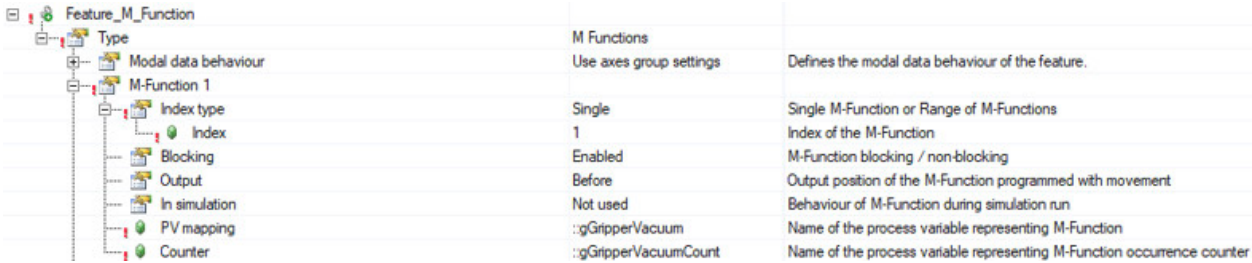


Figure 104: M-functions axis group feature configuration

For additional information about all these possibilities, see Automation Help.

[Motion control \ mapp Robotics/CNC \ Configuration \ Basic elements \ AxesGroup feature \ M-Functions](#)

From the application, it is necessary to use a function block to handle M-functions. This function block is called MC_BR_MFunction and can be used to read and reset M-functions.

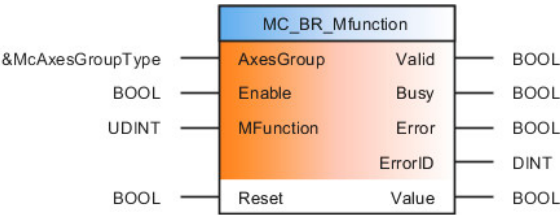


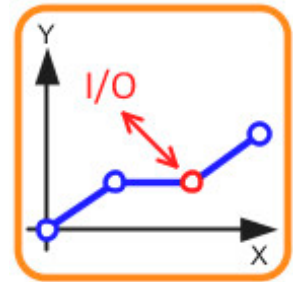
Figure 105: MC_BR_MFunction FB diagram

For additional information, see Automation Help.

[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Libraries \ Core libraries \ McAxesGroup \ Function blocks \ MC_BR_MFunction](#)

From the robotics program on the other hand, an ST command must be called to set M-functions. This command is called SetM and it can be used to set several M-functions each time it is called.

For additional information, see Automation Help.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Programming languages \ Structured text\(ST\) \ Technology \ M-functions](#)

Optional exercise: Controlling the gripper using M-functions

M-functions can be used for communication between a running robotics program and the machine application. In this exercise, the vacuum gripper is controlled with M-functions. To do this, it is first necessary to configure two separate M-functions in the "M-functions" axis group function. One of them is directly connected to the output variable that starts the vacuum for the gripper, the other is used for resetting the signals and also to stop the vacuum.

After configuration, the previously created machine application must be updated to include the reset functionality for both M-functions. For this, it is necessary to use function block "MC_BR_MFunction". Two function blocks are created, one for each M-function, and when they are both active, they are reset. Then it is necessary to adjust the ST program for the movement to use this new function.

Automation Studio

- 1) In the "Axis group feature configuration" file, add a new feature of type "M-functions".
- 2) Create a new M-function by adding index 40 in the "Index" attribute. Disable the "Blocking" property.
- 3) Repeat the previous step to create another function, use index 41 for it and disable "Blocking" here as well.
- 4) Link the "PV mapping" attribute of the first M-function (index 40) to the global PV gGripperVacuum created in a previous exercise.
- 5) Assign the axes group feature "M-functions" to the "Axes Group Feature Configuration" file.
- 6) Open an ST task with your machine program and create two new instances of function block "MC_BR_MFunction". Name them "MC_BR_MFunction_40_FB" and "MC_BR_MFunction_41_FB".
- 7) Bind the "AxesGroup" parameter of both function blocks to the address of the global PV variable of the robot axis group.
- 8) Enable both function blocks.
- 9) Set the "MFunction" parameter of both functions to their respective index, 40 for "MC_BR_MFunction_40_FB" and 41 for "MC_BR_MFunction_41_FB".
- 10) Call both function blocks.
- 11) Create an IF-ELSE condition to reset both M-functions. If "Value" outputs are TRUE, set both "Reset" inputs to TRUE. If this condition is not fulfilled, set both "Reset" inputs to FALSE.

Robotics program

- 1) Modify the robot program from the previous exercise. Use the "SetM(40)" command after reaching the pick up position for the orange cube.
- 2) Add a delay time during which the vacuum is built up.
- 3) Use the "SetM(41)" command after you have reached the position where the orange cube should be placed.
- 4) Enter a delay time during which the vacuum is switched off.

- 5) Repeat the same process for the remaining cubes in the scene.
- 6) Test the program.

Further information



[Motion control \ mapp Robotics/CNC \ Configuration \ Basic elements \ AxesGroup feature \ M-Functions](#)



Result:

The behavior should mimic the previous exercises but in this case the color change of the tool must be operated using the M-functions. By using the Watch window and checking the outputs of the function blocks and vacuum variables, it is possible to check if everything is configured properly and the result obtained is correct.

The application must be expanded to set and reset the M-functions in control of the gripper. In the next example, M-function 40 has been linked to the variable that controls the vacuum on the gripper (gGripperVacuum):

```
// Set up and call M-Function FB: Set Vacuum (Index 40)
MC_BR_MFunction_40_FB.AxesGroup := ADR(gAxesGroup_D4r1100R_1);
MC_BR_MFunction_40_FB.Enable := TRUE;
MC_BR_MFunction_40_FB.MFunction := 40;
MC_BR_MFunction_40_FB();

// Set up and call M-Function FB: Reset Vacuum (Index 41)
MC_BR_MFunction_41_FB.AxesGroup := ADR(gAxesGroup_D4r1100R_1);
MC_BR_MFunction_41_FB.Enable := TRUE;
MC_BR_MFunction_41_FB.MFunction := 41;
MC_BR_MFunction_41_FB();

// Reset both FBs if both have been activated
IF MC_BR_MFunction_40_FB.Value AND
    MC_BR_MFunction_41_FB.Value THEN
    MC_BR_MFunction_40_FB.Reset := TRUE;
    MC_BR_MFunction_41_FB.Reset := TRUE;
ELSE
    MC_BR_MFunction_40_FB.Reset := FALSE;
    MC_BR_MFunction_41_FB.Reset := FALSE;
END_IF;
```

The program used in all previous exercises can also be adapted to use M-functions to control the gripper. The following solution could be used:

```
...
    // Pick Orange Cube
    SetPCS(OH::OrangeCubeFrame); // Set "OrangeCubeFrame" as PCS
    MoveJR(PrePosition); // Move system to point PrePosition
    MoveL(Position); // Move system to point Position
    SetM(40);
    WaitTime(0.5); // Simulate gripper closing
    MoveL(PrePosition); // Move system to point PrePosition

    // Place Orange Cube
    SetPCS(OH::OrangeBaseFrame); // Set "OrangeBaseFrame" as PCS
    MoveJR(PrePosition); // Move system to point PrePosition
    MoveL(Position); // Move system to point Position
    SetM(41);
    WaitTime(0.2); // Simulate gripper opening
    MoveL(PrePosition); // Move system to point PrePosition
...

```

10.2.6 Signaling

Signaling is used to output a signal triggered by a motion program to the machine application at a specific time or distance before the end of a path section. This can become relevant when handling external actuators or systems that have a specific activation or response time. There are two general parameters that apply to all signals used in a program:

- **Prediction time** and **prediction distance** define the minimum signal look ahead to be maintained throughout the program.
- The **axis delay** defines the delay between generation of the position setpoint in the controller and the actual position reaching the drive. It can either be user-defined (calculated or measured on an actual machine) or detected automatically.

		
Type	Signaling	
Prediction	Used	
Distance	0.5	Maximal prediction distance for signals
Time	Not used	
Axis delay	Automatic	The delay from the PLC set value generation to the value of the drive
Signals		
Signal 1		
Name	Signal	Name of the signal in an NC program
Action	M-Function	Action of the signal
Index	1	Index of the M-Function
Reaction time	Not used	Reaction time of the corresponding device
Microsecond precision	Not used	Microsecond precision of the signal
Output shift	0	The signal is output a defined number of cycles earlier
Missed signal behaviour	No output / error	Behaviour of signal in case it can not be output on intended position

Figure 106: Axis group feature signaling example.

The rest of the parameters are used to identify individual signals. Each signal must be identified by a unique name and must be linked to an action. This action defines the signal in the controller that will be triggered, for example an M-function. Configured signals can then be used inside motion programs by triggering specific ST commands like "SignalTime" or "SignalDistance".

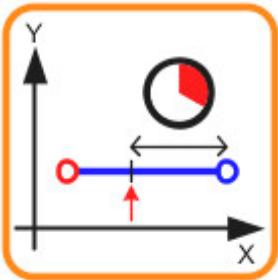


Figure 107: Time signaling

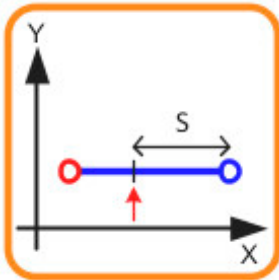


Figure 108: Distance signaling

For additional information about all possible configuration parameters and an example on how to use signaling, see AS Help.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Basic elements \ AxesGroup feature \ Signaling](#)

Optional exercise: Controlling the gripper using signaling

Signaling can be used to control when an output is activated or deactivated depending on the position of the robot while it is performing a path-controlled movement. In this exercise, it is used to activate and deactivate the vacuum of the gripper. The objective is to activate the vacuum at a fixed distance before reaching the pick up position so the vacuum is generated before the pick up position is reached, and then the same when deactivating. This improves the efficiency of the system by eliminating the delay otherwise required to ensure that the gripper grips or releases the box.

It is necessary to use the M-function implemented in the previous exercise, so the previous exercise should be done before starting this one.

Automation Studio

- 1) In file "Axes Group Feature Configuration", add a new feature of type "Signaling".
- 2) Create a new "Signal" by changing the "Name" to "VacuumOn". Set the Action/Index parameter to 40 to use the M-function that activates the vacuum.
- 3) Repeat the previous step to generate the signal for switching off. Use the name "VacuumOff" and index 41.
- 4) Set the "Distance" parameter under the "Prediction" block to "Used".
- 5) Change the attribute "Distance" to 100. This will be the default maximum prediction distance.
- 6) Assign the axes group feature "Signaling" to the "Axes Group Feature Configuration" file.

Robot program

- 1) Modify the robot program from the previous exercise. Use the "SignalDistance(50, ConfiguredSignal:=VacuumOn)" command after reaching the position where the orange cube should be picked up.
- 2) Use the command "SignalDistance(20, ConfiguredSignal:=VacuumOff)" after reaching the position where the orange cube should be placed.
- 3) Add function "SignalPrediction()" at the beginning of both the pick up and place movements. Add it before the first movement so that the prediction starts when the robot begins to move towards the pick up or place position.
- 4) Repeat the same process for the remaining cubes in the scene.
- 5) Test the program.

Further information



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Basic elements \ AxesGroup feature \ Signaling](#)



Results:

The result for this exercise should be very similar to the one obtained in the previous exercises. The biggest difference should be that the vacuum is activated or deactivated a certain distance before reaching the final positions. This should be indicated by the tool color change. Increasing and decreasing the distance value in the robotics program can help determine if "Signaling" has been configured properly. The following example suggests a possible solution to modify the sample program.

```
...
(* Pick Orange Cube *)
SetPCS(OH::OrangeCubeFrame); // Set "OrangeCubeFrame" as PCS
SignalPrediction();
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
SignalDistance(50, ConfiguredSignal:=VacuumOn);
MoveL(PrePosition); // Move system to point PrePosition

(* Place Orange Cube *)
SetPCS(OH::OrangeBaseFrame); // Set "OrangeBaseFrame" as PCS
SignalPrediction();
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
SignalDistance(50, ConfiguredSignal:=VacuumOff);
MoveL(PrePosition); // Move system to point PrePosition
...
```


10.2.7 Probing

Probing is an axis group feature used to store the current axes positions or to stop a movement when a defined trigger event occurs. These events are often external signals that must be evaluated in microsecond resolution to obtain a position with high accuracy. There are two supported trigger events:

- An **ACOPOS trigger event** is an event that triggers when a negative or positive edge on the digital inputs of an ACOPOS drive occurs.
- A **variable trigger event** is an event that triggers when a negative or positive edge of a Boolean process variable occurs. High accuracy of the latch positions can be achieved by using a timestamp taken during a trigger event (e.g. from digital signal processing modules). The timestamp must be synchronized with the system time on the controller.

Name	Value	Unit	Description
Feature_Probing_D4r1100R_1	Probing		
Type			
Triggers			
Trigger 1			
Name	TR1		Name of the trigger in a NC program
Type	ACOPOS		Type of the trigger source
Axis reference	gAxisQ1_D4r1100R_1		Name of the axis reference
> Digital input	Trigger 1		Hardware trigger input
> Edge	Positive		Trigger edge
Trigger 2			
Name	TR2		Name of the trigger in a NC program
Type	Variable		Type of the trigger source
PV mapping	::gStopTrigger		Name of the process variable (BOOL) representing trigger
> Edge	Positive		Trigger edge
> Time stamp	Not used		Trigger time stamp

Figure 109: Axis group feature probing example

Some common use cases for probing are checking the tool length, measuring points for setting up a new coordinate system, moving a tool to a desired position, work piece quality control, etc.

To start scanning for triggers, it is necessary to use the command "Probe" or "ProbeStop" in a motion program. After executing the command, the system will be ready to save any position or/and stop the system if a trigger occurs.

More than one trigger can be configured in the same movement to obtain different positions during the path of the robot. This feature can be very useful, for example when measuring the distance between all the clamps holding a piece in a drilling machine.

For additional information about all possible configuration parameters and an example on how to use probing, see AS Help.



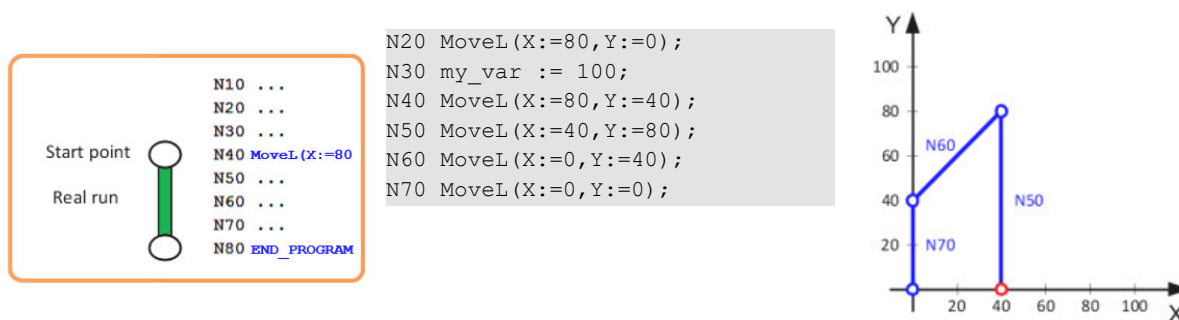
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Basic elements \ AxesGroup feature \ Probing](#)

10.3 Restarting the robotics program

Robotic programs might be interrupted or stopped by errors or users themselves. There are different modes when starting a program that allow different possibilities after it has been stopped. Depending on the mode, it might be possible to reposition the system and restart the program from the place where it stopped.

10.3.1 Non-modal start mode

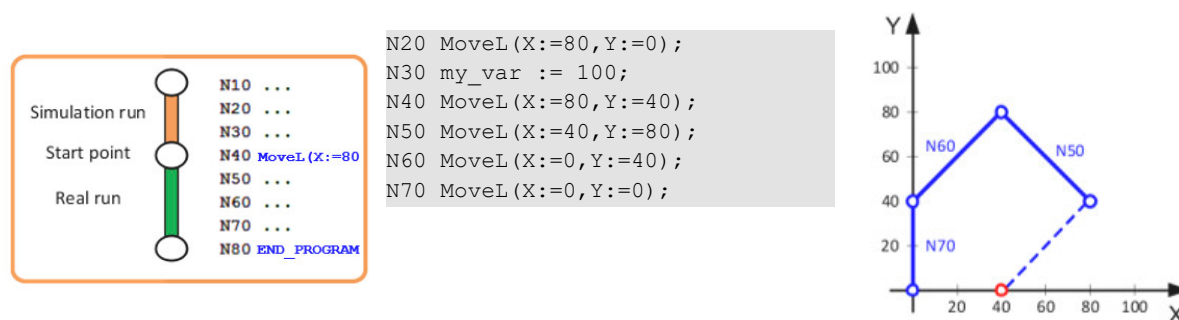
Non-modal start mode starts the program at a user-defined starting point and runs it until it finishes. It does not perform a simulation run of the program. Therefore, it has no information to reposition the robot before starting the program at the defined starting point. All commands programmed before this point are not executed.



Example of non-modal start of a program (Line 40)

10.3.2 Modal start mode

Modal start mode starts the program at the actual execution point and runs it until it finishes. It runs in simulation mode up to the defined starting point. The axes must be repositioned at the starting point before starting actual program execution.



Example of modal start of a program (Line 40)

10.3.3 Restart mode

The restart mode can be used to restart the program at any point after it has been aborted. The abort point where the program was aborted is the last known position on the programmed path. This position may not correspond to the position where the axes stop (e.g. in the event of axis error).

Saving restart data must have been enabled when the aborted program was started. The restarted program runs in simulation mode up to the defined starting point and uses the previously saved restart data.

Actual program execution is started after the starting point is reached. The axes must be repositioned at the starting point before starting actual program execution.

Parameter	Value	Unit	
Start mode	Non modal		^
Start point type	Standard		
Start point parameter	0.000000		
Repositioning type	Single axis auto		
Repositioning string			
Axis velocity factor	0.2000000		
Save restart data	Off		v
Move program ►			

Figure 110: Restart command parameters

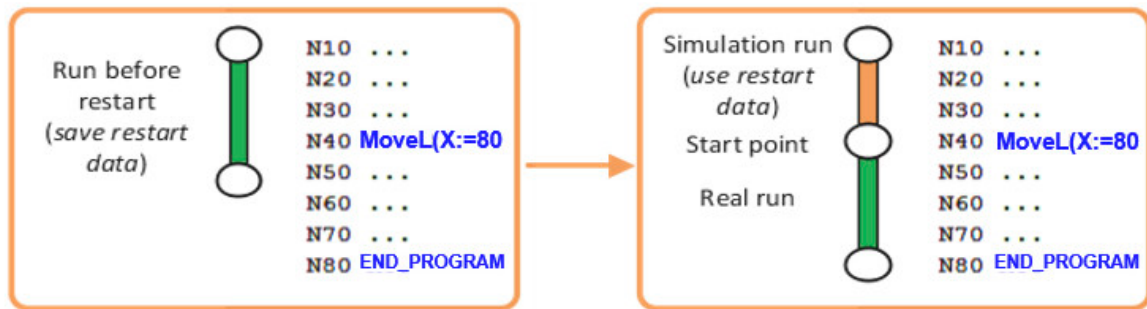


Figure 111: Example of restarting a program

The following image shows the steps necessary to restart a program using the Restart mode:



Figure 112: Steps to restart a program in restart mode.

Optional exercise: Restarting the program after aborting

If a program is aborted (e.g. after an emergency switch-off) it is possible to restart it from the position where it stopped. This requires a specific process as the system must be able to run the program as a simulation to get to the aborted line and be able to continue from there. The axes must be able to reposition properly and then the system can keep going on as it should.

Robotics program:

- 1) Create a robotics program with multiple movements.

mapp Cockpit:

- 1) Enable the 'Save restart data' option for the command 'Move program'.
- 2) Start the robotics program.
- 3) Abort the movement using 'Command error' with the command parameter 'Error stop'.
- 4) Perform a "Reset" to acknowledge the error status of the axis group.
- 5) Set the 'Move program' parameter for the restart. (Start mode = Restart, Start point type = Abort line)
- 6) Start the robotics program. After successful simulation, 'Move program phase' 'Wait for axes repositioning' is displayed.
- 7) By performing 'Move program continue', the axes are moved to the start position. It will display 'Wait for real run' as 'Move program phase'.
- 8) Performing 'Move program continue' again will continue the program.

Further information



[Motion control \ mapp Robotics/CNC \ Programming \ Libraries \ Core libraries \ McAxGroup \ Function blocks \ MC_BR_MoveProgram](#)
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Basic elements \ AxesGroup feature \ Program simulation and restart](#)
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Concept \ Range of functions \ Basic functions \ Operating modes \ Jogging](#)

**Result:**

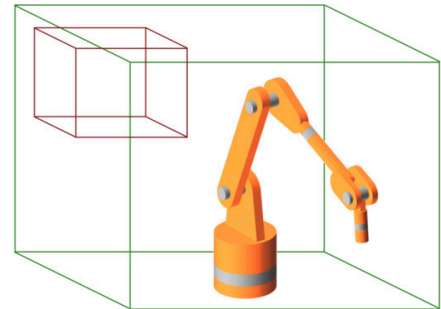
The result should be that the program can be stopped at any point and restarted afterwards. Using mapp Cockpit status variables and Scene Viewer, it is possible to view the behavior of the system and check if the movement is restarted correctly.

11 Workspace monitoring

Workspace monitoring is an axis group feature that prevents the system from exiting a certain space. It also ensures that its flange will not collide with specific spaces around the robot and the robot itself.

Essentially, it is possible to define two kinds of spaces:

- A **safespace** is a space where the robot is not permitted to enter.
- A **workspace** is a space where the robot is not permitted to leave.



There are different possibilities when defining safespaces and workspaces:

- As a **cuboid**, which is defined by specifying its position with respect to the MCS and its X, Y and Z dimensions.
- As a **half-space**, which means a plane defining a barrier that can't be crossed or left. It is necessary to define its area and its position with respect to the MCS.
- As a **truncated cone**, which is defined by its position with respect to the MCS and its dimensions (base radius, top radius and height).

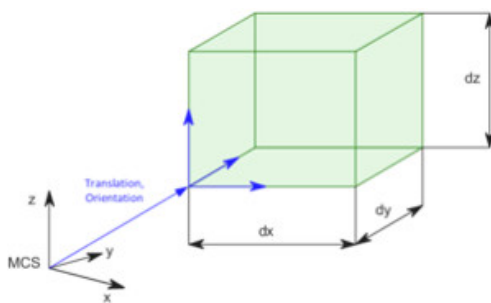


Figure 113: Cuboid workspace

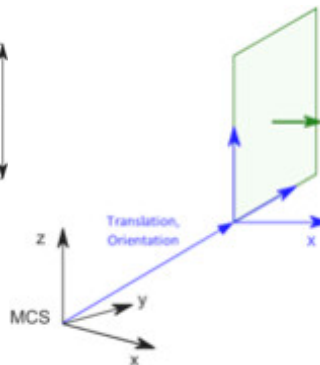


Figure 114: Half-space safespace

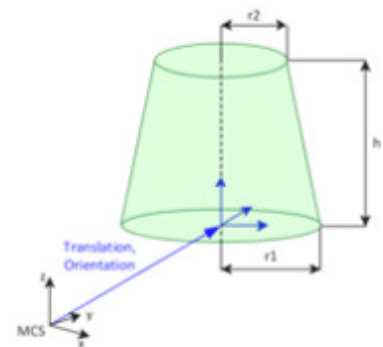


Figure 115: Truncated cone workspace

As many spaces as needed can be defined either as workspaces or safespaces. Combining different shapes and sizes, it is then possible to configure any desired environment and ensure that the robot will not collide with any obstacles or move outside its safe positions.

For additional information, see Automation Help.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Concept \ Range of functions \ Advanced functions \ Workspace monitoring](#)

11.1 Wireframe model

The wireframe model is a way of representing the 3D shape of the robot to detect collisions with workspaces and safespaces, as well as possible collisions with itself.

The model is made up of edges and distances. Each edge connects two important points within the mechanical system, for example two joints. The distance parameters define the minimum permissible distance between an edge and another monitored object, for example a workspace. All these elements are defined in the mechanical system configuration file.

When adding a new robot using the MCR assistant, all the edges are already defined but the distance for all of them is set to a default value. This means that the model of the robot is just a linear structure connecting all joints. For a more accurate representation is necessary to modify these values.

The following image shows how the wireframe model is designed to represent the physical robot.

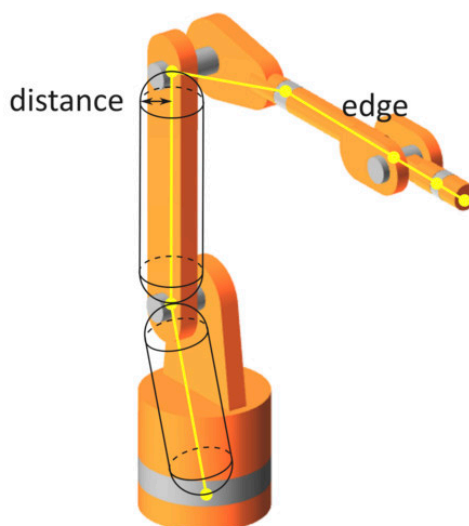


Figure 116: Example of a wireframe model for a 6-axis robot.

One way to easily understand how the system is described and how it detects collisions is to think about every edge line as a cylinder. Its center is the edge itself and its radius is the distance defined in the wireframe model. Then:

- If a safespace intersects with the cylinder assigned to an edge, the safespace is violated by that edge.
- If the cylinder associated with an edge is not fully contained within the workspace, the workspace is violated.
- If two cylinders intersect, their associated edges will collide.

There are different possibilities to define an edge:

- **Connecting line:** The entire line representing the edge is taken into account.
- **Point:** Only the endpoint is taken into account (each mechanical system defines an endpoint for each available edge).
- **Not used:** The edge is not taken into account.

Wireframe models can also be used to check for collisions with tools. The model of the tool can be used to extend the wireframe model of a robot so that the tool is also included within the dimensions of the model. Then the whole model can be used to avoid collisions with workspace monitoring objects and itself.

For additional information about all of these parameters and functionalities, see Automation Help.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Basic elements \ Workspace definition](#)

11.2 Self-collision detection

As previously mentioned, the workspace monitoring axis group feature can also be used to prevent collisions between the robot and itself. To do that, it is necessary to enable the "Self-collision detection" feature in the axis group configuration file.

Feature_WorkspaceMonitoring_D4r1100R_1			
Type	Workspace monitoring		
Modal data behaviour	Use axes group settings		Defines the modal data behaviour of the feature
Workspace reference			Name of the workspace reference
Self-collision detection	Enabled		Self-collision detection
Flange workspace	Enabled		Additional workspace monitoring of flange
Workspace reference			Name of the workspace reference

Figure 117: Self-collision configuration parameter.

When this parameter is enabled, a self-collision occurs when two non-adjacent edges touch each other, always taking into consideration the defined distance. Then it is determined that the robot would collide with itself and the movement is stopped before that occurs.

11.3 Using workspaces and safespaces in robotic programs

Workspaces and safespaces can be enabled and disabled during execution of a program using some ST motion commands. All possible shapes of spaces can be activated and deactivated and it is possible to enable and disable self-collision detection.



SafeCuboid
RemoveSafeCuboid



WorkHalfSpace
RemoveWorkHalfSpace



SafeTruncatedCone
RemoveSafeTruncatedCone



WorkCuboid
RemoveWorkCuboid



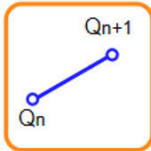
SelfCollisionOn
SelfCollisionOff



WorkTruncatedCone
RemoveWorkTruncatedCone



SafeHalfSpace
RemoveSafeHalfSpace



WireFrameModel

Figure 118: ST commands for workspace monitoring

For additional information about these instructions and how they can be used, see Automation Help.



[Motion Control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Programming languages \ Structured Text \(ST\) \ Workspace monitoring](#)

Exercise: Modeling the workspace

In this exercise, the workspace and safespaces for a machine are simulated in the digital twin (Scene Viewer). The goal is to create a space around the robot where it can move around without harming any mechanical elements or workers. The workspace will simulate a fence around the robot where it can move safely. Furthermore, the fixed elements in the scene, the picking and placing tables, must be added as safespaces so the robot does not collide with them when moving inside the workspaces.

- 1) To model a workspace, add a "Protected area" (fence) into the object hierarchy in Scene Viewer.
- 2) To model a safe area, add two "Protected areas" (one for the picking table and another for the placing table) in the object hierarchy in Scene Viewer.

Modify the color and opacity properties to have a better view of the different areas and how they interact with the robot digital twin.

**Result:**

For the suggested pick-and-place application, suitable positions for the workspace and the safespaces are proposed in the following section:

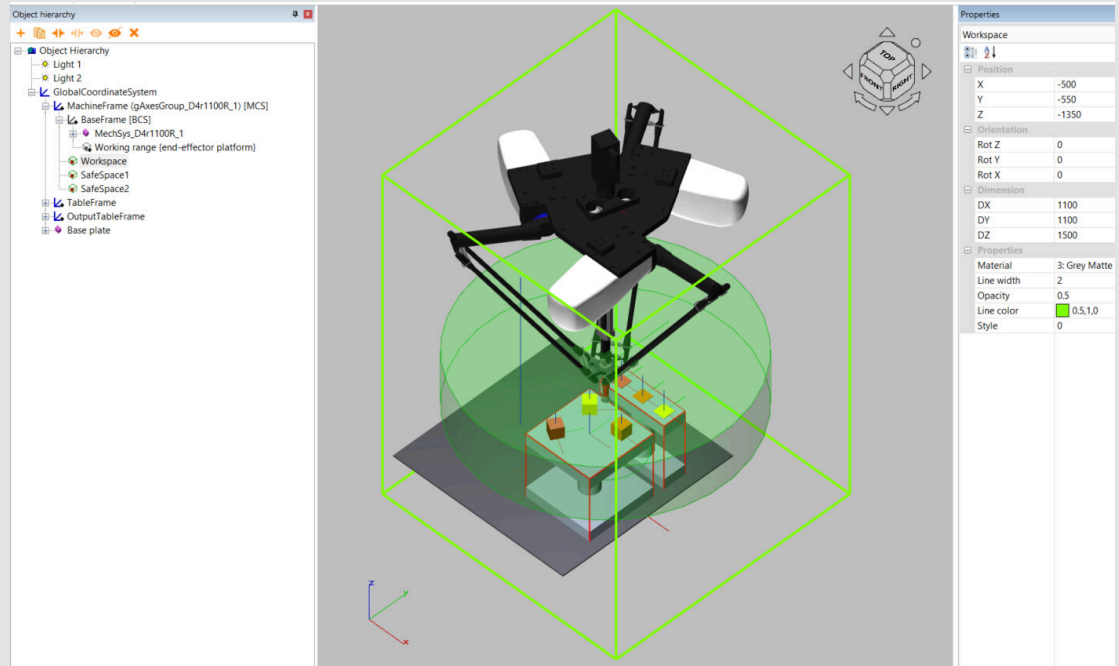


Figure 119: Workspace and safespaces in the scene

Delta robot:**Workspace:**

- Position: X = -500, Y = -550 and Z = -1350
- Dimension: X = 1100, Y = 1100 and Z = 1500

Safespace for the picking table:

- Position: X = -150, Y = -225 and Z = -1350
- Dimension: X = 300, Y = 300 and Z = 300

Safespace for the placing table:

- Position: X = -150, Y = 125 and Z = -1350
- Dimension: X = 300, Y = 100 and Z = 300

Exercise: Configuring and enabling workspace monitoring

In this exercise, workspace monitoring for the machine is configured in mapp Motion. A work area where the robot can move inside a controlled space and two restricted areas to protect the work tables are defined.

Configuration

- 1) Add the object catalog element "Workspace Definition" to the Configuration View (mapp Motion).
- 2) Define "WorkSpace" for the permitted workspace.
- 3) Define two "SafeSpaces" for the work tables.
- 4) Switch to the mechanical system configuration to adjust the spacing in the wireframe model.
- 5) Add the axis group feature "Workspace Monitoring" to the Configuration View (mapp Motion).
- 6) Select the workspace definition in the workspace monitoring feature.
- 7) Enable self-collision detection.

- 8) Link the "Workspace Monitoring" feature in the axis group configuration.
- 9) Transfer the project and perform a warm restart.

Checking the configuration

- 1) Perform jog movements and check whether the robot comes to a standstill within the defined working space.
- 2) Perform robot movements with a target position outside the workspace and check whether these are aborted with an error on time.

Further information



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Concept \ Range of functions \ Advanced functions \ Workspace monitoring](#)
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Basic elements \ Workspace definition](#)
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Basic elements \ AxesGroup feature \ Workspace monitoring](#)
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Basic elements \ Mechanical System \ Delta](#)
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Basic elements \ Wireframe](#)
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Concept \ Range of functions \ Basic functions \ Operating modes \ Jogging](#)



Results:

The position and dimension parameters needed to configure the workspace and both safespaces are provided in the previous exercise. The Workspace and Safespace refer to the MCS.

12 Commissioning and diagnostics

The section below shows general information about commissioning and diagnostics of robotics. Commissioning robots is often a crucial part, and MCR is designed to reduce the effort as much as possible. Since the robotic system developed up to now is fully configured to be executed on real hardware, there are not many steps needed to be able to move the robot. As well, all released MCR robot configurations are prepared to fit a wide range of applications, so no tuning is needed.

Furthermore, diagnostics options are listed here even though some of them have already been explained in order to provide a summary of the most common features.



Prerequisites for commissioning

When using Codian with MCR, the robot always comes with safe motors and ACOPOS P3 with SafeMOTION.

Therefore it is also necessary to get familiar with mapp Safety and SafeMOTION to understand the concept and how to commission with safety devices.

B&R offers training materials after successful login on the homepage as well as trainings remotely and in classroom.



For more info regarding our seminars and materials, please have a look to our homepage in the section Academy and log in:

<https://www.br-automation.com/de/academy/>



[Hardware \ Mechatronic systems \ MCR Codian Robots](#)

[Motion control \ mapp Motion \ mapp Robotics/CNC \ Getting started \ MCR Codian Delta robots](#)

12.1 Unboxing and assembling Codian robots

Ordering a robot in terms of MCR (Machine Centric Robot-ic) brings a lot of advantages for the user.

On the one hand there is the MCR package in software which means to get a setup package that has a pre-tuned robot and all necessary settings for Automation Studio to start the project with the desired robot. These packages are available on the B&R homepage or via the Tools Upgrade dialog known from Automation Studio.

On the other hand the MCR package also means in hardware terms to get a robot like Codian ready for final assembly with B&R motors. The mechanics fits to the software package and can be used after project insertion. Once the project is set up and tested in simulation the last step is to mechanically prepare the robot itself to change from simulation to real world.

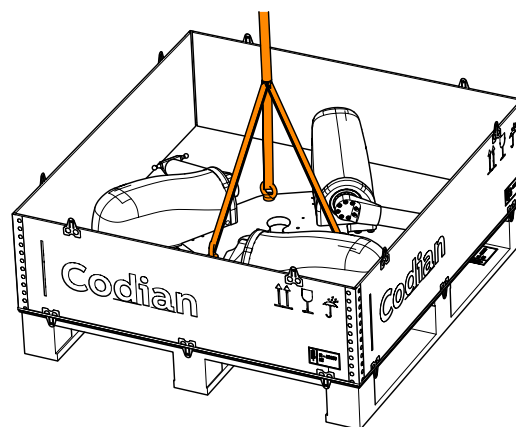


Figure 120: Robots of Codian comes within a wooden box on an euro pallet



Be careful when lifting the robot out of the box and refer to the MCR user manual for safety advise.

Lifting other robots

Depending on the size and amount of axes the robots look slightly different to each other. Nevertheless it's mandatory to take care of the instructions on the Codian or MCR manual that comes with each robot.

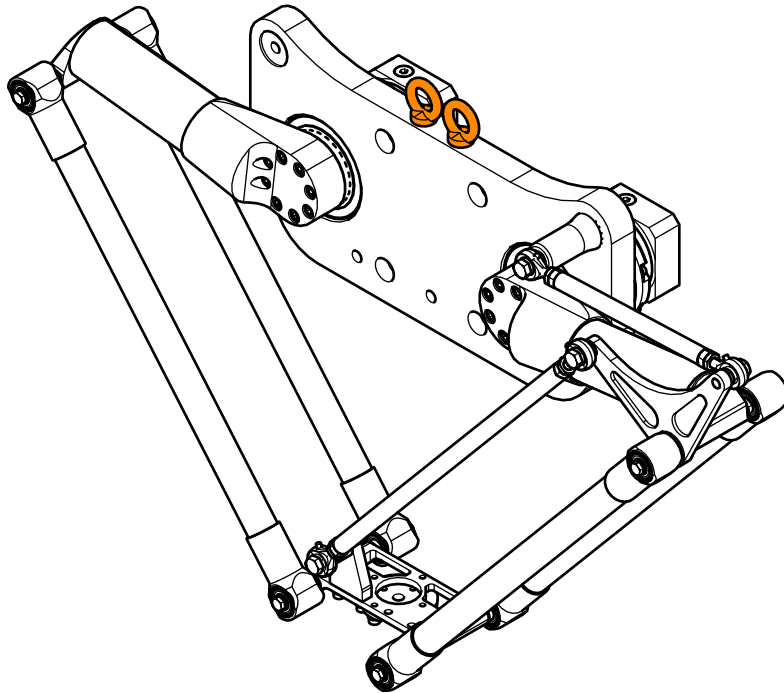


Figure 121: Lifting a D2 with the predefined eyes

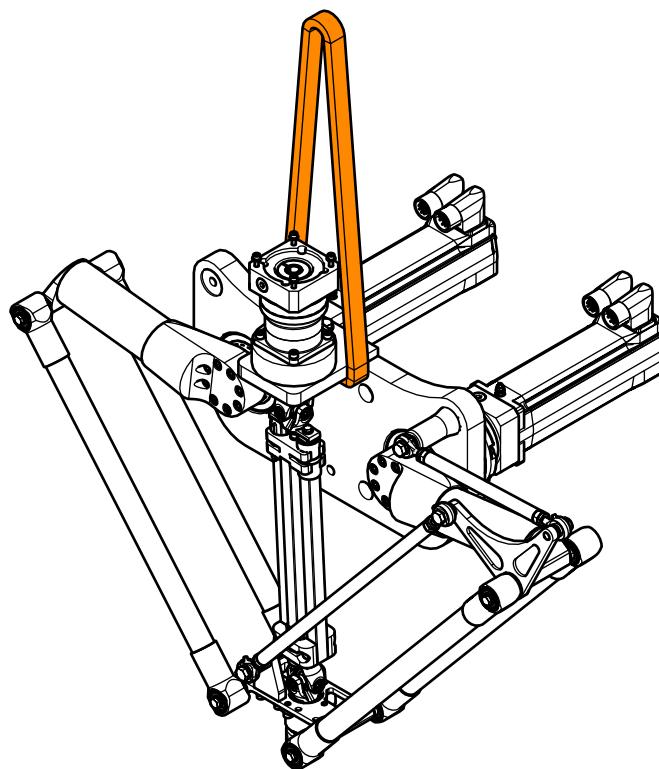


Figure 122: Lifting a D2 with recommended straps

Arms mounting

After the base plate of the robot is mounted on the final position, the arms can be easily assembled.

Depending on the size of the robot there might be a required tool to attach the arms with the springs on the bearings. On the smaller sizes it does not require any tool at all.

These mounted arms also comes with the feature of protection for the delta robot mechanics. Its light weight enables fast movements and whenever a mechanical obstacle is hit, it does not necessarily mean the whole mechanic is broken or out of shape. The arms might easily fall off during collision but this can also be fixed very easily without the help of B&R.

If the stretching of the springs went too far and they get loose, they have to be replaced. Otherwise the arms might fall off even on faster movement. They are available as spare parts from B&R or Codian respectively.

See following pictures for more details:

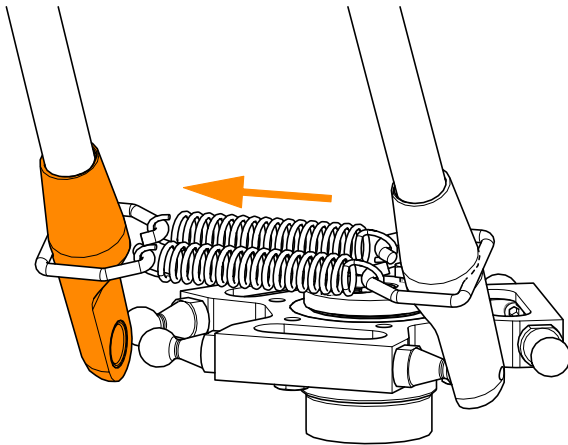


Figure 123: ... attach one arm and be careful to not stretch the spring too much ...

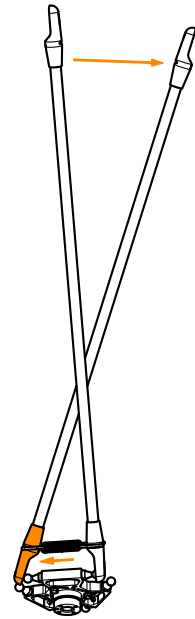


Figure 124: Start with the endeffector platform first...

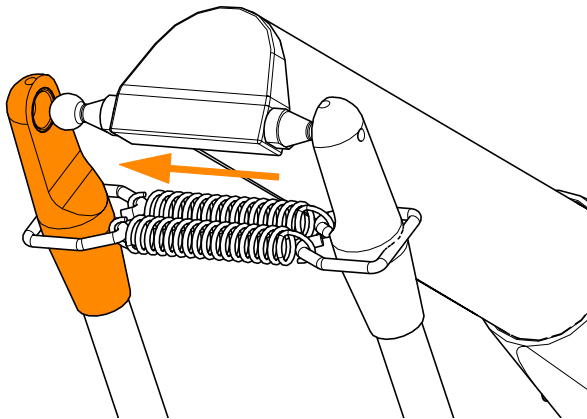


Figure 125: ... then mount the secondary arms with the arms from the base plate

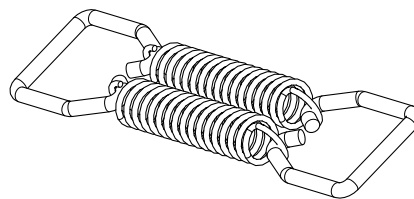


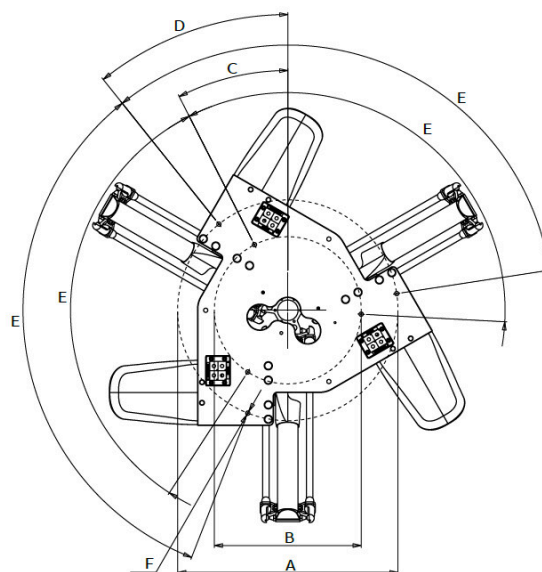
Figure 126: Spare spring of a Codian robot

12.2 MCR user's manual

B&R has a user manual for each type of robot. In this user manual, it is possible to find information about different variants, dimensions and working ranges and topologies as well as technical data for individual components, wiring and much more. In this training manual, a short excerpt of the most relevant information is listed.

Mounting

Mount the robot mechanics on a suitable, high-quality frame. Ensure the baseplate is installed horizontally with the TCP facing downwards. For other orientations, contact sales support for guidance.



Limits of movements

The image shows the positive and negative limit for an upper arm.

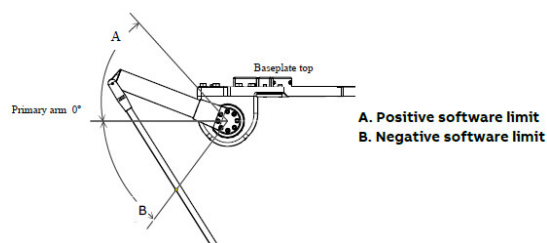


Figure 127: Direction of movements of a delta robot

Working range

The working range in most cases is a very important requirement that is decided before getting the actual robot. But it is also important to check before doing the first movement if the workspace is free of any obstacles that could cause damage. On the other hand, it is worth having a look whether the axis limits are set correctly in the Automation Studio configuration.

For MCR installers the working range is set in the mechanical system and protects the robot for movements outside the working range. Following picture shows a possible definition for different delta robots.

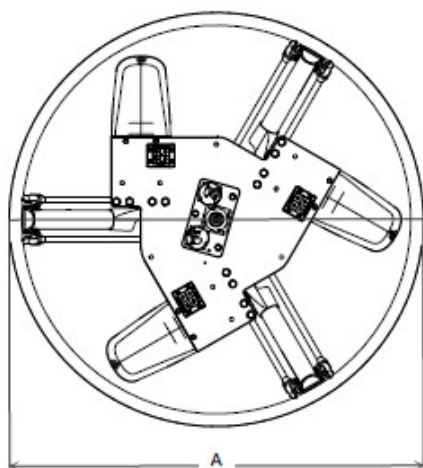


Figure 128: Working range seen from above

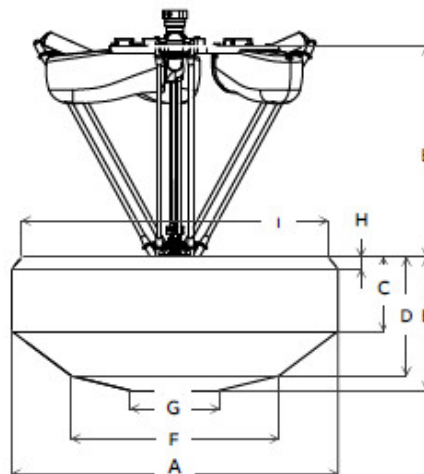


Figure 129: Working range seen from the side

	D4-ST21-700	D4-ST21-900	D4-ST21-1100	D4-ST21-1300	D4-ST21-1600
A	700 mm	900 mm	1,100 mm	1,300 mm	1,600 mm
B	525 mm	595 mm	710 mm	720 mm	875 mm
C	205 mm	305 mm	250 mm	305 mm	350 mm
D	305 mm	455 mm	400 mm	510 mm	670 mm
E	355 mm	530 mm	500 mm	680 mm	810 mm
F	400 mm	500 mm	700 mm	900 mm	900 mm
G	100 mm	150 mm	200 mm	200 mm	200 mm

Figure 130: Working range values of different delta robots

Load diagram

A load diagram shows the maximum load depending how far its center of gravity is offset in respect to the TCP. The further away the tool, including product, is mounted the lower the weight can be to still achieve the given requirements in terms of speed and accuracy. To easily check a specific load case, there might also be a software program to assist in simulation.

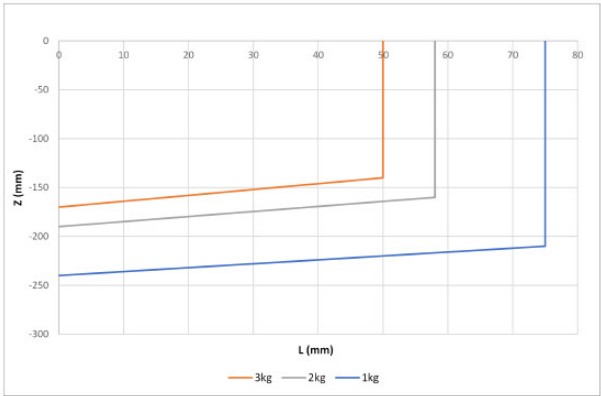


Figure 131: Load diagram for a Delta robot

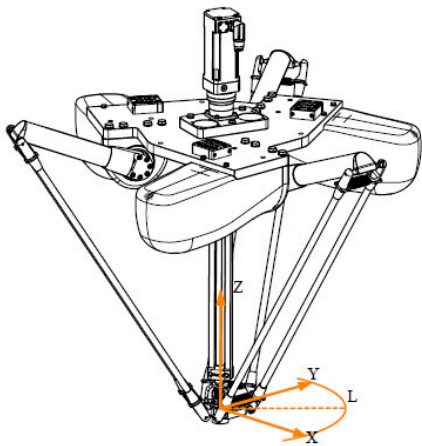
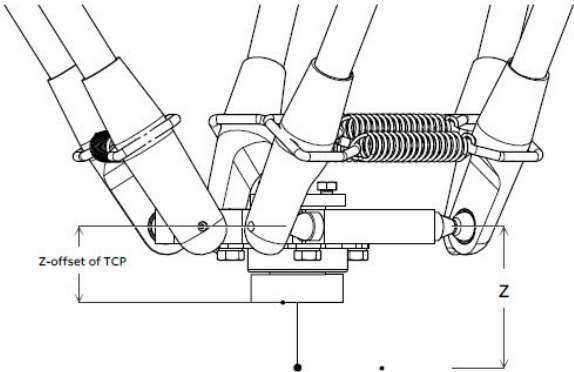


Figure 132: End effector platform and flange of a Delta robot



Definitions	Description
L	Distance in the X-Y plane from the Z-axis to the mass center of gravity of the load.
Z	Distance from the center of the TCP to the mass center of gravity of the load.
Jz	Inertia load on the rotation axis around the Z-direction.
Mass	Mass load applied to the robot.

Figure 133: Abbreviations load diagram



For additional information, see the MCR user manual for each robot.

12.3 Commissioning checklist

The following list shows general hints to help the operator when commissioning MCR robotic systems. Nevertheless, we strongly recommend looking in the user manual before installing the robot and executing commands. The list is not complete and the operator is responsible for any action taken.

Check Automation Studio configurations

- ▶ Axis settings, especially limits
- ▶ Mechanical system
- ▶ Workspace

Check environment

- ▶ Remove all obstacles surrounding the robot
- ▶ Make sure the robot arm cannot touch anything

Check safety application

- ▶ I/O mapping (trigger emergency stop)
- ▶ Check variable status in SafeDESIGNER
- ▶ Check brake output (in case it is controlled by the safety application)
- ▶ Check drive enable - if it drops with e-stop (mapp Cockpit, single axis)

Check home positions (in case of 6-axis robot)

- ▶ Home the axis group and check all axes positions
- ▶ All axes should be in their home positions if the robot is standing on the calibration marks
- ▶ If not, calibration is not done correctly, calibrate again

Check direction of movement

- ▶ Trigger movement, e.g. move absolute to a reachable position
- ▶ Verify direction and position, e.g. 45° and visually check the position

Check mechanics against digital twin

- ▶ Move robot to a certain position in MCS
- ▶ Do the same with the model in Scene Viewer
- ▶ Compare joint positions of both systems

Check workspace monitoring

- ▶ Keep emergency stop close to you
- ▶ Trigger an absolute movement just a little outside the workspace
- ▶ Check if movement is executed
- ▶ If so, hit the emergency stop

Execute robotics program

- ▶ Simple program with easy instructions, e.g. movement around a small cube with fixed feed rate (e.g. Exercise: Create robotics programs)
- ▶ Start with low override (e.g. 5%) and increase in small steps



Please read the user manual for the respective robot before commissioning.

12.4 Optimization

Optimization of robotic systems can achieve better repeatability and positioning accuracy or shorter execution times or even both. There are many different parameters that can be adjusted.

There are parameters that can be set together with movement commands to generate a smoother path, e.g. to use rounding corners where the point itself doesn't have to be approached. In this case, the robot doesn't need to decelerate, stop at the point and accelerate again, which results in higher speeds and thus a lower processing time. Other

parameters like the limit check resolution are influencing the trajectory, but on the other side are stressing the controller. Therefore, this parameter should not be changed lightly and if done it should be done carefully to avoid problems in the system.

Calibration

Standard or fine calibration can improve accuracy, see section "Maintenance and calibration".

Trajectory and geometry

Parameter	Description	Influence
Limit check resolution	Time-based resolution in which the set limits and new stop trajectory are calculated	Trajectory
Buffer time	The "Buffer time" is used to specify the predicted processing time of the trajectory planner. It sets the response time of the planner to modifications in the trajectory (e.g. override change or controlled stop)	Trajectory
Rounding mode	Influences the algorithm used for rounding calculations	Geometry
Maximum corner deviation	Corner rounding can be configured so that smooth transitions are added automatically. Any type of geometry that is not tangentially connected to each other is smoothed	Geometry
Maximum tangential transition deviation	Tangential rounding can be configured so that smooth transitions are added automatically. Any type of geometry that is tangentially connected to each other is smoothed	Geometry
Maximum radius deviation	Defines the tolerance if circle interpolation is programmed with numeric inconsistencies	Geometry

Table 5: Influencing parameters



[Motion control \ mapp Robotics/CNC \ Configuration \ Basic elements \ Axes Group \ PathGen Axes-Group](#)
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Concept \ Technical information \ Maximum stop trajectory length \ Restricting path velocity \ Details about LimitCheckResolution \(TLCR\)](#)
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Programming languages \ Structured Text \(ST\) \ Geometry planning](#)

Velocity limitation

If the velocities at the monitoring points of the robot are taken into account, the velocity at these points always remains below the limit. This allows other points and joints of the robot to be limited to a velocity setpoint in addition to the TCP (the current limiting point).



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Concept \ Range of functions \ Advanced functions \ Velocity limitation of monitoring points](#)

Jerk limits

If the jerk limits of the robot axes are taken into account during movement planning, it is possible to achieve time-optimized movements and protect the mechanics at the same time. This particularly benefits mechanical systems where vibrations should be avoided.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Concept \ Range of function \ Basic function \ Limit values \ Jerk limits](#)

Force and torque limits

If the force and torque limits of the robot axes are taken into account during movement planning, the maximum dynamics of a movement should ideally be coordinated with the current position of the robot. This enables the full potential of a robot to be exploited.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Concept \ Range of function \ Basic function \ Limit values \ Force and torque limits](#)

12.5 Synchronized stopping of an axis group

Stopping individual axes, e.g. via emergency switch-off with torque cutoff, usually only causes minor problems, which can be solved by application of a friction brake if there are problems with coasting to a stop.

In the case of an axis group or path-controlled movement, however, the situation is more complex.

In the event of an emergency switch-off, the user should be aware that some axes come to a stop more quickly than others.



Figure 134: Emergency switch-off button

To remain on path until a standstill is reached, it would be recommended to implement a time delay between when the emergency switch-off button is pressed and when the torque is cut off (if possible from a safety perspective).

To do this, a stop must be performed at the robotics level – e.g. stop program with MpRoboArmXxx, stop group with MC_GroupStop (McAxGroup) or another function or function block able to implement a stop on the path.

Only then should the axes' torque be shut off.

For more details, see the Automation Help.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Libraries \ Technology libraries \ MpRobotics \ Function blocks](#)
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Libraries \ Core libraies \ McAx-Group](#)
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Concept \ Range of functions \ Basic functions \ Stopping a path-controlled movement](#)

12.6 Maintenance and calibration

Maintenance

There are different components to be maintained depending on the robot. Instructions which components that have to be maintained in specific intervals are shown in the user manuals. In most cases, the operator of a 6-axis robot has to take care of following components:

Component	Interval	Task
Gearbox oil for Q1 - Q4	40000 h	Maintenance free
Gearbox oil for Q5 - Q6	20000 h	Replace
Battery for measurement system	Alarm for low battery	Replace
Manipulator	Regularly	Cleaning

Table 6: 6-axis robot example: Maintenance information

Calibration

There are different reasons for a robot to require a calibration of its axis. Since MCR is covering a wide range of robots with different systems, the calibration method can vary and always needs to be taken from the user's manual for each robot. In this section, some information regarding different systems is explained. Basically, calibration is used to

synchronize the mechanical position with the software position of each axis and can be used to improve the accuracy of robots. To align the mechanical position with the software position, different steps must be done for different robots. There are different reason why calibration is needed and some of them are listed here.

Reasons for calibration

- Encoder values have changed (e.g. in case motor or parts of transmission are replaced)
- The revolution counter memory is lost (e.g. battery is discharged, signal between encoder and measurement board is interrupted or axis is moved with control system disconnected)
- Robot is rebuilt
- Robot is not floor mounted
- After emergency stop

For 6-axis articulated robots, in most cases there are calibration marks, also called synchronization marks, created for each axis on the housing of the robot. Depending on the calibration process needed, the operator has to move the axis to the synchronization mark.

For delta robots, the calibration positions are achieved by either moving the primary arm to the upper end position or by using certain calibration tools where the user can move the primary arms Q1 to Q3 (e.g. Codian D4). The fourth axis, if one exists, has a calibration mark near the flange. The operator must move the robot to the axis calibration marks and trigger the calibration via mapp Cockpit.



Please read the user- or MCR manual for the respective robot for the correct calibration procedure.

To reach the synchronization marks of a robot, the brakes maybe needs to be opened manually and carefully move the axis. mapp Cockpit can be used to move the axis more precisely. Once the synchronization marks are reached, it is recommend to power off the axis group in order to close the brakes, which results in a more accurate calibration position.

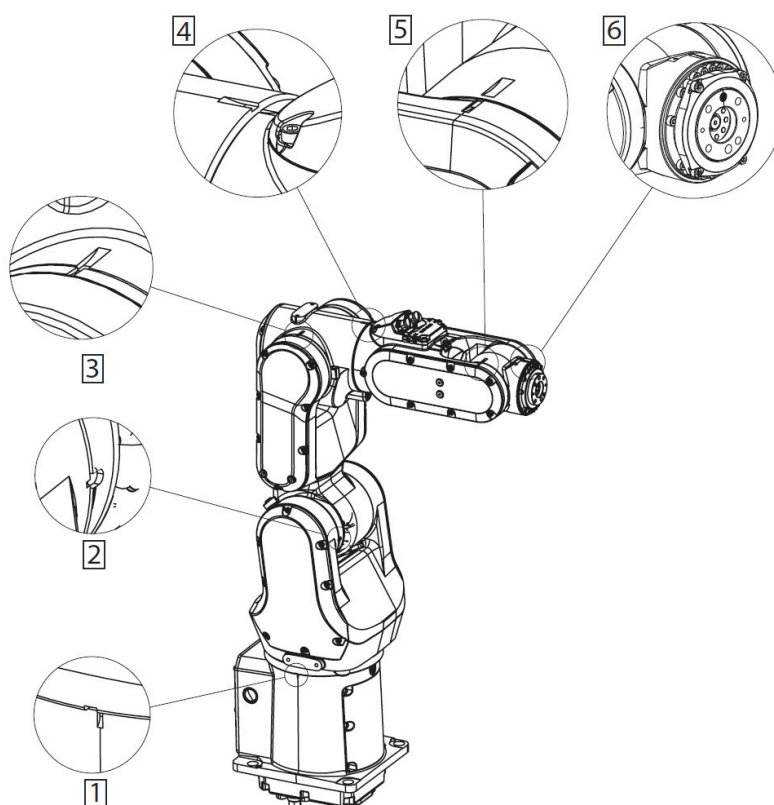
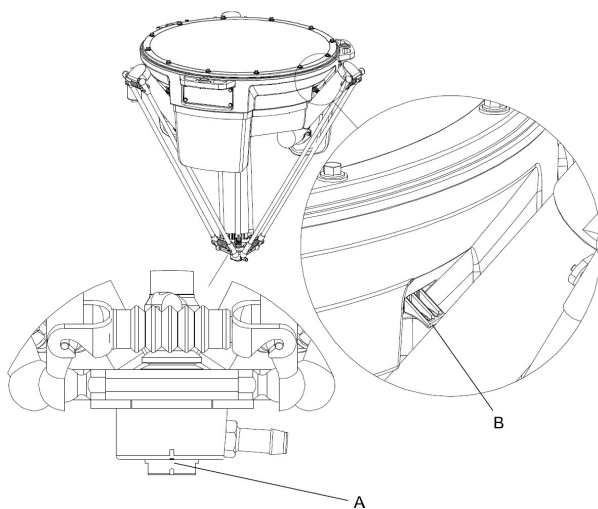


Figure 135: Example of a robot - Calibration or synchronization marks



The image shows the synchronization marks for axes Q1 to Q3 of a delta robot. It is recommended to consistently move the axis towards the calibration position in the same direction in order to avoid position errors, e.g. caused by backlash. The fourth axis is calibrated using the mark near the rotatory flange.

Figure 136: Delta robot example - Calibration or synchronization marks

Codian delta robot - Calibration

- Q1 - Q3
 - Mount the calibration tool on the base plate of the robot at the axis that needs to be calibrated
 - Move the primary arm to the end position of the calibration tool
- Q4
 - Move the axis to the calibration mark
- Close the brake once the end position is reached
- Perform absolute homing
- Save the offset in the drive configuration "Home position"

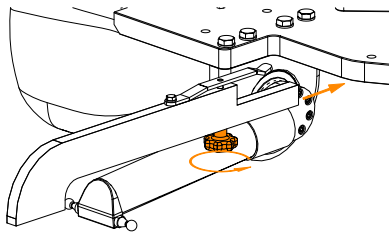


Figure 137: Codian D4 calibration tool mounting on base plate ...

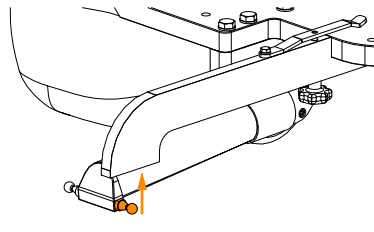


Figure 138: ... release motor holding brake and move to touch point of calibration tool ...

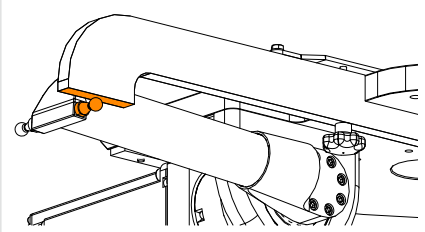


Figure 139: ... engage the brake and save this position

Mounting arms to Codian Delta

The arms of a Codian Delta robot are held with springs on bearings. This offers the feature, that on high load with high acceleration, crashing with obstacles or other unwanted scenarios the arms and the robot are not going to be damaged. So during first set up or during runtime this scenario might happen. If the spring is getting loose, think of replacing it. More information on commissioning can be found in the MCR user manual.

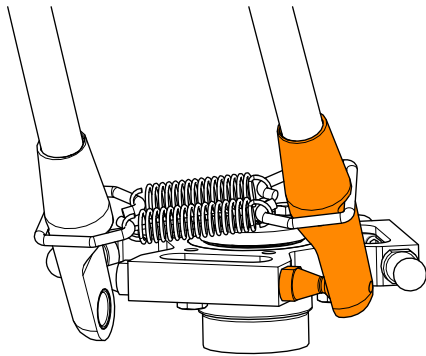


Figure 140: Start with one side...

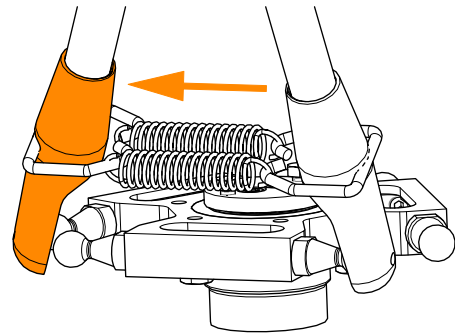


Figure 141: ... push the spring just enough to mount the endeffector platform ...



Figure 142: ... use the leverage in the beginning

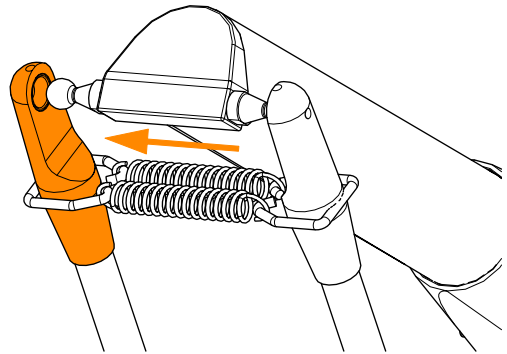


Figure 143: ... finally also mount on the robot arm and maybe tools are required to extend the spring.

Replace bearings at Codian Delta

The bearings are maintenance free in terms of lubrication. They consist of plastic and wear out over some time. When this happens, they need to be replaced. This can be done quite easily. More information on commissioning can be found in the MCR user manual.

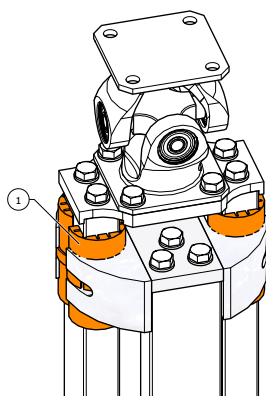


Figure 144: For a rotary axis this are the plastic bearings...

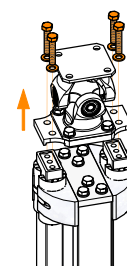


Figure 145: ... remove the screws step-by-step to get access to the bearings

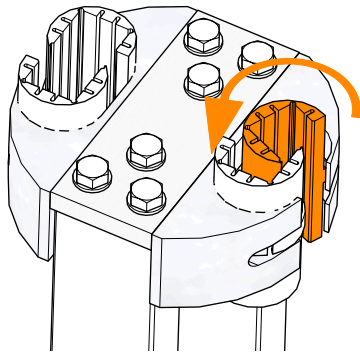


Figure 146: Remove the bearings by turning them.

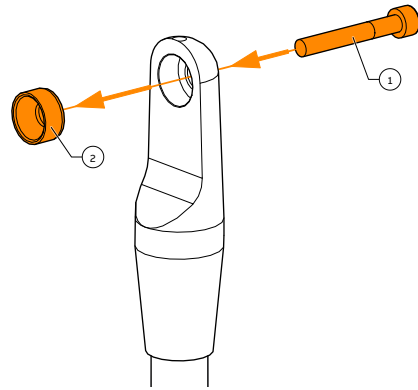


Figure 147: On the arms either with air pressure or with a bolt the bearing can be removed easily

12.7 Diagnostic tools

There is a wide range of diagnostic tools available, which are briefly introduced in the following chapter. Some of the tools are integrated in Automation Studio (e.g. watch window and logger) whereas others are web-based such as the mapp Cockpit or the System Diagnostics Manager (SDM).

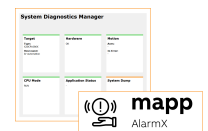
Automation Studio

- Watch
- Status outputs
- Info structure (MpRobotics function blocks)
- Logger
- Drive Log (Network Command Trace)



Integrated in tools

- System Diagnostics Manager (SDM)
- MpAlarmX
- WebXs
- Monitoring elements (axis group feature)
- Single-step mode (McAxGroup)
- Motion packet log (or MP log)



mapp Cockpit

- Watch
- Trace
- ParlDs
- MCR service site



Scene Viewer

- Check mechanics against digital twin
- Check workspace and safespace and compare with features
- Move robot to target points and see if reachable or how the axes are positioned
- Verification of movement commands and ST programs in simulation



User's manual for each robot

- Verify mechanical system, workspace and direction of movement for each axis
- Cabling and wiring
- Manual brake operation
- Maintenance and life cycle of components



In the following paragraphs, you can find details about the diagnostic tools that have not yet been explained.

Info structure (MpRobotics function blocks)

All mapp technology function blocks have an output that displays additional information. In case of a mapp Robotics function block such as the MpDelta4Axis, a lot of information can be intercepted:

- Joint positions (JACS)
- Program information with program monitor
- Path information
- Information about internal error (Diag)

[illegible]

Figure 148: MpDelta4Axis: Info structure of function block

Drive Log (Network Command Trace)

The mapp Motion Drive Log primarily logs commands sent to and from the drive and therefore also saves commands coming from the motion chain. In addition, data records for PLC-controlled axes or information from the PLC itself are also logged. In the following example, you can see axis commands sent to the drives after executing an ST program.

[illegible]

Figure 149: Network command trace - Axis commands after executing ST program

An instantaneous recording of the network command trace can be created and loaded from the target system in mapp Cockpit or under "Motion \ "NW command trace" in the SDM. The evaluation is performed either directly in mapp Cockpit or can be exported and diagnosed in Automation Studio.

 Use the tool tip to get more detailed information about the log entry.

 [Diagnostics and service \ mapp Cockpit \ Web-based HMI application \ Tools \ Drive log](#)
[Diagnostics and Service \ Diagnostics tools \ System Diagnostics Manager \(SDM\) \ Accessing System Diagnostics Manager \ SDM - SVG pages \ SDM page - Motion / NW Command trace](#)

System Diagnostics Manager (SDM)

The System Diagnostics Manager (SDM) can be used to diagnose the controller via a web browser⁵.

The only requirement for these diagnostics is an Ethernet connection to the controller.

SDM functions:

- General system overview
- Showing and saving Logger files
- Overview of installed software objects
- Hardware modules and I/O status
- Motion control diagnostics
- Creating system dumps

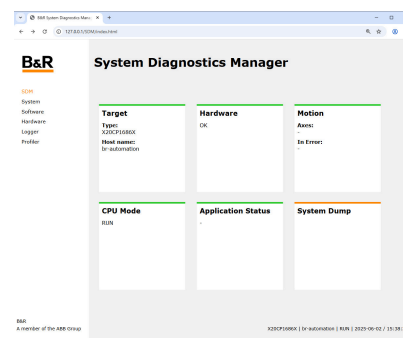


Figure 150: SDM startup screen



[Diagnostics and service \ Diagnostics tools \ System Diagnostics Manager \(SDM\)](#)

MpAlarmX

mapp AlarmX is a modular, full-featured alarm system. The controller registers alarms with microsecond accuracy and can forward them on to other systems. Alarms can trigger actions such as opening a PDF file, playing an instructional video or displaying a virtual model of the machine with the location of the fault highlighted.

With mapp AlarmX, it is possible to develop alarm handling for a whole line where alarms from an integrated machine part, e.g. the axis group of the robot, can be escalated to a higher machine group component, e.g. a line control that reacts accordingly. Configuration options enable the modular use of each alarm from the single axis to axis group alarms. The alarm details are used to get more specific information about the respective alarm.

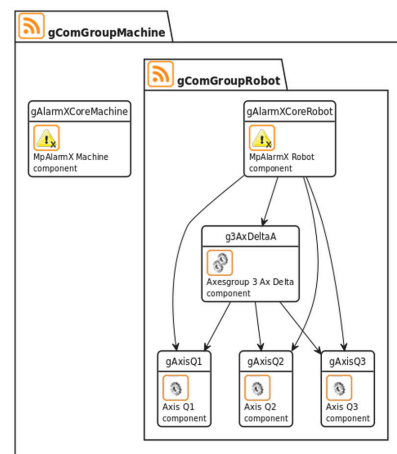


Figure 151: MpAlarmX: Possible mapp hierarchy



[Services \ mapp Services \ mapp AlarmX: Alarm management](#)

Single-step mode (McAxGroup)

Single-step mode interrupts the selected motion chain module at each executable program line (empty lines or lines with comments are skipped). The motion chain module is interrupted at the start of an executable line and programs can be processed line-by-line. This option makes it possible to diagnose each line by itself and respective actions can be observed individually. The single-step mode can be enabled via mapp Cockpit as well as with a function block from library McAxGroup. In both ways, there is the option to go in single steps through the motion chain of the interpreter or the set value sampler.



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Programming languages \ Structured Text \(ST\) \ Diagnostic functions \ Suppressing single-step mode](#)
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Libraries \ Core libraries \ McAxGroup \ Function blocks \ MC_BR_SingleStep](#)

Motion packet log (MP log)

The motion packet log MP log is an axis group feature and can be used to log the NC program during runtime. Later the log can be used to evaluate the commands sent from the interpreter to the path planner. To enable the MP log, a configuration file has to be added and a file device chosen. A log entry is created automatically whenever a program is started. Another helpful feature is that the MP log can be executed using the function block MC_BR_MoveProgram in order to generate a motion packet data stream.

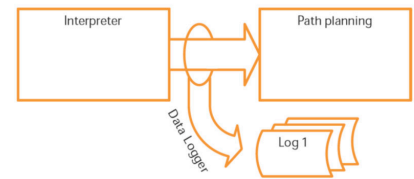


Figure 152: Motion packet log MP log



[Motion control \ mapp Motion \ mapp Robotics/CNC \ Programming \ Programming languages \ Structured Text \(ST\) \ Diagnostic functions \ Motion packet log](#)
[Motion control \ mapp Motion \ mapp Robotics/CNC \ Configuration \ Basic elements \ AxesGroup feature \ Motion packet log](#)

ACOPOS parameter IDs (ParIDs)

ACOPOS parameter IDs are unique codes for accessing drive data. In terms of MCR, in most cases there is no need to use ParIDs because all information or functionalities are accessible via the user interface mapp Cockpit. Nevertheless, there is a full list of available parameters in Automation Help.



[Motion control \ Drive functions \ ACOPOS Parameter-IDs](#)

13 Summary

B&R offers robots and others as an integral part of its automation system. Customers benefit from unprecedented precision in synchronization between robotics and machine control. Merging robotics and machine control into a unified architecture enables manufacturers to follow the trend of individualized mass production and optimize their processes in batch size one. With B&R, machine builders will receive robots and automation solutions from a single source. They will only need one controller and a system for development, diagnostics and maintenance. The entry hurdle for the use of robotics will be massively reduced. Codian robots are programmed in the B&R development environment just like any other automation component. The robotics controller is then an integral part of the machine application. Through the fusion of machine and robot controllers, movements can be synchronized with microsecond precision.

Creating applications easily

B&R simplifies the creation of robotics applications with numerous functions. With the ready-made mapp technology software modules, developers can easily parameterize the machine application, including robots. Knowledge of special robotics languages is not required. Safe robot applications can also be simple and easy.



Robots are easily inserted into Automation Studio using the MCR assistant. After choosing a specific robot out of huge portfolio, the associated drives and all necessary components are inserted automatically. The first movements are done with mapp Cockpit and can be observed using the digital twin software Scene Viewer in simulation. The powerful simulation environment provided by Automation Studio allows testing of robotic applications right at the workstation, further helping to reduce the amount of time needed for commissioning.

First, basic information is shown in regards to movements and parameters. Structure Text robotic programs are created and developed in Automation Studio, stored on a file device and executed to be analyzed with various tools. Positions and orientations of components in the scene or other target points are easily defined in frame hierarchies, which simplifies complex robotics tasks. The use of tools is another feature that makes robotic programming easier and enables a tool change without changing the target points.

MCR is bringing robotics and machine control together and all tasks are developed in Automation Studio. mapp Robotics includes technology function blocks designed in accordance with other mapp technologies and the user benefits from a high quality ecosystem. Now, the machine application and robotic programs can easily be synchronized, e.g. by using different commands for blocking further execution or functionalities like M-functions. Exchanging data from one to the other has never been this effortless. The robot can be protected from crashing into itself by activating the self-collision detection or set up a workspace in which the robot is allowed to move.

At the end of the seminar participants will be shown where to find important technical information, how to optimize a robot system, how to commission, calibrate and maintain a real robot system. Furthermore, a list of the most commonly used diagnostic tools is listed.

14 Appendix



On the [B&R GitHub repository](#) you will find an Automation Studio project that provides a complete solution for all exercises of this training manual:
[MCR solution project](#)

Final machine application for a Delta robot:

```
PROGRAM _INIT
// Set Initial Robot Parameters
  RobotParameters.ProgramName := 'Program1.st';
  RobotParameters.CoordSystem := 0;
  RobotParameters.Velocity := 100;
  RobotParameters.Acceleration := 1000;
  RobotParameters.Deceleration := 1000;
END_PROGRAM
PROGRAM _CYCLIC
// Error detection
  IF MpDelta4Axis_0.Error THEN
    Step := ROBOTCTRL_ERROR;
  END_IF;
// Robot Control Step Machine
CASE Step OF
  ROBOTCTRL_DISABLED:
    IF cmdTurnOn THEN
      cmdTurnOn := FALSE;
      Step := ROBOTCTRL_POWER;
    END_IF;
  ROBOTCTRL_POWER:
    IF MpDelta4Axis_0.Info.ReadyToPowerOn THEN
      MpDelta4Axis_0.Power := TRUE;
      Step := ROBOTCTRL_WAIT_POWER;
    END_IF;
  ROBOTCTRL_WAIT_POWER:
    IF MpDelta4Axis_0.PowerOn THEN
      Step := ROBOTCTRL_HOME;
    END_IF;
  ROBOTCTRL_HOME:
    MpDelta4Axis_0.Home := TRUE;
    Step := ROBOTCTRL_WAIT_HOME;
  ROBOTCTRL_WAIT_HOME:
    IF MpDelta4Axis_0.IsHomed THEN
      MpDelta4Axis_0.Home := FALSE;
      Step := ROBOTCTRL_READY;
    END_IF;
  ROBOTCTRL_READY:
    IF cmdTurnOff THEN
      cmdTurnOff := FALSE;
      MpDelta4Axis_0.Power := FALSE;
    END_IF;
    IF NOT(MpDelta4Axis_0.PowerOn) THEN
      Step := ROBOTCTRL_DISABLED;
    END_IF;
  ROBOTCTRL_ERROR:
    // Error handling
    IF MpDelta4Axis_0.Error THEN
      IF MpDelta4Axis_0.PowerOn THEN
        Step := ROBOTCTRL_READY;
      ELSE
        Step := ROBOTCTRL_DISABLED;
      END_IF;
    END_IF;
END_CASE
```

```

END_CASE;
// Call MpDelta4Axis FBs
MpDelta4Axis_0.MpLink := ADR(gAxesGroup_D4r1100R_1);
MpDelta4Axis_0.Parameters := ADR(RobotParameters);
MpDelta4Axis_0.Override := 100.0;
MpDelta4Axis_0.Enable := TRUE;
MpDelta4Axis_0();
END_PROGRAM

```

Robotics programs for each exercise:

- Exercise 7: Create robotics programs

```

VAR CONSTANT
    qInitialPos : McAxisTargetType := (JointAxis:=[0,0,0,0]);
END_VAR

VAR CONSTANT
    VertexPos1 : McPointType := (Pos:=(X:=300,Y:=+300,Z:=-1000),
    Orient:=(Angle1:=0,Angle2:=0,Angle3:=0));
    VertexPos2 : McPointType := (Pos:=(X:=300,Y:=-300,Z:=-1000),
    Orient:=(Angle1:=0,Angle2:=0,Angle3:=0));
    VertexPos3 : McPointType := (Pos:=(X:=-300,Y:=-300,Z:=-1000),
    Orient:=(Angle1:=0,Angle2:=0,Angle3:=0));
    VertexPos4 : McPointType := (Pos:=(X:=-300,Y:=300,Z:=-1000),
    Orient:=(Angle1:=0,Angle2:=0,Angle3:=0));
END_VAR

VAR CONSTANT
    CircPosCenter : McPointType := (Pos:=(X:=0,Y:=0,Z:=-1000),
    Orient:=(Angle1:=0,Angle2:=0,Angle3:=0));
    CircPos1 : McPointType := (Pos:=(X:=300,Y:=0,Z:=-1000),
    Orient:=(Angle1:=0,Angle2:=0,Angle3:=0));
END_VAR

PROGRAM _MAIN

    Feedrate(10000); // Move with reduced speed
    MoveAR(qInitialPos); // Move to initial position
    WaitTime(1); // Wait for 1 seconds
    // Perform a square
    MoveJR(VertexPos1); // Move system to Vertex Position 1
    MoveL(VertexPos2); // Move system to Vertex Position 2
    MoveL(VertexPos3); // Move system to Vertex Position 3
    MoveL(VertexPos4); // Move system to Vertex Position 4
    MoveL(VertexPos1); // Move system to Vertex Position 1
    WaitTime(1); // Wait for 1 seconds
    MoveAR(qInitialPos); // Move back to initial position
    WaitTime(1); // Wait for 1 seconds
    // Perform a square with Rounding Edges
    MoveJR(VertexPos1); // Move system to Vertex Position 1
    RoundingEdges(150);
    MoveL(VertexPos2); // Move system to Vertex Position 2
    MoveL(VertexPos3); // Move system to Vertex Position 3
    MoveL(VertexPos4); // Move system to Vertex Position 4
    MoveL(VertexPos1); // Move system to Vertex Position 1
    RoundingEdges(0);

    WaitTime(1); // Wait for 1 seconds
    MoveAR(qInitialPos); // Move back to initial position
    WaitTime(1); // Wait for 1 seconds
    // Circular movement
    MoveJR(CircPos1); // Move system to Vertex Position 1
    CirclePointAbsolute();
    MoveCW(CircPos1, CircPosCenter);
    WaitTime(1); // Wait for 1 seconds
    MoveAR(qInitialPos);

```

```
END_PROGRAM
```

- Exercise 10: PCS programming

```
VAR
```

```
  InitialPos : McAxisTargetType
    := (JointAxis:=[0, 0, 0, 0, 0, 0]);
  Position : McPointType := (Pos:=(X:=0,Y:=0,Z:=50),
    Orient:=(Angle1:=0, Angle2:=90, Angle3:=0));
  PrePosition : McPointType := (Pos:=(X:=0,Y:=0,Z:=150),
    Orient:=(Angle1:=0, Angle2:=90, Angle3:=0));
```

```
END_VAR
```

```
PROGRAM _MAIN
```

```
  Feedrate(2000); // Move with reduced speed
  MoveAR(InitialPos); // Move to initial position
  // Get Ready --> Go to Table Frame PrePosition
  SetPCS(OH::TableFrame); // Set "TableFrame" as PCS
  MoveJR(PrePosition); // Move system to point PrePosition

  (* Pick Orange Cube *)
  SetPCS(OH::OrangeCubeFrame); // Set "OrangeCubeFrame" as PCS
  MoveJR(PrePosition); // Move system to point PrePosition
  MoveL(Position); // Move system to point Position
  WaitTime(0.5); // Simulate gripper closing
  MoveL(PrePosition); // Move system to point PrePosition

  (* Place Orange Cube *)
  SetPCS(OH::OrangeBaseFrame); // Set "OrangeBaseFrame" as PCS
  MoveJR(PrePosition); // Move system to point PrePosition
  MoveL(Position); // Move system to point Position
  WaitTime(0.2); // Simulate gripper opening
  MoveL(PrePosition); // Move system to point PrePosition

  (* Pick Red Cube *)
  SetPCS(OH::RedCubeFrame); // Set "RedCubeFrame" as PCS
  MoveJR(PrePosition); // Move system to point PrePosition
  MoveL(Position); // Move system to point Position
  WaitTime(0.5); // Simulate gripper closing
  MoveL(PrePosition); // Move system to point PrePosition

  (* Pick Red Cube *)
  SetPCS(OH::RedBaseFrame); // Set "RedBaseFrame" as PCS
  MoveJR(PrePosition); // Move system to point PrePosition
  MoveL(Position); // Move system to point Position
  WaitTime(0.2); // Simulate gripper opening
  MoveL(PrePosition); // Move system to point PrePosition
  WaitTime(1); // Wait for 1 second
  MoveAR(InitialPos); // Move back to initial position
```

```
END_PROGRAM
```

- Exercise 12: Tools in the robotics program

```
//Exercise: Tools in the robotics program (ST)
```

```
VAR
```

```
  InitialPos : McAxisTargetType
    := (JointAxis:=[0, 0, 0, 0, 0, 0]);
  Position : McPointType := (Pos:=(X:=0,Y:=0,Z:=50),
    Orient:=(Angle1:=0, Angle2:=90, Angle3:=0));
  PrePosition : McPointType := (Pos:=(X:=0,Y:=0,Z:=150),
    Orient:=(Angle1:=0, Angle2:=90, Angle3:=0));
```

```
END_VAR
```

```
PROGRAM _MAIN
```

```
  Feedrate(2000); // Move with reduced speed
```

```

SetTool(1);
MoveAR(InitialPos); // Move to initial position
// Get Ready --> Go to Table Frame PrePosition
SetPCS(OH::TableFrame); // Set "TableFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition

(* Pick Orange Cube *)
SetPCS(OH::OrangeCubeFrame); // Set "OrangeCubeFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
WaitTime(0.5); // Simulate gripper closing
MoveL(PrePosition); // Move system to point PrePosition

(* Place Orange Cube *)
SetPCS(OH::OrangeBaseFrame); // Set "OrangeBaseFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
WaitTime(0.2); // Simulate gripper opening
MoveL(PrePosition); // Move system to point PrePosition

(* Pick Red Cube *)
SetPCS(OH::RedCubeFrame); // Set "RedCubeFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
WaitTime(0.5); // Simulate gripper closing
MoveL(PrePosition); // Move system to point PrePosition

(* Place Red Cube *)
SetPCS(OH::RedBaseFrame); // Set "RedBaseFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
WaitTime(0.2); // Simulate gripper opening
MoveL(PrePosition); // Move system to point PrePosition
WaitTime(1); // Wait for 1 second
MoveAR(InitialPos); // Move back to initial position

```

END_PROGRAM

- Exercise 13: Dynamic parameters for tools

// Exercise: Dynamic parameters for tools

VAR

```
InitialPos : McAxisTargetType := (JointAxis:=[0, 0, 0, 0]);
```

```
Pos1: McAxisTargetType := (JointAxis:=[10, 15, 23, 10]);
```

```
Pos2 : McAxisTargetType := (JointAxis:=[-15, -25, -10, -10]);
```

END_VAR

PROGRAM _MAIN

```
Feedrate(60000); // Move with reduced speed
```

```
SetTool(2);
```

```
MoveA(InitialPos); // Move to initial position
```

```
WaitTime(0.1);
```

```
MoveA(Pos1);
```

```
WaitTime(0.1);
```

```
MoveA(Pos2);
```

```
WaitTime(0.1);
```

```
MoveA(InitialPos); // Move back to initial position
```

```
WaitTime(0.1);
```

END_PROGRAM

- Exercise 16: Control the gripper

// Exercise: Control the Gripper

VAR

```
InitialPos : McAxisTargetType
```

```

:= (JointAxis:=[0, 0, 0, 0, 0, 0]);
Position : McPointType := (Pos:=(X:=0,Y:=0,Z:=50),
    Orient:=(Angle1:=0, Angle2:=90, Angle3:=0));
PrePosition : McPointType := (Pos:=(X:=0,Y:=0,Z:=150),
    Orient:=(Angle1:=0, Angle2:=90, Angle3:=0));
END_VAR

PROGRAM _MAIN

    Feedrate(2000); // Move with reduced speed
    SetTool(2); // Set Tool vacuum gripper
    MoveAR(InitialPos); // Move to initial position
    // Get Ready --> Go to Table Frame PrePosition
    SetPCS(OH::TableFrame); // Set "TableFrame" as PCS
    MoveJR(PrePosition); // Move system to point PrePosition

    (* Pick Orange Cube *)
    SetPCS(OH::OrangeCubeFrame); // Set "OrangeCubeFrame" as PCS
    MoveJR(PrePosition); // Move system to point PrePosition
    MoveL(Position); // Move system to point Position
    gGripperVacuum := TRUE; // Close Gripper
    WaitTime(0.5); // Simulate gripper closing
    MoveL(PrePosition); // Move system to point PrePosition

    (* Place Orange Cube *)
    SetPCS(OH::OrangeBaseFrame); // Set "OrangeBaseFrame" as PCS
    MoveJR(PrePosition); // Move system to point PrePosition
    MoveL(Position); // Move system to point Position
    gGripperVacuum := FALSE; // Close Gripper
    WaitTime(0.2); // Simulate gripper opening
    MoveL(PrePosition); // Move system to point PrePosition

    (* Pick Red Cube *)
    SetPCS(OH::RedCubeFrame); // Set "RedCubeFrame" as PCS
    MoveJR(PrePosition); // Move system to point PrePosition
    MoveL(Position); // Move system to point Position
    gGripperVacuum := TRUE; // Close Gripper
    WaitTime(0.5); // Simulate gripper closing
    MoveL(PrePosition); // Move system to point PrePosition

    (* Pick Red Cube *)
    SetPCS(OH::RedBaseFrame); // Set "RedBaseFrame" as PCS
    MoveJR(PrePosition); // Move system to point PrePosition
    MoveL(Position); // Move system to point Position
    gGripperVacuum := FALSE; // Close Gripper
    WaitTime(0.2); // Simulate gripper opening
    MoveL(PrePosition); // Move system to point PrePosition
    WaitTime(1); // Wait for 1 second
    MoveAR(InitialPos); // Move back to initial position
END_PROGRAM

```

- **Exercise 17: Sensor feedback**

```

// Exercise: Waiting for sensor feedback
VAR
InitialPos : McAxisTargetType
    := (JointAxis:=[0, 0, 0, 0, 0, 0]);
Position : McPointType := (Pos:=(X:=0,Y:=0,Z:=50),
    Orient:=(Angle1:=0, Angle2:=90, Angle3:=0));
PrePosition : McPointType := (Pos:=(X:=0,Y:=0,Z:=150),
    Orient:=(Angle1:=0, Angle2:=90, Angle3:=0));
END_VAR

PROGRAM _MAIN

    Feedrate(2000); // Move with reduced speed

```



```

SetTool(2); // Set Tool vacuum gripper
MoveAR(InitialPos); // Move to initial position
// Get Ready --> Go to Table Frame PrePosition
SetPCS(OH::TableFrame); // Set "TableFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition

(* Pick Orange Cube *)
SetPCS(OH::OrangeCubeFrame); // Set "OrangeCubeFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
gGripperVacuum := TRUE; // Close Gripper
CYCLIC_PS_BL
IF gGripperVacuumOk THEN
    EXIT; // Wait for sensor feedback
END_IF
END_CYCLIC // Wait for 1 second
MoveL(PrePosition); // Move system to point PrePosition

(* Place Orange Cube *)
SetPCS(OH::OrangeBaseFrame); // Set "OrangeBaseFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
gGripperVacuum := FALSE; // Close Gripper
CYCLIC_PS_BL
IF NOT(gGripperVacuumOk) THEN
    EXIT; // Wait for sensor feedback
END_IF
END_CYCLIC // Wait for 1 second
MoveL(PrePosition); // Move system to point PrePosition

(* Pick Red Cube *)
SetPCS(OH::RedCubeFrame); // Set "RedCubeFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
gGripperVacuum := TRUE; // Close Gripper
WaitTime(0.5); // Simulate gripper closing
MoveL(PrePosition); // Move system to point PrePosition

(* Pick Red Cube *)
SetPCS(OH::RedBaseFrame); // Set "RedBaseFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
gGripperVacuum := FALSE; // Close Gripper
WaitTime(0.2); // Simulate gripper opening
MoveL(PrePosition); // Move system to point PrePosition
WaitTime(1); // Wait for 1 second
MoveAR(InitialPos); // Move back to initial position
END_PROGRAM

```

- Exercise 18: Gripper control with M-functions

```

// Optional exercise: Controlling the gripper using M-functions
VAR
InitialPos : McAxisTargetType
    := (JointAxis:=[0, 0, 0, 0, 0, 0]);
Position : McPointType := (Pos:=(X:=0,Y:=0,Z:=50),
    Orient:=(Angle1:=0, Angle2:=90, Angle3:=0));
PrePosition : McPointType := (Pos:=(X:=0,Y:=0,Z:=150),
    Orient:=(Angle1:=0, Angle2:=90, Angle3:=0));
END_VAR

PROGRAM _MAIN

Feedrate(2000); // Move with reduced speed
SetTool(2); // Set Tool vacuum gripper
MoveAR(InitialPos); // Move to initial position

```



```

// Get Ready --> Go to Table Frame PrePosition
SetPCS(OH::TableFrame); // Set "TableFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition

(* Pick Orange Cube *)
SetPCS(OH::OrangeCubeFrame); // Set "OrangeCubeFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
SetM(40);
WaitTime(0.5); // Simulate gripper closing
MoveL(PrePosition); // Move system to point PrePosition

(* Place Orange Cube *)
SetPCS(OH::OrangeBaseFrame); // Set "OrangeBaseFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
SetM(41);
WaitTime(0.2); // Simulate gripper opening
MoveL(PrePosition); // Move system to point PrePosition
MoveL(PrePosition); // Move system to point PrePosition

(* Pick Red Cube *)
SetPCS(OH::RedCubeFrame); // Set "RedCubeFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
gGripperVacuum := TRUE; // Close Gripper
WaitTime(0.5); // Simulate gripper closing
MoveL(PrePosition); // Move system to point PrePosition

(* Pick Red Cube *)
SetPCS(OH::RedBaseFrame); // Set "RedBaseFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
gGripperVacuum := FALSE; // Close Gripper
WaitTime(0.2); // Simulate gripper opening
MoveL(PrePosition); // Move system to point PrePosition
WaitTime(1); // Wait for 1 second
MoveAR(InitialPos); // Move back to initial position
END_PROGRAM

```

- Exercise 19: Gripper control with signaling

```

// Optional exercise: Controlling the gripper using signaling
VAR
InitialPos : McAxisTargetType
    := (JointAxis:= [0, 0, 0, 0, 0, 0]);
Position : McPointType := (Pos:= (X:=0, Y:=0, Z:=50),
    Orient:= (Angle1:=0, Angle2:=90, Angle3:=0));
PrePosition : McPointType := (Pos:= (X:=0, Y:=0, Z:=150),
    Orient:= (Angle1:=0, Angle2:=90, Angle3:=0));
END_VAR

PROGRAM _MAIN

Feedrate(2000); // Move with reduced speed
SetTool(2); // Set Tool vacuum gripper
MoveAR(InitialPos); // Move to initial position
// Get Ready --> Go to Table Frame PrePosition
SetPCS(OH::TableFrame); // Set "TableFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition

(* Pick Orange Cube *)
SetPCS(OH::OrangeCubeFrame); // Set "OrangeCubeFrame" as PCS
SignalPrediction();
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position

```

```
SignalDistance(50, ConfiguredSignal:=VacuumOn);
MoveL(PrePosition); // Move system to point PrePosition

(* Place Orange Cube *)
SetPCS(OH::OrangeBaseFrame); // Set "OrangeBaseFrame" as PCS
SignalPrediction();
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
SignalDistance(50, ConfiguredSignal:=VacuumOff);
MoveL(PrePosition); // Move system to point PrePosition

(* Pick Red Cube *)
SetPCS(OH::RedCubeFrame); // Set "RedCubeFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
gGripperVacuum := TRUE; // Close Gripper
WaitTime(0.5); // Simulate gripper closing
MoveL(PrePosition); // Move system to point PrePosition

(* Pick Red Cube *)
SetPCS(OH::RedBaseFrame); // Set "RedBaseFrame" as PCS
MoveJR(PrePosition); // Move system to point PrePosition
MoveL(Position); // Move system to point Position
gGripperVacuum := FALSE; // Close Gripper
WaitTime(0.2); // Simulate gripper opening
MoveL(PrePosition); // Move system to point PrePosition
WaitTime(1); // Wait for 1 second
MoveAR(InitialPos); // Move back to initial position
END_PROGRAM
```




B&R
Industrial Automation GmbH
A member of the ABB Group
B&R Straße 1
5142 Eggelsberg, Austria
office@br-automation.com

t +43 7748 6586-0
f +43 7748 6586-26

br-automation.com

